机器学习

目录

1　人工智能与机器学习全景　1

2　Python 语言基础　23

3　Python 程序组织与调试　57

4　NumPy 与向量化计算　82

5　pandas、数据探索与可视化　109

6　线性代数复习　126

1 人工智能与机器学习全景

先修知识、学习目标与本章位置

本章只要求理解函数、平均数和基本代数运算，不要求编程经验。学习后应能区分人工智能的研究范围与不同学习方式；把一个需求改写为输入、目标和评价协议；辨认样本、特征、标签、参数、超参数、损失和指标；解释训练、验证、测试各自承担的工作。

本章给出全书的共同语言。实验先按文字说明和手算理解，代码可在第二章学完函数、列表和循环后回读。近邻方法只作为流程示例，完整算法将在第 15 章讨论。

从一张图片判断物体类别，从一段历史记录预测下一次需求，从一批文档找出相关证据，以及控制一个不断与环境交互的系统，都是人工智能中的典型问题。这些问题的输入、目标和错误代价不同，不能只用“让计算机变聪明”概括。建立机器学习问题的第一步是明确计算机接收什么信息、产生什么输出，以及怎样判断这个输出是否有用。

本章使用两个贯穿例子：根据包裹重量预测配送费用，以及根据一个评分判断消息是否需要进一步检查。所用数值都是为解释概念而构造的教学数据，不代表真实业务规律。第一个例子展示连续预测和数据划分，第二个例子展示阈值与错误类型。

1.1 人工智能、机器学习与深度学习

研究目标与实现方法

人工智能（artificial intelligence，AI）研究能够完成感知、推理、学习、规划、语言处理和决策等任务的计算方法。它包含多种技术路线。搜索算法可以在候选状态之间寻找路径；规则系统可以把已有知识写成条件和结论；机器学习可以利用数据改变模型的行为。一套系统还可以把这些方法组合起来。

设某配送公司已经公布了完整收费规则：起步价为 4 元，每千克增加 2 元。对重量 x，直接计算 $4 + 2x$ 即可。这里的规则由人给定，无须从数据中估计。若收费还受到拥堵、区域和服务类型影响，无法写出足够准确的固定规则，就可能利用历史记录拟合预测函数。是否采用学习方法，取决于规律是否已知、数据能否获得、误差是否可接受，而不取决于任务名称是否带有“智能”。

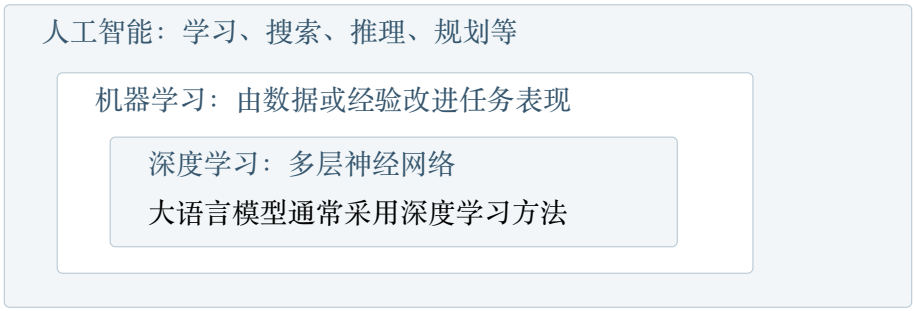
定义：机器学习

机器学习（machine learning，ML）研究利用数据或交互经验改进任务表现的方法。一个学习问题至少要交代任务、经验来源和表现标准。改进必须在规定的评价条件下成立，不能只表现为对已见数据记得更准确。

学习过程并不意味着程序自行确定一切。数据来自哪里、目标如何定义、模型允许多复杂、训练何时停止，通常需要设计者选择。训练算法则在这些约束下，利用数据确定模型中的可调整部分。例如模型形式选为直线以后，程序可以根据历史数据确

定直线的斜率和截距。

深度学习（deep learning，DL）是机器学习中的一类方法，以多层神经网络逐级变换输入表示。以图片为例，输入最初是一组像素数值，网络通过多层计算形成适合任务的表示，最后输出类别得分。层数较多并不自动带来可靠性；数据、训练目标、优化方法和评价协议仍然必要。神经网络也不是所有机器学习模型的统称，决策树、近邻法和线性模型都有各自的表达方式。



该图表示常见研究方法的包含关系，不是系统部件的排他划分。一个语言问答系统可以使用深度网络生成文本，使用传统检索查找文档，再使用人工编写的规则检查输出格式。这些部件并不因为处于同一个产品中就成为同一种算法。

大语言模型的基本位置

语言模型（language model）为文本序列建立概率模型。文本通常先被拆成**词元**（token），一个词元可能对应一个字、一个词或一个子词片段，不能固定等同于一个汉字。**大语言模型**（large language model，LLM）通常使用规模较大的深度神经网络，并在大量文本上进行训练。参数规模、训练数据规模和计算量都影响模型，不能仅凭“大”字推出某一具体能力。

以自回归语言模型为例，已知词元序列 $t_1, \dots, t_{j-1}$，模型对下一个词元 t_j 给出条件概率

$$p_\theta(t_j \mid t_1, \dots, t_{j-1}).$$

下标 θ 表示模型参数，竖线表示“在前面这些词元给定的条件下”。这里暂不推导概率公式，只需理解：模型先给候选词元分配概率，再按照某种规则选择后续词元，逐步形成文本。输入上下文相同，选择规则不同，生成结果也可能不同。

语言模型的生成目标不直接等同于事实核对。一个句子可能形式完整、语义连贯，却包含错误日期或不存在的文献。因此面向资料问答的系统往往还需要检索、证据组织和引用验证。训练一个模型、调用一个现成模型，以及搭建包含模型的应用系统，是三种不同层次的工作。后续第 29–37 章分别讨论结构、训练、生成和应用评价。

> **注意：任务、模型与系统**
>
> “文本问答”是任务，“Transformer”是一类网络结构，“语言模型”描述建模目标，“检索增强生成”是一种系统组织方式。分析技术方案时，应分别说明这些层次，避免用一个名称替代整个流程。

1.2 学习方式：监督、无监督、自监督与强化学习

学习方式的区别主要来自训练信号如何获得，而不是输入必须是表格、图片还是文字。同一种数据形式可以支持多种学习任务，一套系统也可能先使用一种方式预训练，再使用另一种方式适配具体任务。

监督学习：给定输入与目标的对应关系

监督学习（supervised learning）使用带有目标值的样本。第 i 个样本记为 $(\mathbf{x}_i, y_i)$，其中 $\mathbf{x}_i$ 是输入，y_i 是期望预测的目标。例如，包裹重量和区域是输入，最终配送费用是目标；邮件正文是输入，是否为垃圾邮件是目标。

“有标签”只表示存在训练目标，不保证标签准确。标签可以来自人工标注、传感器测量或后续业务记录。人工判断可能不一致，设备可能存在测量误差，业务结果也可能被历史决策影响。若使用“审核是否通过”作为目标，模型学到的首先是历史审核结果的规律，而不是某种不受流程影响的客观属性。

监督学习的关键是输入与标签的对齐。如果一张图片对应了另一张图片的类别，即使每个字段看起来都合法，训练问题仍然错误。后续数据处理中的合并、排序和切片，都必须保留这种对应关系。

无监督学习：没有预先指定的逐样本标签

无监督学习（unsupervised learning）从输入数据中寻找结构，而没有为每个样本提供任务所需的目标标签。把用户按行为相似性分组是聚类，把高维数据压缩到较低维表示是降维。输入之间的距离、重建误差或密度可以形成优化目标。

无监督并不等于没有目标函数，也不等于不需要人作判断。聚类算法可能把一批用户分成三组，但“第三组”本身不是天然存在的业务类别。要解释这些组，仍需检查特征、稳定性和用途。若对不同特征采用不同尺度，得到的相似关系也可能改变。因此一个结构清晰的可视化图并不能独立证明分组有实际意义。

自监督学习：由数据构造训练目标

自监督学习（self-supervised learning）通过数据本身构造训练信号。对一句文本，可以把前半段作为输入，后续词元作为目标；对一张图片，可以遮挡一部分内容并要求模型预测被遮挡部分。目标来自原数据，不需要人为逐条标注新的类别。

自监督训练仍然存在明确的输入、目标和损失。从计算形式看，它常使用与监督学习相似的优化流程；从训练信号的来源看，又与人工标注监督不同。因而不能把监督、无监督和自监督画成在所有定义下都互不相交的三个集合。此处按训练信号来源区分，是为了说明数据如何被使用。

例题 1.1：同一批文本的三种用途

已有一批未标注新闻，另有其中 2000 篇新闻的主题标签。设计三个不同的训练问题，并说明目标从哪里来。

解　第一种是监督分类：输入完整新闻，目标为人工标注的“体育”“科技”等主题，只有带标签的部分直接提供分类监督。

第二种是无监督聚类：输入新闻的某种数值表示，让内容相似的新闻靠近或进入同一组，没有预先要求某组对应哪个主题。聚类结果可以再由人检查和命名。

第三种是自监督语言建模：从原文本截取前面的词元作为输入，将紧接其后的词元作为目标。未标注新闻也能生成这样的训练样本。若随后使用 2000 篇带标签新闻训练分类器，就形成“自监督预训练加监督适配”的两阶段方案。同一篇新闻在不同阶段的角色不同。

强化学习：动作改变后续经验

强化学习（reinforcement learning，RL）研究与环境交互的决策。**智能体**（agent）观察当前信息，选择**动作**（action），环境返回新的信息和**奖励**（reward）。选择动作的规则称为**策略**（policy）。目标通常是提高一段时间内的累计奖励，而不是只预测当前样本的一个标签。

以网格中的路径规划为例，位置是状态，向上下左右移动是动作，到达终点可能获得正奖励，碰到障碍可能受到惩罚。一步绕路可能降低即时奖励，却有助于最终到达目标。这体现了延迟结果和连续决策。为了发现更好的路线，智能体有时必须尝试尚不熟悉的动作，这称为探索；使用目前认为最好的动作则称为利用。

如果只有历史用户记录和一个购买标签，通常首先是监督预测问题。若系统选择展示哪件商品，而这个选择会影响用户后续行为和可观察的数据，就出现交互决策因素。不能仅因系统“做了选择”就断言必须使用强化学习；固定规则、监督模型加阈值，以及搜索规划也可以产生决策。

方式	训练信号	首先需要检查的条件
监督	输入对应的目标值	标签定义、正确性与对齐
无监督	数据结构形成的目标	相似性、尺度与结果解释
自监督	从原数据构造的目标	目标是否泄漏、任务是否有用
强化	交互产生的奖励与转移	奖励含义、延迟与探索条件

1.3　任务类型：回归、分类、生成、检索与决策

回归与分类

回归（regression）预测数值型目标，通常允许输出在某个连续范围内变化。预测费用、温度和时长都是回归。目标虽然写成数字，也不一定是回归：邮政编码、班级编号和类别编号的大小一般没有可用于误差计算的连续意义。

分类（classification）预测离散类别。两种类别构成二分类，多于两种互斥类别构成多分类。一个样本允许同时属于多个类别时，是多标签问题，例如一张图片同时含

有“树”和“汽车”。多分类通常要求从多个类别中选一个，多标签则需要分别判断各标签是否存在。

分类模型可以先输出得分或概率估计，再通过规则生成类别。设二分类的正类是“需要检查”，得分 $s(\mathbf{x}) \in [0,1]$，选择阈值 τ 后可定义

$$\hat{y} = \begin{cases} 1, & s(\mathbf{x}) \geq \tau, \\ 0, & s(\mathbf{x}) < \tau. \end{cases}$$

符号 $\hat{y}$ 读作“预测的 y”。阈值是由分数得到行动的规则，并不等于模型内部的所有参数。只有在概率估计经过适当验证时，0.8 才适宜解释为相应条件下约 80% 的发生概率；任意归一化得分不能自动获得这种含义。

生成与检索

生成（generation）产生新的文本、图片、音频或其他结构化对象。输出可能很长，也可能存在多个合理答案。例如对一段文本进行摘要，不能简单要求输出与某一参考摘要逐字相同；评价还需要考察信息是否保留、事实是否一致和内容是否适合任务。

检索（retrieval）从已有集合中找出与查询相关的对象。输入为查询，输出为文档、图片或候选条目的排序。检索输出应当来自待检索集合，而生成输出可以是新组织的内容。检索质量既取决于相关性判断，也取决于候选集合是否包含所需资料。

一个资料问答系统可能先检索若干段落，再根据段落生成答案。此时应分别检查：相关材料有没有被找到；材料是否包含问题答案；生成内容是否忠实于材料；引用是否对应支持该结论的段落。只检查回答是否流畅，无法定位这些不同环节的失败。

预测与决策的联系

决策（decision）是在目标与约束下选择行动。预测可以为决策提供信息，但预测最准确的方案不一定产生最低的业务成本。设需要检查的消息只有 10%，人工检查每条消息都有成本，漏掉危险消息又有另一种成本。怎样选择阈值，取决于这两种错误的相对代价和检查容量。

同一个模型也可能在不同用途下采用不同阈值。如果只是给审核人员排序，重点是靠前条目的相关性；如果直接阻断消息，误判普通消息的代价可能更高。任务说明必须交代模型输出会如何被使用，才能选择适当指标。

例题 1.2：把含糊需求改写成任务

需求为“建立智能配送系统”。给出一个可执行的预测子任务和一个决策子任务，并说明两者的区别。

解 预测子任务可以规定为：在订单被接受时，根据当时已知的重量、区域和服务类型，预测最终配送费用，单位为元；评价新订单上的平均绝对误差。目标明确到一个数值，输入也限定在预测时刻可获取的字段。

决策子任务可以规定为：对当天已确认订单选择配送车辆和路径，在车辆容量、时间窗和可行道路约束下减少总成本。其输出是方案，评价涉及实际成本与约束是否满

足。费用预测可以作为决策中的成本估计，但它不能单独解决路径的组合选择问题。

若不限定预测时点，容易把订单完成后才出现的“实际里程”作为输入，从而得到无法在订单接受时运行的模型。问题定义中的时间条件因此与算法选择同样重要。

1.4 样本、特征、标签与数据表示

逐个样本的记号

样本（sample）是数据中的一个观测单位。订单预测中通常一笔订单是一个样本；图像分类中通常一张图像是一个样本。**特征**（feature）是用于描述样本、供模型使用的输入信息。**标签**（label）或目标（target）是监督学习希望预测的量。

对 n 个样本、每个样本 d 个数值特征，常记

$$\mathbf{x}_i = (x_{i1}, \dots, x_{id})^{\mathrm{T}} \in \mathbb{R}^d, \qquad i = 1, \dots, n.$$

$\mathbb{R}^d$ 表示 d 维实数向量的集合，上标 T 表示转置。下标 i 选第几个样本，j 选第几个特征，x_{ij} 因而是第 i 个样本的第 j 个输入数值。维度 d 不表示样本数量。

把样本按行排列，得到**设计矩阵**或特征矩阵

$$X = \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix} \in \mathbb{R}^{n \times d}.$$

本讲义默认行对应样本、列对应特征。回归的目标向量写作 $\mathbf{y} = (y_1, \dots, y_n)^{\mathrm{T}} \in \mathbb{R}^n$。具体程序库可能要求目标形状为 $(n,)$ 或 $(n, 1)$，数学上的同一组数值在程序中仍需检查形状，第 4 章将详细讨论。

订单编号	重量/kg	距离/km	费用/元	字段角色
A01	1.0	3.0	8.0	前两项为特征，费用为标签
A02	2.0	5.0	12.0	编号用于追踪，不默认当特征
A03	1.5	2.0	7.0	每一行是一笔订单

这里 $n = 3, d = 2$，X 的形状为 3×2。编号虽然存放在表格中，却不是自然有预测意义的数值。把 A01 转换成 1、A02 转换成 2，不会自动建立合理的大小关系。类别、文本和图片也可以变成输入表示，但转换规则本身需要设计和验证。

观测单位、重复与缺失

一名用户有十笔订单，意味着十个订单样本，却不意味着十个完全独立的用户。若模型需要预测新用户的行为，同一用户的记录跨入训练和测试可能使测试变得过于容易。若目标是预测既有用户的下一笔订单，按时间评价又可能更合适。数据划分必须与使用情境一致。

缺失值表示某字段未被观测或未被记录，不应未经说明地当成 0。重量为 0 和重量缺失具有不同含义。重复记录也有多种情况：数据导出时重复了一行、同一订单更新了状态、两笔真实订单恰好字段相同。只有理解样本单位，才能决定去重规则。

特征越多也不必然越好。无关字段增加复杂性，有些字段在部署时无法取得，有些字段会把目标的后续结果直接暴露给模型。检查字段的定义、生成时间和可靠性，是模型训练之前的必要工作。

例题 1.3：输入形状与信息可用性

有 1000 笔订单，每笔记录重量、收件区域、下单时刻、最终签收时刻和最终费用。模型在下单时预测费用。哪些字段可以作为候选输入？能否直接说 $d = 4$？

解 重量、收件区域和下单时刻在预测时已知，可以作为候选输入；最终签收时刻只有配送完成后才知道，不能用于该预测情境。最终费用是标签。

原始候选字段有三个，但处理后的数值特征维度未必是 3。区域可能需要多个指示变量，下单时刻可能分解为小时与是否休息日。因此必须区分原始字段数和输入向量维度。$n = 1000$ 是样本数，d 应在具体编码方案确定后再给出。

1.5 模型、参数、超参数与训练

模型是输入到输出的映射

模型（model）用某种规则把输入映射为输出。参数化模型可记为

$$\hat{y} = f_\theta(\mathbf{x}).$$

f 表示函数形式，θ 表示可调整的**参数**（parameter）。对只有重量一个特征的线性预测，

$$\hat{y} = ax + b, \qquad \theta = (a, b).$$

x 的单位是千克，$\hat{y}, b$ 的单位是元，a 的单位是元每千克。函数形式限制预测曲线是一条直线，参数决定具体是哪条直线。这种限制是模型假设，不是数据自然保证的事实。

训练（training）是根据训练数据确定模型可调整部分的过程。训练结束后，对新输入计算输出称为**推理**（inference）或预测。普通推理不需要重新计算全部训练过程；但不同算法保存的信息不同。线性模型可以只保存系数，近邻模型通常需要保存用于查询的训练样本。

模型并不必然通过梯度下降训练。树模型可以选择分裂规则，近邻法可以存储样本并在预测时寻找邻近点。有些学习方法只有少量显式系数，有些方法保存复杂结构。因而“训练就是不断修改神经网络权重”只适用于其中一类方法。

损失、经验风险与优化目标

损失函数（loss function）量化一次预测的错误，例如平方损失

$$\ell(\hat{y}, y) = (\hat{y} - y)^2.$$

本书将 $\hat{y}-y$ 称为**有符号预测误差**，正值表示高估，负值表示低估。统计学中**残差**（residual）通常采用相反方向 $y-\hat{y}$；读取其他资料时应先核对约定。二者取绝对值或平方后的结果相同，因而不影响本书的 MAE 与 MSE。平方后两种方向都产生非负代价，大误差增长更快。若费用用元计，平方损失的单位为元的平方，不能直接把它解释为平均多收了多少元。

把损失在训练样本上平均，得到**经验风险**（empirical risk）

$$\widehat{R}_{\text{train}}(\theta)=\frac{1}{n}\sum_{i=1}^{n}\ell(f_\theta(\mathbf{x}_i),y_i). \tag{1.1}$$

求和符号表示逐个样本累加；n 为参与该次平均的样本数；每项损失为一个标量，因此结果也是标量。训练算法通常试图降低这个目标，或降低加入复杂度惩罚后的目标。

优化（optimization）研究怎样寻找使目标函数较小的参数。**学习**除了优化，还包括模型与数据能否支持对新样本的预测。训练损失很低但新样本表现差，可能是泛化问题；训练损失始终很高，则也可能是模型限制、数据错误或优化失败。两种现象需要不同的诊断。

> **例题 1.4：平方误差的完整计算**
>
> 三笔订单的重量为 $(1,2,3)$，真实费用为 $(6,8,9)$。模型为 $\hat{y}=2x+4$。计算预测、逐样本误差、平均平方损失和平均绝对误差。

解 依次代入输入，得到预测 $(6,8,10)$。预测减真实值为 $(0,0,1)$，说明前两项恰好吻合，第三项高估 1 元。平方损失为 $(0,0,1)$，因而

$$\text{MSE}=\frac{0^2+0^2+1^2}{3}=\frac{1}{3}\ \text{元}^2,\qquad \text{MAE}=\frac{|0|+|0|+|1|}{3}=\frac{1}{3}\ \text{元}.$$

数值恰好相同是该数据的偶然结果，二者单位和对大误差的敏感程度不同。如果第三项误差改成 3 元，MSE 变成 3 元 ^{2}，MAE 变成 1 元。

超参数与选择规则

超参数（hyperparameter）是训练流程或模型结构中需要预先设定、通常由验证方案选择的设置，例如近邻法的邻居个数 k、树允许的最大深度、梯度下降的学习率。参数由某次训练决定，超参数则在不同训练方案之间进行选择。这是相对于具体训练流程的区分，不能仅通过某个数的名字判断。

例如，直线的 a,b 在通常线性回归中是参数；若人为规定几种固定斜率、只让程序估计截距，则斜率已成为候选方案的一部分。应按它在当前流程中是否被拟合来解释。另一个常见误区是把“训练样本数”与“迭代次数”混淆：前者来自数据，后者描述训练算法重复计算的次数。

概念	线性预测中的例子	作用
模型形式	$ax + b$	限制输入到输出的函数族
参数	训练得到的 a, b	决定具体预测函数
超参数	正则化强度等设置	决定训练或结构方案
训练算法	求解最小二乘等方法	寻找符合目标的参数
指标	测试 MAE	概括指定数据上的表现

表中的**正则化**（regularization）是通过惩罚或限制模型复杂性约束拟合的方法；强度控制这种约束的程度，第 9 章和第 11 章再给出具体目标与推导。“把指标作为训练目标”在某些任务中可行，但二者角色仍不同。训练目标强调可优化性，评价指标强调任务用途。分类中的某些平滑损失有利于训练，最终却可能以准确率、召回率或其他指标比较模型。

1.6 怎样从实际需求定义学习问题

任务说明的六个要素

一个可执行的任务说明应包含样本单位、预测时点、输入范围、目标定义、评价总体和错误代价。下面逐项解释。

样本单位决定一行数据代表什么。一行是“订单”还是“订单中的一个包裹”，会改变数据量、标签和分组关系。预测时点决定可用信息，例如下单时预测费用与配送完成后检查账单，是两个不同任务。

输入范围说明哪些字段可读，哪些字段必须屏蔽。目标定义要给出测量方法、单位和时间范围。预测“未来销量”必须说明是下一小时、下一天还是下一月，是否包含取消订单。评价总体说明需要对哪些新数据有效：相同城市的后续订单、新城市订单，或全新用户。错误代价则把统计误差与业务使用联系起来。

任务卡：下单时费用预测

观测单位是一笔已接受订单。预测在下单确认后、开始配送前进行。候选输入为当时已知的重量、区域、服务类型和下单时间；目标为订单最终结算费用，单位为元。评价对象为同一服务范围内后续新增订单。主要报告 MAE，同时检查高费用订单的误差；先与只预测训练平均费用的基线比较。

该任务卡还没有说明算法，但已经排除了许多不合理做法。如果未来应用是进入新城市，必须增加城市之间的评价；如果某种服务只出现在测试期，需讨论模型是否有处理它的表示。问题定义往往比模型名称更早决定实验是否可信。

基线、数据来源与标签构造

基线（baseline）是作为比较起点的简单方案，例如回归中始终预测训练平均值，分类中始终预测多数类，检索中使用简单关键词匹配。基线可以暴露数据难度和指标陷阱。复杂模型如果没有优于合适基线，首先应检查它是否带来实际价值。

只预测训练均值的方案记为 $\hat{y} = c$，其中

$$c = \frac{1}{n}\sum_{i=1}^{n} y_i.$$

这个常数由训练数据确定，不能使用测试标签计算。平均值是平方损失下的最优常数：记训练均值为 $\bar{y}$，对任意常数 c，展开平方可得

$$\begin{aligned}\sum_i (y_i - c)^2 &= \sum_i \big[(y_i - \bar{y}) + (\bar{y} - c)\big]^2 \\ &= \sum_i (y_i - \bar{y})^2 + 2(\bar{y} - c)\sum_i (y_i - \bar{y}) + n(\bar{y} - c)^2 \\ &= \sum_i (y_i - \bar{y})^2 + n(\bar{y} - c)^2.\end{aligned}$$

中间和为零，因为 $\sum_i y_i = n\bar{y}$；最后一项非负，取 $c = \bar{y}$ 时为零。如果目标改成绝对损失，最优常数一般为训练标签的中位数，不能沿用上述平方展开作证明。本章实验将均值作为固定基线，同时用 MAE 评价；这是一种明确的比较约定，不宣称它在 MAE 下最优。

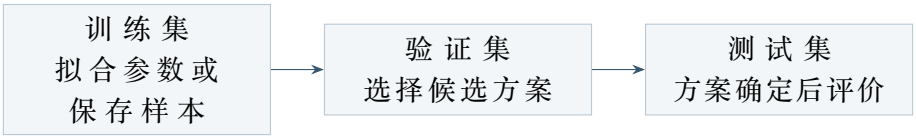
数据来源需要记录采集条件。人工筛选出的“正常订单”可能遗漏异常场景；只记录已经完成的订单可能排除取消订单；历史审核数据可能偏向曾经被检查的样本。清洗不能简单等同于删除难例，因为这些难例可能恰好是模型实际会遇到的情况。

标签还要说明延迟。订单是否最终送达、用户是否在 30 天内购买，只有等待相应时间之后才能确定。把尚未观察完整的记录一律标成“未发生”，会混入错误标签。应按数据截止时间和观察窗口处理。

1.7 训练、验证、测试与泛化

三类数据的职责

训练集（training set）用于拟合模型可调整部分。**验证集**（validation set）用于选择候选模型、超参数和决策阈值。**测试集**（test set）用于在方案确定后，给出对新数据表现的最终评估。三者的区别是用途，通常不能只靠文件名称保证。



例如比较近邻个数 1、3 和 5，三个候选都只使用训练样本预测，在验证集上比较 MAE。选定 k 后再计算测试 MAE。如果观察测试结果后继续更换 k，测试集已经参与选择，它的分数不再是一次未参与选择的独立最终评价。

最终方案有时会在训练集与验证集合并后重新拟合，再在保留的测试集上评价。这需要在协议中预先规定，尤其要说明合并后重新拟合的具体内容。本章实验为避免混淆，选定 k 后保持原训练集，不做合并重训。

泛化、欠拟合与过拟合

泛化（generalization）指模型对未参与相应拟合或选择的新样本的表现。训练集只是一批有限观测，不能列举未来所有输入。理想目标是降低目标使用环境中的平均错误，而不只是降低训练数据上的平均错误。

欠拟合（underfitting）表示模型或训练过程未能捕捉数据中与任务有关的规律，常表现为训练与验证都不好。**过拟合**（overfitting）表示模型对训练数据中的偶然细节适应过强，新数据表现较差。训练好、验证差是需要检查的信号，但也可能来自错误划分、分布变化或实现不一致，不能只根据两行分数断言原因。

一个极端模型可以记住每个训练编号及其费用。如果预测任务中的新订单有新编号，记忆便无法直接提供有用预测。训练误差为零只说明存储或拟合了已见数据。具有较强表达能力的模型需要足够数据、合理限制和可靠评价，才能判断其优势。

从统计观点，常假设训练和目标新样本来自同一分布，并且样本之间具有适当独立性。该假设有助于把有限样本表现与总体表现联系起来，但现实中可能不满足。费率调整、城市改变、用户行为变化，都可能引起**分布变化**（distribution shift）。遇到这种变化，即使先前评价完全合规，模型也可能失效。

划分方式应模拟实际使用

随机划分适合样本近似独立、未来使用条件与当前样本混合分布一致的情况。时间划分用较早记录训练、较晚记录评价，更接近预测未来。分组划分让同一用户、设备或病人的记录只进入一个集合，用于评价对新组的泛化。分类中分层划分可尽量保留各集合的类别比例，但它不能替代时间或分组约束。

教学中常见的比例不是普遍规则。数据少时，固定留出集的分数可能波动很大；测试集中少数类过少时，召回率可能缺乏稳定性。第 14 章将使用交叉验证和不确定性分析处理这些问题。本章重点在于先决定各集合的用途，而不是背诵一个固定百分比。

> **注意：数据泄漏**
>
> **数据泄漏**（data leakage）指拟合或选择过程使用了在规定预测情境中不应获得的信息。包括目标产生后的字段、跨集合的重复或关联记录，以及用全部数据估计预处理规则。泄漏使评价条件偏离实际使用条件，分数可能因此过于乐观。

即使标准化没有读取标签，也不应在常规独立测试协议下用测试数据估计均值和方差。它仍然让训练过程利用了测试输入的分布信息。通常先划分，再用训练集拟合预处理规则，最后将同一规则应用到验证和测试。具体算法和特殊协议可能不同，必须明确说明。

例题 1.5：一次高分评价的排查

某模型训练准确率 99%，测试准确率 98%。训练与测试按照片随机划分，其中同一个人的连续拍摄照片出现在两边。应用目标是识别从未出现过的新人的某种表情。这个分数能否直接证明对新人有效？

解 不能。连续照片共享人物外观、背景和拍摄条件，样本之间有明显关联。照片级随机划分可能让模型在测试时遇到与训练非常接近的输入，而应用要求对新人物有效。

应按人物分组，将某个人的照片全部放在同一集合，重新训练与评价。若需要在新场景使用，还应考虑拍摄场景是否也跨集合。99% 与 98% 的接近只能说明当前划分下两组表现接近，无法消除划分与实际目标不一致的问题。

1.8 评价指标与错误的含义

回归指标：MAE、MSE 与 RMSE

对评价集中的 m 个样本，真实值为 y_i，预测为 $\hat{y}_i$，常用

$$\mathrm{MAE} = \frac{1}{m}\sum_{i=1}^{m}|\hat{y}_i - y_i|, \tag{1.2}$$

$$\mathrm{MSE} = \frac{1}{m}\sum_{i=1}^{m}(\hat{y}_i - y_i)^2, \tag{1.3}$$

$$\mathrm{RMSE} = \sqrt{\mathrm{MSE}}. \tag{1.4}$$

MAE 是平均绝对误差，MSE 是均方误差，RMSE 是均方根误差。这里 m 专指当前评价集合的大小，避免与训练样本数混淆。MAE 和 RMSE 具有与目标相同的单位，MSE 具有目标单位的平方。三者都越小越好，但比较时必须使用相同样本、目标单位和计算约定。

如果误差为 $(0, 0, 0, 4)$，MAE 为 1，RMSE 为 2。若误差为 $(1, 1, 1, 1)$，两者都为 1。MAE 在这两个方案间打平，RMSE 则更偏好均匀的小误差，因为平方对单个大误差施加更强代价。选择指标因此隐含了对错误分布的偏好。

只看平均值还可能掩盖子群体差异。例如整体 MAE 较小，但远距离订单误差很大，部署到以远距离订单为主的区域仍会失败。应结合分组结果和具体错例；分组指标样本少时，也要同时报告样本数，不能将偶然变化解释为稳定规律。

分类指标：先数清四种结果

二分类中先规定正类，例如“需要检查”。**真阳性**（true positive，TP）是实际正类且预测正类；**假阳性**（false positive，FP）是实际负类但预测正类；**真阴性**（true negative，TN）是实际负类且预测负类；**假阴性**（false negative，FN）是实际正类但预测负类。

	预测正类	预测负类
实际正类	TP	FN
实际负类	FP	TN

这个表称为**混淆矩阵**（confusion matrix）。不同资料可能把实际与预测放在相反轴上，因此读取矩阵时必须先看轴标，不能只凭位置认定哪一格是 FP。

准确率（accuracy）、**精确率**（precision）和**召回率**（recall）定义为

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{FP} + \text{TN} + \text{FN}}, \tag{1.5}$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \tag{1.6}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}. \tag{1.7}$$

准确率回答“所有判断中有多少正确”；精确率回答“判为正的样本中有多少确实为正”；召回率回答“实际为正的样本中找出了多少”。精确率与召回率的分母不同，不能互换。

若模型从不预测正类，精确率分母为零，数学上未定义；若评价集中没有实际正类，召回率分母为零。程序必须明确处理约定，例如返回缺失标记并报告原因。不能悄悄把未定义值当成真实的 0 或 1，然后与正常指标混合比较。

例题 1.6：多数类与准确率陷阱

100 条消息中，实际有 5 条需要检查。方案 A 把全部消息判为不需要检查；方案 B 找出其中 4 条，但同时错报 10 条普通消息。比较准确率、精确率与召回率。

解 方案 A 的 TP 为 0、FP 为 0、TN 为 95、FN 为 5，因此准确率为 95%，精确率未定义，召回率为 0。高准确率来自负类占多数，不能证明模型找到了需要检查的消息。

方案 B 的 TP 为 4、FP 为 10、TN 为 85、FN 为 1，准确率为 $(4 + 85)/100 = 89\%$，精确率为 $4/14 \approx 28.57\%$，召回率为 $4/5 = 80\%$。方案 B 总体准确率较低，却覆盖了大部分正类，同时带来更多人工检查负担。是否有用，需结合漏检与误报成本判断。

阈值不是模型质量的唯一刻度

提高正类阈值通常减少被判为正的样本，因此 TP 与 FP 都不会增加，召回率通常下降。精确率却不保证单调提高：如果被删除的样本中真阳性的比例高于保留部分，精确率也可能下降。它的实际变化取决于得分排序和标签。

若要求每天最多检查 100 条消息，就还需要考虑候选数量和排序。若要求漏检率不高于某个水平，应根据验证数据选择阈值，并在未参与选择的数据上核对。阈值的选择属于方案选择，不能在测试集上反复尝试后仍把最好结果当成独立最终评价。

本章只讨论三个基本分类指标。F1、ROC、PR 曲线、校准和区间估计会在第 14 章展开。现阶段的重点是能把指标的每一个分母与具体样本集合对应起来。

1.9 完整小案例：从数据到最终评价

任务、教学数据与比较方案

假设需要在包裹进入配送系统时，仅根据重量预测费用。简化输入 x 的单位为 kg，输出 y 的单位为元。教学数据在约 0.5–6.0 kg 范围内构造，费用大致随重量增加，同时含有小幅偏差。真实费用还受到许多因素影响，本例刻意只保留一个特征，以便手算和逐步追踪。

本例预先固定训练 12 条、验证 4 条、测试 4 条。它不是从真实总体随机抽样得到，因此只能展示协议和计算，不能据此估计真实系统的预期误差。后续项目中必须根据实际采集方式、时间和分组关系建立划分。

训练集：前半		训练集：后半		验证集	
x/kg	y/元	x/kg	y/元	x/kg	y/元
0.5	5.2	3.5	11.4	0.8	5.7
1.0	5.6	4.0	11.7	2.2	8.2
1.5	7.1	4.5	13.1	3.8	11.8
2.0	8.3	5.0	14.4	5.2	14.3
2.5	8.6	5.5	14.7		
3.0	10.2	6.0	16.1		

测试输入为 1.2, 2.8, 4.2, 5.8 kg，相应标签为 6.5, 9.5, 12.3, 15.5 元。标签虽在讲义中公开，实际执行方案仍须坚持先完成验证选择，再计算测试结果。公开教学数据适于核对代码，不适于模拟长期保密测试集。

比较方案包括均值基线和一个简单的近邻回归规则。对新重量 x，计算它与每个训练重量的距离 $|x - x_i|$，选最近的 k 个训练样本，取这些样本费用的平均值：

$$\hat{y}(x) = \frac{1}{k} \sum_{i \in N_k(x)} y_i.$$

$N_k(x)$ 是选中训练样本的下标集合，包含 k 项。距离与重量单位相同，费用平均的单位仍为元。候选 k 预先规定为 1、3 和 5；验证 MAE 越低越好，同分时取较小的 k。距离同分时保留训练表中的先后次序，代码中的稳定排序实现这一规则。

手算一次预测与基线

验证输入 $x = 2.2$ 到训练重量 2.0、2.5 和 1.5 的距离分别为 0.2、0.3 和 0.7，其他距离更大。因此

$$\hat{y}_{k=1} = 8.3, \qquad \hat{y}_{k=3} = \frac{8.3 + 8.6 + 7.1}{3} = 8.0.$$

真实值为 8.2，所以两种预测的绝对误差分别是 0.1 和 0.2 元。该点上 $k = 1$ 更好，但选 k 不能只观察这一点，要对整个验证集平均。

训练费用总和为 126.4 元，训练均值为

$$c = \frac{126.4}{12} \approx 10.533333 \text{ 元}.$$

均值基线对每个输入都输出这个数。它忽略重量，却利用训练集的总体费用水平。验证集上的四项绝对误差约为 4.833333、2.333333、1.266667、3.766667 元，平均为 3.05 元。

候选选择与最终报告

将每个候选对四个验证样本的误差平均，得到

方案	验证 MAE/元
均值基线	3.050000
近邻 $k = 1$	0.100000
近邻 $k = 3$	0.241667
近邻 $k = 5$	0.540000

依照预定规则选 $k = 1$。保持原训练集，在测试集上预测，得到

x/kg	真实费用/元	预测费用/元	预测减真实/元	绝对误差/元
1.2	6.5	5.6	−0.9	0.9
2.8	9.5	10.2	0.7	0.7
4.2	12.3	11.7	−0.6	0.6
5.8	15.5	16.1	0.6	0.6

最终测试 MAE 为 $(0.9 + 0.7 + 0.6 + 0.6)/4 = 0.7$ 元，均值基线测试 MAE 为 2.95 元。验证与测试 MAE 不同是正常现象，它们包含不同样本。测试只有 4 条，更不能将 0.7 元当作某个真实总体的精确平均误差。

这个结果支持的结论是：在当前固定教学划分上，按验证集选出的近邻规则优于指定均值基线。它不支持“近邻必然优于线性回归”，因为没有进行相应比较；也不支持“实际配送误差不超过 0.7 元”，因为平均误差不限制每个样本，更没有实际数据证据。

适用边界与可复现步骤

当输入 $x = 10$ kg 时，$k = 1$ 仍取最接近的 6.0 kg 训练样本，输出 16.1 元。它不会根据重量继续增长自动推算费用。输入落在已见范围之外的预测称为**外推**（extrapolation）；这种边界行为提醒模型有自己的归纳规则，不能只因范围内结果好就信任范围外结果。

本例的执行顺序为：固定任务与数据划分，拟合基线或保存训练记录，计算验证指标，选定候选，计算一次最终测试指标，检查逐样本误差并说明局限。其完整预测

计算通过对 n 个训练距离排序完成，每个查询通常需要 $O(n \log n)$ 排序时间，保存训练表需要 $O(n)$ 空间。这里的记号只表示增长趋势：数据规模翻倍，排序成本会增长；不是精确运行秒数。第 15 章将讨论更高效的实现。

> **注意：简化实现的边界**
>
> 本例只有一个数值特征、20 条构造记录，无缺失值和复杂编码。它展示完整学习流程的骨架。实际项目仍需要数据质量检查、合适划分、预处理、更多基线、统计不确定性和部署后的监测。

1.10 实验 1.1：可运行的近邻预测流程

环境、输入与运行

代码入口为 `ch01_regression.py`，保存在“配套实验”目录。使用 Python 3 标准库，不读取外部文件，不下载数据，不安装第三方库。在该目录打开终端后运行：

```
python3 ch01_regression.py
```

Windows 环境若 Python 由启动器管理，可用 `py -3 ch01_regression.py`。解释器和终端的区别在第 2 章说明。此时可以先看输出、手算核对，再于第二章后阅读函数细节。

数据列表与上一节完全对应。核心函数如下：

```
1  def neighbor_predict(data, x, k):
2      if not 1 <= k <= len(data):
3          raise ValueError("invalid k")
4      ordered = sorted(data, key=lambda row: abs(row[0] - x))
5      selected = ordered[:k]
6      return sum(y for _, y in selected) / k
7
8
9  def mae(data, predict):
10     errors = [abs(predict(x) - y) for x, y in data]
11     return sum(errors) / len(errors)
```

`data` 包含若干 (x_i, y_i) 对。第 2 行检查 k 是否在 1 到训练数量之间；第 4 行按距离排序，`row[0]` 取训练重量；第 5 行只保留前 k 条；第 6 行把费用相加后除以 k。`lambda` 是一个短的匿名函数，这里只表达“按重量距离排序”，无需在本章掌握语法。

`mae` 接受一个预测函数，对评价样本逐个计算绝对误差，最后平均。这样指标函数不必知道预测来自均值、近邻还是其他模型。代码中的函数式组织会在第二章函数一节再次分析。

选择逻辑与预期输出

```
results = []
for k in [1, 3, 5]:
    score = mae(valid, lambda x: neighbor_predict(train, x, k))
    results.append((score, k))
_, best_k = min(results)
final_score = mae(
    test, lambda x: neighbor_predict(train, x, best_k)
)
```

每个候选使用同一份 `train`。验证标签只进入误差计算，未被传给近邻预测函数。`results` 保存“验证分数、邻居数”对；`min` 按元组先比较分数，再比较 k，实现预定同分规则。最终测试行位于选择之后。

完整脚本预期主要输出为：

```
split sizes: 12 4 4
baseline validation MAE: 3.050000
k=1 validation MAE: 0.100000
k=3 validation MAE: 0.241667
k=5 validation MAE: 0.540000
selected k: 1
baseline test MAE: 2.950000
selected model test MAE: 0.700000
```

随后还会打印四个测试样本的预测和误差。输出保留六位小数是显示约定，不表示原始费用测量有六位小数精度。使用相同数据和规定排序规则时，应得到相同的打印结果；浮点内部结果与手算小数允许约 10^{-9} 的绝对差异。

修改任务与结果解释

先将训练标签的第二项 5.6 改成 9.6，重新运行，并手算验证输入 0.8 和测试输入 1.2 的最近邻预测。修改的是训练数据，允许重新执行完整选择；应记录这一变化，不能把新输出与原实验视为同一次固定评价。

再增加一个待预测重量 10.0，只打印预测，不把它混入原测试 MAE。该输入没有真实标签，不能计算正确误差。观察输出是否随重量继续线性增长，并联系上一节的外推限制。

最后尝试令 $k = 0$ 或 $k = 13$，确认程序主动拒绝不合法邻居数。如果删去检查，$k = 0$ 会导致除以零；$k = 13$ 则只有 12 个样本被选中，却错误地除以 13。这说明边界检查不仅为防止程序崩溃，也为防止产生看似正常但数学意义错误的数值。

1.11 实验 1.2：阈值、混淆矩阵与检查成本

输入与目标

代码入口为 `ch01_classification.py`。八条消息的标签与固定分数为

下标	0	1	2	3	4	5	6	7
标签	1	1	0	0	1	0	0	0
分数	.90	.60	.80	.30	.40	.20	.55	.10

标签 1 表示需要检查，0 表示不需要。分数在本实验中是人为给定的示意值，不宣称具有已校准概率含义。实验不训练分类模型，只分析从固定分数到类别的过程。因此本表也不是用于选定真实业务阈值的最终测试集。

先以阈值 0.5 手算预测：下标 0、1、2、6 判为正，其余判为负。逐条对照标签得到 TP=2、FP=2、TN=3、FN=1。阈值 0.7 时只有下标 0、2 判为正，得到 TP=1、FP=1、TN=4、FN=2。

实现与逐条核对

```
tp = fp = tn = fn = 0
for label, score in zip(labels, scores):
    predicted = int(score >= threshold)
    if label == 1 and predicted == 1:
        tp += 1
    elif label == 0 and predicted == 1:
        fp += 1
    elif label == 0 and predicted == 0:
        tn += 1
    else:
        fn += 1
```

`zip` 将相同位置的标签和分数配成一对。`score >= threshold` 得到真假值，转成整数后真为 1、假为 0。四个分支互斥，每条有效二分类记录恰好使一个计数加 1。完整脚本在循环前检查长度一致且非空，本实验输入预先保证标签属于 0 或 1。

运行

```
python3 ch01_classification.py
```

得到

```
threshold: 0.5
TP FP TN FN: 2 2 3 1
accuracy=0.625000 precision=0.500000 recall=0.666667
threshold: 0.7
TP FP TN FN: 1 1 4 2
accuracy=0.625000 precision=0.500000 recall=0.333333
```

两个阈值准确率都为 5/8，精确率都为 1/2，召回率却从 2/3 降至 1/3。因此只观察准确率，可能看不出漏检增加。所有计数应与手算完全一致；分数仅存在打印舍入差异。

修改任务

假设一次误报成本为 1，一次漏检成本为 5，定义总错误成本为 $C = \mathrm{FP} + 5\mathrm{FN}$。阈值 0.5 的成本为 7，阈值 0.7 的成本为 11。成本单位是人为定义的统一代价，不能在没有实际业务估计时当成真实金额。

将下标 4 的分数由 0.4 改成 0.6，其标签仍为 1。先不运行程序，分别预测两个阈值下四个计数怎样变化，再运行核对。还可将阈值设为 1.1，检查“没有预测正类”的情形：精确率返回 `None`，召回率为 0。配套脚本已先判断指标是否为 `None`，未定义值显示为 `undefined`，因此阈值 1.1 的修改也可直接运行。不能直接对 `None` 套用小数格式；显示标记表示缺少定义，并不把它当成 0 分。

1.12 知识联系、常见误区与自检

从问题到结果可用一条因果关系清楚的工作链描述：任务决定可用信息与目标；目标和错误成本决定评价；数据采集与划分决定评价条件；模型和训练算法决定候选预测规则；验证决定选择；测试提供在该协议下的结果。这条链中任何一环出现偏差，都可能使精确计算得到的分数失去原本含义。

本章常见误区有以下几类。把 AI 与神经网络等同，会忽略搜索和规则等方法；把所有数字目标当成回归，会给类别编号施加虚假的大小关系；把训练损失低当成新数据可靠，会忽略泛化；把验证和测试互换，会让最终结果受到选择影响；把评分当成概率，会误解阈值；把平均误差当成最大误差，会低估异常样本风险。

模型表现还不是因果结论。若较重的包裹往往同时使用更远的路线，模型可能利用重量预测费用，但这不证明“仅增加重量就会使费用按同样规律变化”。预测描述已观测关系，因果问题需要额外假设、设计或证据。本书主线首先建立预测建模与可靠评价基础。

完成本章后，应能不看公式解释 MAE 分母是什么；用具体记录说明 FP 与 FN；给出一个预测时不可用的字段；解释为何选 k 要读验证集而非测试集；说明一个实验结论的适用范围。如果只能复述名词而无法对新场景作上述判断，应回到任务卡和两个实验逐项对应。

1.13 分层习题

以下题目编号为 1.1–1.12。概念题要求用完整任务说明回答，计算题需写出中间量，实验题需同时给出数值与结论的边界。

1.1 **概念**。固定规则判断银行卡号格式、从标注图片学习识别缺陷、对文本预测下一个词元、在网格中根据奖励学习移动，分别对应什么方法或学习方式？能否把固定规则排除出所有人工智能系统？

1.2 **任务定义**。“预测学生成绩”过于含糊。以学期开始时预测期末分数为目标，写出样本单位、预测时点、三个候选输入、一个不能使用的字段和一个评价指标。说明标签单位。

1.3 **数据表示**。200 张图像已被表示为每张 64 个数值，标签有 5 类。写出 X 的形状；类别标签必须表示为 200×5 矩阵吗？若采用整数类别编号，怎样解释编号？

1.4 **计算**。真实值为 $(2, 4, 6)$，预测为 $(3, 4, 4)$。计算有符号误差、MAE、MSE 与 RMSE，并说明目标单位为元时各指标的单位。

1.5 **训练目标**。标签为 $(2, 2, 8)$。求平方损失下最佳常数预测与相应 MSE；再比较常数 2 与常数 4 的 MAE。解释为什么均值基线不一定是 MAE 最优基线。

1.6 **模型选择**。候选 A、B、C 的训练 MAE 分别为 0.2、0.5、0.8，验证 MAE 分别为 1.5、0.7、0.9。按最小验证 MAE 规则选哪个？能否从这些分数确定 A 一定发生了过拟合？

1.7 **泄漏判断**。预测订单最终费用时，分别讨论使用最终付款状态、在全部数据上拟合填补均值、同一订单复制记录跨入训练和测试。写出每种情况的修正措施。

1.8 **分类计算**。20 个实际正类、80 个实际负类中，模型找到 15 个正类，同时错报 12 个负类。给出混淆矩阵、准确率、精确率与召回率。将正类改成“普通消息”后，原 FP 对应新的哪种错误？

1.9 **划分**。同一用户有多条记录，数据跨越两年。任务甲是预测已有用户的下一次购买；任务乙是预测全新用户是否购买。分别提出合适的主要划分约束，说明为什么随机按行划分不一定合理。

1.10 **手算近邻**。训练记录为 $(1, 5), (2, 7), (4, 11), (5, 13)$。对 $x = 3.2$ 计算 $k = 1$ 和 $k = 3$ 的预测；若真实值为 10，比较绝对误差。对 $x = 8$ 的 $k = 1$ 预测又是多少？

1.11 **实验**。在实验 1.2 中把下标 4 的分数改成 0.6，其他不变。分别计算阈值 0.5 和 0.7 的指标、成本 FP + 5FN，并运行程序核对。若要选择真实系统阈值，还需要哪些数据安排？

1.12 **综合**。某报告写道：“尝试 100 次后选出测试准确率最高的模型，准确率 97%，因此任何未来消息都至少有 97% 概率被正确处理。”指出报告中的两个不同错误，并改写为条件明确的报告流程。

1.14 习题参考答案

1.1–1.3：概念与数据

1.1 银行卡号格式判断使用人工给定的规则，不必从样本学习；缺陷识别是监督分类；下一词元预测是从文本构造目标的自监督学习；按交互奖励学习移动是强化学习。固定规则可以成为 AI 系统的一部分，例如约束输出、组织推理或校验数据，但不能仅因使用计算机和条件语句，就把任意普通程序都赋予一个有实质意义的 AI 能力结论。

1.2 样本单位为某个学期的一名学生，预测时点为学期开始前或开始时，需明确到数据可用截止时刻。候选输入可以是先修课程成绩、已完成课程数量、当前课程类别；期末考试得分本身、期末后的教师评分与全学期出勤汇总均不能作为此时输入。标签为该学期最终分数，假设满分 100，单位为分。可用 MAE 评价，另检查不同课程或

历史水平群体的误差。该示例是任务定义练习，不假设这些输入足以产生可靠预测。

1.3 $X \in \mathbb{R}^{200\times 64}$。标签可以是长度 200 的整数序列，每项属于 $\{0,1,2,3,4\}$；也可以根据算法需要使用其他编码，并非必须为 200×5。整数是类别代号，不意味着类别 4 是类别 2 的两倍，也不宜对代号直接计算回归误差。

1.4–1.6：损失与选择

1.4 以预测减真实定义有符号误差，得到 $(1,0,-2)$。绝对值为 $(1,0,2)$，平方为 $(1,0,4)$，所以

$$\mathrm{MAE}=1,\qquad \mathrm{MSE}=\frac{5}{3},\qquad \mathrm{RMSE}=\sqrt{\frac{5}{3}}\approx 1.2910.$$

MAE 与 RMSE 单位为元，MSE 单位为元 ^{2}。有符号误差平均为 $-1/3$，它不能代替 MAE，因为正负错误可能抵消。

1.5 训练均值为 $(2+2+8)/3=4$。平方误差为 $(4,4,16)$，MSE 为 8。常数 2 的 MAE 为 $(0+0+6)/3=2$，常数 4 的 MAE 为 $(2+2+4)/3=8/3$。均值优化平方损失，中位数 2 在本例中给出更小的绝对损失。两种目标对偏离较大的 8 赋予不同权重。

1.6 选 B，因为验证 MAE 为 0.7，三者中最低。A 训练很低但验证高，符合需要排查过拟合的现象，也可能受划分偏差、分布变化或预测实现错误影响。应检查流程后再诊断，不能仅靠两个指标证明唯一原因。C 训练与验证都较高，可能模型能力不足或训练不充分，也要结合具体目标判断。

1.7–1.9：评价条件

1.7 最终付款状态产生在预测时点之后，应屏蔽该字段或重新定义预测时点。全数据填补均值把测试输入信息用于处理规则，应先划分，仅从训练集估计均值，再应用到其他集合。复制的订单跨集合会使测试包含训练已见实例，应先识别重复并按订单标识整体分配。去重规则需保留真正不同的订单，不能只因重量相同便删除。

1.8 TP=15、FN=5、FP=12、TN=68。准确率为 $83/100=83\%$，精确率为 $15/27\approx 55.56\%$，召回率为 $15/20=75\%$。若正负类整体交换，原来“实际负、预测正”的 FP 变成“实际正、预测负”的 FN。这说明错误名称依赖正类约定。

1.9 任务甲应以时间为主要约束，训练用较早记录，评价用后续记录；同一用户可按任务需要在前后时期出现，但只能使用截止时刻以前的信息。任务乙应以用户分组为主要约束，测试用户不能在训练中出现；若部署还面向后续时间，应同时设计时间边界。随机按行会混合前后时期，也会让同一用户跨集合，可能偏离两个任务中要求的条件。

1.10–1.12：计算、实验与报告

1.10 对 3.2，四个距离依次为 2.2、1.2、0.8、1.8。最近一个是 $(4,11)$，所以 $k=1$ 预测 11，误差 1。最近三个依次为 $(4,11),(2,7),(5,13)$，预测 $31/3\approx 10.3333$，误差 $1/3$。该点上 $k=3$ 更好，不能从一个点决定全局最优。对 8，最近为 $(5,13)$，$k=1$ 预测 13，表现为边界常值而非继续增长。

1.11 阈值 0.5 时，下标 4 由漏检变为检出，TP=3、FP=2、TN=3、FN=0。准确率 6/8 = 0.75，精确率 3/5 = 0.6，召回率 1，成本 2。阈值 0.7 时该样本仍被判负，因此计数与原实验相同，准确率 0.625，精确率 0.5，召回率 1/3，成本 11。真实系统需要独立训练与验证数据，在验证集选阈值，保留测试集作最终评价，并确保类别比例、采集条件和成本定义与实际使用相符。

1.12 第一个错误是反复使用测试集选模型，它已经成为选择数据；第二个错误是把有限集合上的平均正确比例解释为每个未来消息都满足的概率下界。未来分布可能变化，个体错误概率也不相同。

可先固定任务、划分和指标，用训练集拟合，用验证集完成 100 次候选比较，记录选择规则；选定方案后只作一次保留测试评价，并报告测试样本数、类别比例、指标和采集条件。若测试数据已被用于比较，需要另行准备未参与选择的评价数据。报告只陈述指定协议下观察到的结果，再分析不确定性与适用边界。

1.15 本章小结

机器学习通过数据或交互经验改进任务表现，深度学习和大语言模型是其中的重要方法与模型类别。学习方式取决于训练信号来源，任务类型取决于希望获得的输出。样本、特征、标签定义了数据，模型、参数、训练目标和算法定义了拟合过程，验证与测试定义了选择和最终评价的职责。

实际任务必须交代预测时点、可用字段、目标单位、评价对象和错误代价。训练表现、有限测试表现与未来总体表现是不同对象。下一章建立运行和阅读实验所需的 Python 基础，将本章的逐样本计算变成可核对的程序。

2 Python 语言基础

先修知识、学习目标与本章位置

本章从运行程序开始，只要求基本算术和第一章中的样本、误差概念。学习后应能区分终端、解释器与编辑器；运行并修改脚本；使用基本类型、条件、循环和容器；编写具有明确输入、输出和边界条件的函数；按顺序追踪程序中的数值变化。本章使用 Python 3，不使用 NumPy 或机器学习库。配套实验只依赖标准库。模块、文件读写、异常的完整机制、对象引用与调试流程在第 3 章继续；本章提前使用的少量输入检查会给出局部解释，不以此替代后续系统讨论。

机器学习实验把输入表示为程序对象，将公式中的逐项运算写为表达式，再利用循环或库函数处理许多样本。程序出现错误时，需要能够判断问题出在数据、控制流程还是计算规则。本章先建立这种基本阅读与编写能力，随后才进入数组和向量化。

程序的核心不是记住所有语法，而是对每一步的状态作出明确判断：这一行读取哪些值、产生什么值、修改了什么对象、下一行从哪里继续。为此，本章使用小规模且可手算的数据，并在重要代码后提供逐行分析。

2.1 运行程序：解释器、终端与编辑器

三个工具的职责

解释器（interpreter）执行 Python 程序。**终端**（terminal）提供输入系统命令的窗口，窗口内通常由命令行 **Shell** 解析命令。**编辑器**（editor）用于创建、保存和修改文本文件。集成开发环境可以把编辑、终端和运行按钮放在同一个窗口，但这些职责仍然不同。

终端里输入 `python3 hello.py`，Shell 启动 Python 解释器，再让解释器执行 `hello.py` 中的代码。Python 交互环境里输入 `2 + 3`，则是已经启动的解释器直接计算表达式。若把 `python3 hello.py` 写进 Python 文件，它不是一条合法 Python 语句；若在普通终端直接输入 `print(2 + 3)`，也不等于让 Python 运行这段代码。

位置	输入内容示例	实际负责者
普通终端	`python3 hello.py`	Shell 启动解释器
Python 交互环境	`2 + 3`	解释器求值
文本编辑器	`print(2 + 3)`	保存代码，运行时才执行
程序输出区域	数字 `5`	显示结果，不是新代码

本章代码框中的行号仅用于说明，不属于程序文本。终端命令框不含行号。将代码复制到文件时，不要加入解释器的提示符 `>>>`，也不要复制输出结果。

检查解释器与第一次交互

在 macOS 或 Linux 终端中运行

```
python3 --version
```

它应打印 Python 3 的版本信息。Windows 可尝试 `py -3 --version`；若系统将命令注册为 `python`，也可使用 `python --version`，但需确认确实是 Python 3。若命令不存在，说明解释器未安装、启动器未配置或路径未设置，应通过 Python 官方安装说明处理，不能把未找到命令误认作程序语法错误。

随后运行 `python3`，进入交互环境。下列框为输入与输出的完整示意：

```
>>> 2 + 3
5
>>> value = 8
>>> value * 2
16
>>> print(value)
8
>>> exit()
```

`>>>` 是解释器显示的提示符。赋值行 `value = 8` 通常不自动打印值，表达式 `value * 2` 在交互环境会显示求值结果。`exit()` 离开交互环境，返回普通终端。此后才输入执行脚本的系统命令。

保存与运行脚本

脚本（script）是包含程序文本的文件。创建 UTF-8 编码的纯文本文件 `hello.py`，保存如下内容：

```
name = "Python"
count = 3
print("language:", name)
print("count:", count)
print(2 + count)
```

在文件所在目录打开终端，运行 `python3 hello.py`，输出应为

```
language: Python
count: 3
5
```

解释器按顺序执行：先把名称 `name` 绑定到字符串，把 `count` 绑定到整数，再进行三次打印。`print` 括号中的多个项默认用空格分隔，打印后换行。

脚本中的单独表达式 `2 + count` 会被计算，但通常不显示结果。若要看到它，必须写 `print(2 + count)`。这是交互环境和脚本在初学阶段最容易混淆的区别之一。

目录、路径与空格

终端有**当前工作目录**。相对文件名 `hello.py` 会在当前目录中寻找，不会自动在整个电脑上搜索。macOS 或 Linux 使用 `pwd` 查看当前目录，`ls` 查看文件；Windows 命令提示符可用 `cd` 显示当前目录、`dir` 列出文件。

本讲义配套实验位于含中文和空格的目录，可在 macOS 终端中运行：

```
cd "/Users/mac/Desktop/机器学习 ML/配套实验"
python3 ch02_basics.py
```

路径两侧的引号让 Shell 把它看成一个整体。这里的引号用于系统命令中的路径，不是 Python 文件中的字符串定义。也可以使用脚本的绝对路径运行，不必先切换目录；但更复杂的程序可能读取相对文件，届时工作目录仍需核对。

保存时应核对扩展名。正确名称为 `hello.py`；若名称为 `hello.py.txt`，实际仍是文本扩展名。代码中的括号、冒号、引号和逗号使用英文半角符号。中文全角冒号或智能引号看起来相近，却不属于相同的语法字符。

2.2 虚拟环境、标准库与依赖

为什么区分环境

标准库随 Python 提供，包含数学、随机数、文件路径等许多功能。**第三方包**是另外安装的代码，例如后续使用的 NumPy、pandas 和 PyTorch。程序所依赖的包及其版本称为**依赖**（dependency）。本章两个实验以及第 1 章的两个实验均无需安装第三方包。

不同项目可能要求不同包版本。如果把所有包混装在一个环境中，一次升级可能影响另一项目。**虚拟环境**（virtual environment）为一个项目隔离包安装位置，并提供与该环境关联的解释器入口。它不是虚拟机，不会另建一套完整操作系统。

在项目目录中创建环境：

```
python3 -m venv .venv
```

`-m venv` 表示让选定的解释器运行标准库中的 `venv` 模块；`.venv` 是环境目录名。创建成功后，再根据 Shell 激活：

终端类型	激活命令
macOS/Linux 的常见 Shell	`source .venv/bin/activate`
Windows 命令提示符	`.venv\Scripts\activate.bat`
Windows PowerShell	`.venv\Scripts\Activate.ps1`

激活后通常使用 `python` 调用环境解释器，退出可用 `deactivate`。PowerShell 可能受脚本执行策略限制；可按官方说明处理，或直接调用环境解释器，无须为运行本章实验修改系统级策略。

安装位置与解释器对应

无需激活也可以明确调用环境中的解释器，macOS/Linux 为`.venv/bin/python`，Windows 为`.venv\Scripts\python.exe`。这种方式减少“包安装到了一个环境，脚本却在另一个环境运行”的混淆。

需要安装第三方包时，优先使用 `python -m pip`，例如后续章节的环境中可能运行 `python -m pip install numpy`。这会使用当前指定解释器对应的包管理工具。安装不是本章的实验步骤，提前安装机器学习库也不能代替掌握基础语法。

编辑器可能另有解释器选择设置。即使终端激活了环境，编辑器运行按钮也不必自动采用相同解释器。可用下列小程序查看实际执行者：

```
import sys
print(sys.executable)
print(sys.version)
```

`import sys` 导入标准库模块，下一行打印解释器路径，再下一行打印版本。模块的一般组织方式将在第 3 章展开。此处只把这段程序作为环境核对工具。

> **注意：环境准备的最小要求**
>
> 能确认 Python 3 解释器、保存 UTF-8 脚本并在正确目录运行，就能完成本章实验。虚拟环境为后续依赖管理建立习惯；它不会改变 `if`、`for`、列表或函数的基本语义。

2.3 变量、赋值、对象与表达式

名称绑定与顺序执行

变量在本章可理解为指向某个值的名称。更准确地说，赋值建立名称与对象的绑定。写 `weight = 2.5` 时，右边表达式先求值，再把左边名称绑定到结果。= 不是数学中两个表达式永远相等的声明。

```
x = 3
x = x + 2
y = 2 * x
print(x, y)
```

第一行后 `x` 为 3；第二行先读取旧 `x`，算出 5，再更新绑定；第三行读取当前 `x`，得到 `y` 为 10。最终输出 `5 10`。不存在代数方程 $x = x + 2$ 所导致的矛盾，因为它描述了两个不同执行时刻。

如果名称还未赋值就被读取，例如脚本第一行直接写 `print(weight)`，会产生 `NameError`。给名称赋值时不需要先声明固定类型，但对象具有类型。`count = 3` 后又写 `count = "three"` 是允许的；后续代码若期待数字，仍可能产生类型错误。因此名称应保持稳定含义，避免随意切换角色。

命名与注释

名称可以包含字母、数字和下划线，不能以数字开头，不能使用 `for`、`if` 等关键字。Python 允许部分 Unicode 名称，本讲义代码采用清晰的英文名称与下划线，例如 `train_size`、`predicted_cost`。大小写不同表示不同名称：`Score` 和 `score` 不相同。

不要将 `sum`、`list`、`str`、`min` 等常用内置功能名称用作变量。若写 `sum = 10`，再写 `sum(values)`，名称已经指向整数，便不能作为函数调用。变量名称应表达意义，而不是仅方便输入。

`#` 后直到行末为**注释**，解释器不执行这些文字。注释应解释规则、单位或约束，而不是重复显而易见的代码。如下写法记录了单价单位：

```
price_per_kg = 2.0  # unit: yuan per kg
weight_kg = 3.5
cost_yuan = price_per_kg * weight_kg
```

字符串中的 `#` 只是普通字符，例如`"order#17"`。长说明可以放在文档字符串中，函数一节会给出示例。

表达式、语句与增强赋值

表达式求出一个值，例如 `2 * x + 1`、`score >= 60` 或 `len(values)`。**语句**执行某种动作或控制流程，例如赋值、条件分支和循环。函数调用既可以在表达式中提供返回值，也可以作为独立语句执行。

`total += value` 在数值累加场景中相当于把 `total + value` 的结果重新赋给 `total`。它不能省略初始化：第一次累加前要给 `total` 赋初值。容器的增强赋值可能涉及对象修改，引用与副作用的完整区别将在第 3 章讨论，本章主要使用数值累加。

多重赋值如 `a, b = 2, 5` 先求右边，再绑定左右对应名称；`a, b = b, a` 因此能交换数值。若分两行 `a = b`、`b = a`，第一行已经覆盖旧 `a`，第二行将得到与 `a` 相同的值。

> **例题 2.1：赋值追踪与交换**
>
> 执行下列代码后，`a`、`b` 和 `c` 分别是多少？为什么？

```
a = 4
b = a + 3
a = 2 * a
c = b - a
a, b = b, a
print(a, b, c)
```

解 第 1 行建立 `a=4`。第 2 行得到 `b=7`。第 3 行把 `a` 改为 8，不会让先前计算好的 `b` 重新计算。第 4 行得到 `c=-1`。第 5 行同时使用旧 `b=7` 和旧 `a=8`，交换后 `a=7,b=8`。输出为 `7 8 -1`。变量不是自动维护的电子表格公式，修改输入名称不会回溯执行旧语句。

2.4 数值类型、运算与浮点误差

整数、浮点数与类型转换

整数类型为 `int`，例如 3、0 和 −7；浮点数类型为 `float`，例如 3.0、0.25 和 `1e-3`。科学计数形式 `1e-3` 表示 10^{-3}。Python 整数可以随数值增长使用更多存储空间，不等于机器学习数组中的固定宽度整数；浮点数精度则有限。

用 `type(value)` 查看类型。`int(3.9)` 得到 3，它朝零截断，并不是四舍五入；`int(-3.9)` 得到 −3。`float("3.5")` 把合法数字字符串转为浮点数，`int("3.5")` 会失败，因为该字符串不是整数格式。`str(3.5)` 则产生文本`"3.5"`，不再是可直接进行数值加法的浮点数。

类型转换需要有数据规则。用户输入`"abc"` 不能直接转为浮点数；空串也不能。字符串中有数字并不保证整体是合法数字，例如`"score=82"`。完整输入校验与异常处理在第 3 章学习，本章实验采用范围有限且能事先辨认的整数文本。

运算符与优先级

符号	含义	示例与结果
`+ - *`	加、减、乘	`3 * 4` 为 12
`/`	真除法	`7 / 2` 为 3.5
`//`	向下取整除法	`7 // 2` 为 3
`%`	余数	`7 % 2` 为 1
`**`	幂	`2 ** 3` 为 8
`abs(x)`	绝对值	`abs(-2)` 为 2

加减低于乘除，幂的优先级较高；应优先通过括号明确数学意义。均方误差的一项应写 `(prediction - target) ** 2`，若写 `prediction - target ** 2`，只会把真实值平方再相减。

负数除法尤其需要核对：`-7 // 2` 为 −4，因为 −3.5 向下取整到 −4，不是朝零截断到 −3。配套余数`-7 % 2` 为 1，仍满足

$$a = (a//b)\,b + (a\%b), \qquad b \neq 0.$$

若 a, b 都为整数，则这些算术规则可用于整除与余数判断。写 `i % 2 == 0` 判断整数是否偶数。除数为零会报错，不能将“没有样本”的平均值直接计算为 `total / 0`。

`-2 ** 2` 得到 −4，解析为`-(2 ** 2)`；`(-2) ** 2` 得到 4。多层幂如 `2 ** 3 ** 2` 按 `2 ** (3 ** 2)` 解释。复杂表达式宜用括号和中间名称分解，减少依赖记忆优先级。

例题 2.2：把数学式译为程序

给定预测 $\hat{y} = 9$、真实值 $y = 6$，计算绝对误差、平方误差，以及 $\sqrt{\frac{1+4+9}{3}}$。写出不会混淆优先级的代码。

```
prediction = 9
actual = 6
absolute_error = abs(prediction - actual)
squared_error = (prediction - actual) ** 2
root_mean_square = ((1 + 4 + 9) / 3) ** 0.5
print(absolute_error, squared_error, root_mean_square)
```

解 绝对误差为 3，平方误差为 9，最后一项为 $\sqrt{14/3} \approx 2.160246899$。第 5 行先求和、再平均、最后开方，括号直接对应公式结构。指数 0.5 在非负实数上用于平方根；这里被开方量非负，条件满足。

浮点数不是所有小数的精确表示

常见 Python 浮点数使用有限位二进制表示。十进制有限小数不一定有有限二进制展开，类似 1/3 在十进制中必须近似。因此

```
print(0.1 + 0.2)
print(0.1 + 0.2 == 0.3)
```

通常输出 `0.30000000000000004` 和 `False`。这不是加法规则改变，而是输入数值和运算结果以邻近的可表示数近似。

`round(value, 2)` 和格式化保留两位小数可以改善显示，但不能让任意内部数值变成精确十进制数。`round` 还采用特定的舍入规则，对恰好中点的可精确表示数倾向于保留偶数，例如 `round(2.5)` 为 2、`round(3.5)` 为 4。`round(2.675, 2)` 可能为 2.67，因为 2.675 本身已被二进制近似，不宜仅用十进制手工舍入直觉推断。

对计算结果检查，应采用容差：

```
import math
value = 0.1 + 0.2
print(math.isclose(value, 0.3, rel_tol=1e-9, abs_tol=1e-12))
```

输出 `True`。`rel_tol` 随数值规模设定相对容差，`abs_tol` 为接近零的比较提供绝对容差。大致判断条件为

$$|a - b| \le \max(r \max(|a|, |b|), \varepsilon),$$

其中 r 为相对容差，ε 为绝对容差。具体容差应依目标单位、计算误差和问题要求设定，而不是所有问题都固定使用同一值。

> **注意：误差来源需要区分**
>
> 输入测量误差、有限样本造成的统计波动、模型预测误差和计算机浮点舍入误差，是不同来源。显示更多小数不能减少前几种误差。对费用、成绩等问题，首先保持单位与计算规则一致，再讨论数值精度。

2.5 字符串、输入与输出

字符串及其索引

字符串（string，`str`）表示文本，使用单引号或双引号包围，例如`"train"`。引号不是字符串内容的一部分。`"12" + "3"` 得到`"123"`，`12 + 3` 得到 15；`"12" + 3` 因类型不兼容而失败。

字符串可拼接或重复：`"ab" * 3` 得到`"ababab"`。`len(text)` 给出字符串长度；`text[0]` 读取第一个字符；`text[-1]` 读取最后一个字符。Python 索引从 0 开始，长度为 n 时，正常非负索引为 0 至 $n-1$。空字符串没有第一个或最后一个字符，取相应索引会报错。

```
word = "Python"
print(len(word))
print(word[0], word[-1])
print(word[:2], word[2:])
```

输出依次为 6、`P n`、`Py thon`。切片语法会在容器一节系统解释。对普通中文字符串，`len("机器学习")` 为 4，但 Unicode 字符串长度一般不等于用户视觉上感知的字符数量，带组合符号的文字可能需要更细处理。本章数据不涉及这些复杂情形。

常用文本处理与不可变性

`text.strip()` 返回去掉两端空白的新字符串，`text.lower()` 返回小写形式，`text.split(",")` 按逗号拆分，`",".join(parts)` 将多个字符串用逗号连接。点号形式表示访问对象提供的**方法**（method）；类与对象的系统讲解在第 3 章进行，本章先按“对这段文本执行某种操作”理解。

```
raw = "  Train,VALID,test  "
clean = raw.strip().lower()
parts = clean.split(",")
print(raw)
print(clean)
print(parts)
print(" | ".join(parts))
```

`raw` 仍含原来两端的空格，`clean` 为`"train,valid,test"`，`parts` 为三个字符串组成的列表，最后输出 `train | valid | test`。方法可以连续调用，是因为 `strip()` 返回的仍是字符串，接着可调用 `lower()`。

字符串是**不可变**对象，不能写 `word[0] = "p"` 修改单个字符。若需要改变，可构

造新字符串"p" + word[1:] 并重新绑定名称。列表则允许逐项修改，二者不能混用。

格式化、转义与交互输入

格式化字符串 f"MAE={mae:.3f}" 将变量 mae 的值嵌入文本，并按三位小数显示。这里的 mae 应是数值变量。百分比格式 f"rate={rate:.1%}" 把比例 0.8 显示为 80.0%，不改变原变量。

```
score = 82
rate = 0.8
print(f"score={score}, rate={rate:.1%}")
```

输出 score=82, rate=80.0%。第 1 章脚本使用"...".format(...)，也是格式化方法，采用位置参数替代占位符。两种形式都可用，初学阶段选一种并保持一致即可。

\n 表示换行，\t 表示制表符，字符串中的反斜杠需按转义规则处理。写 Windows 路径时可使用原始字符串如 r"C:\data\input.txt"，但一般文件路径操作会在第 3 章采用更稳妥的路径工具。

input("weight: ") 显示提示并读取一行输入，返回字符串，即使输入的是 2.5。若任务要求数值，应明确转换：

```
text = input("weight in kg: ")
weight = float(text)
print(4 + 2 * weight)
```

输入 2.5 时输出 9.0。非法文本会在转换时失败；本章自动运行的实验使用固定数据而不是等待输入，方便重复核对。

2.6 布尔值、比较与条件分支

比较产生真假值

布尔值（boolean）只有 True 和 False，注意首字母大写。比较运算包括 ==、!=、<、<=、> 和 >=。= 是赋值，== 才是比较两个值是否相等。

and 表示两个条件均满足，or 表示至少一个满足，not 否定条件。对布尔条件 score >= 0 and score <= 100，也可以写链式比较 0 <= score <= 100。数学中的区间条件与程序表达因此直接对应。

Python 使用**短路求值**：A and B 在 A 为假时不必求 B，A or B 在 A 为真时不必求 B。例如 len(values) > 0 and values[0] > 0，空列表时前半为假，不会访问不存在的第一个元素。若顺序交换，则可能先触发索引错误。

and 和 or 一般返回被选中的操作数，不一定返回布尔值。例如 0 or 5 为 5，3 and 7 为 7。在本章的条件表达中主要使用真正的比较，避免依赖这种更宽泛语义。数字 0、空字符串、空列表和 None 等被视为假，但“假”不等于“无效数据”：成绩 0 是合法数值，实验将专门保留它。

缩进、冒号与分支结构

```
score = 76
if score >= 90:
    grade = "A"
elif score >= 60:
    grade = "pass"
else:
    grade = "fail"
print(grade)
```

if 从第一个条件开始；不满足时才检查 elif；前面的条件全不满足才执行 else。每个分支标题末尾有冒号，分支内部缩进四个空格。同一层级的缩进必须一致，避免混用制表符与空格。

当分数为 76 时，第一个条件为假，第二个为真，执行 grade = "pass"，不再执行其他分支。最后 print 没有缩进，属于分支结构之后，无论哪个分支被选中都会运行。

三个独立 if 与一个 if/elif/else 结构不同。独立 if 会逐个检查，可能执行多个代码块。如果先判断 score >= 60，再用 elif score >= 90，高分样本会被第一分支提前处理，后面的 90 分条件永远无机会成立。

例题 2.3：边界和条件顺序

将非负重量按收费规则处理：重量不超过 1 kg 收 6 元；超过 1 kg 但不超过 3 kg 收 10 元；超过 3 kg 收 15 元。负重量应拒绝。写出完整分支并核对 0、1、3 和 3.1 kg。

```
weight = 3.0
if weight < 0:
    cost = None
elif weight <= 1:
    cost = 6
elif weight <= 3:
    cost = 10
else:
    cost = 15
print(cost)
```

解　四个核对输入的结果分别为 6、6、10、15。None 是表示“没有普通结果”的特殊对象，不是字符串"None"，也不是数值 0。当前教学规则允许 0 kg，真实收费规则是否允许需要另行定义。前面分支已经排除 weight <= 1，到第 6 行时无需再重复 weight > 1。

输入若为字符串而非数值，比较仍可能失败；若为某些特殊浮点数，也需要额外校验。此例的假设是输入为普通有限数值，只负责范围和边界规则，不宣称是完整的生产级输入处理。

2.7 列表、元组与切片

列表与逐项访问

列表（list）保存按顺序排列的一组对象，使用方括号，例如 `scores = [82, 91, 76]`。顺序属于数据含义的一部分，`scores[0]` 为 82，`scores[-1]` 为 76，`len(scores)` 为 3。`scores[3]` 越界，因为最大非负索引是 2。

列表可以混合不同类型，但用于数值实验时应尽量保持含义和类型一致。`[1, "2", None]` 不能直接当成三个数值平均；需要先说明文本转换和缺失处理。空列表写作 `[]`，它有长度 0，不能直接取首项。

列表是**可变**对象，可修改、增加和删除元素：

```
scores = [82, 91, 76]
scores[1] = 93
scores.append(88)
last = scores.pop()
print(scores)
print(last)
```

第 2 行将第二项替换为 93；第 3 行把 88 加到末尾；第 4 行删除并返回末尾 88，所以最后列表为 `[82, 93, 76]`，`last` 为 88。`append` 是在原列表上修改，并返回 `None`。若写 `scores = scores.append(88)`，会把名称 `scores` 绑定到 `None`，这是常见错误。

`extend` 逐项加入另一个可迭代对象，`append` 则将参数作为一个整体加入：

```
a = [1, 2]
a.append([3, 4])
b = [1, 2]
b.extend([3, 4])
print(a)
print(b)
```

输出分别是 `[1, 2, [3, 4]]` 和 `[1, 2, 3, 4]`。前者长度为 3，第三项本身是列表；后者长度为 4，所有项为整数。嵌套数据读取错误往往来自没有先确定层级与形状。

切片与半开区间

切片 `values[start:stop:step]` 按指定步长选取一段序列。最常见的正步长形式包含起点、不包含终点，即**半开区间** $[\text{start}, \text{stop})$。省略起点默认从开头，省略终点默认到末尾，省略步长默认为 1。

```
values = [10, 20, 30, 40, 50, 60]
print(values[1:4])
print(values[:3])
print(values[3:])
print(values[::2])
print(values[::-1])
```

输出依次为 `[20, 30, 40]`、`[10, 20, 30]`、`[40, 50, 60]`、`[10, 30, 50]` 和 `[60, 50, 40, 30, 20, 10]`。`[::-1]` 按反向顺序选取所有项，不会修改原列表。

长度为 n 的序列在边界内可以用 `values[:r]` 和 `values[r:]` 拆成前后两段，不遗漏也不重复第 r 项。把切片误读为包含终点，就会在数据划分中多取一个样本。切片越界通常会被截到可用范围，例如 `values[:100]` 取得全部六项；单项索引越界则报错。步长不能为 0。

对普通列表，切片创建一个新列表。若原列表中的元素本身仍是可变对象，切片并不递归复制内部对象。本章一维数值列表不用展开该问题，第 3 章将通过引用和浅复制详细讨论。不要将列表切片的行为直接套用到第 4 章的 NumPy 数组，数组切片可能是视图。

例题 2.4：划分与配对不能分离

特征为 `x = [1, 2, 3, 4, 5]`，标签为 `y = [5, 7, 9, 11, 13]`。以前三条训练、后两条评价。写出切片，并说明若只把 `x` 倒序会发生什么。

```
x_train, x_eval = x[:3], x[3:]
y_train, y_eval = y[:3], y[3:]
print(x_train, y_train)
print(x_eval, y_eval)
```

解 训练对应关系为 $(1,5),(2,7),(3,9)$，评价为 $(4,11),(5,13)$。若只令 `x = x[::-1]`，标签顺序不变，就会得到 $(5,5),(4,7),\ldots$，破坏样本与标签配对。统一把数据表示为记录对，再对记录整体排序和切片，可减少此类错误。这里的前后切片仅用于解释语法，不自动意味着任意任务都应按当前行顺序划分。

元组、解包与嵌套记录

元组（tuple）也是有序序列，常用于固定结构的记录。`(2.5, 8.6)` 可以表示“重量、费用”对；不能通过 `row[0] = 3.0` 修改其中的引用。单元素元组写成 `(5,)`，末尾逗号不能省略，否则 `(5)` 只是带括号的整数。

解包（unpacking）将序列中的项分配给多个名称：

```
row = (2.5, 8.6)
weight, cost = row
print(weight, cost)
```

右边有两项，左边有两个名称，因此一一对应。数量不匹配会报错。前文的 `a, b`

`= b, a` 也使用了这种机制。

多个记录可保存为“列表中的元组”：

```
records = [(1.0, 6.0), (2.0, 8.0), (3.0, 10.0)]
print(records[1])
print(records[1][0])
```

第一项索引 1 取出第二笔记录 `(2.0, 8.0)`；随后索引 0 取出该记录的重量 2.0。读取嵌套结构时应从左到右判断每一步返回的对象。

元组的不可变性限制的是它自身各位置的引用。如果元组内部装着列表，那个列表仍可能被修改。因此不能仅因数据外层是元组，就推断内部数据深度不可变。为保持本章语义简单，实验记录对只含数值或字符串。

2.8 字典、集合与容器选择

字典按键组织数据

字典（dict）把键（key）映射到值（value），例如

```
summary = {"count": 5, "mean": 67.4, "pass_rate": 0.8}
print(summary["mean"])
summary["maximum"] = 91
print(summary.get("median"))
```

按键`"mean"` 查到 67.4，新键`"maximum"` 被加入。`get` 在键不存在时默认返回 `None`，也可写 `summary.get("median", 0)` 提供默认值。但是默认 0 是否合理取决于任务，统计量缺失通常不应悄悄当成 0。

普通索引 `summary["median"]` 在键不存在时会报 `KeyError`，有助于发现程序假设错误。`if "mean" in summary` 检查键是否存在，而不是检查 67.4 是否在所有值中。`summary.keys()`、`summary.values()` 和 `summary.items()` 分别用于访问键、值和键值对。

字典适合表示固定命名字段，如一份统计报告或一条订单记录。列表适合按位置遍历一批样本。实际数据也可以是记录字典组成的列表。读取时先用 `records[i]` 选出一个样本，再按键`"weight"` 读取重量；完整写法如下。

```
records[i]["weight"]
```

Python 3 字典保留插入顺序，但“按插入顺序迭代”不等于“按键的大小排序”。重复赋值同一个键会替换旧值，不会保存两条同名键值对。

```
counts = {"pass": 0, "fail": 0}
counts["pass"] = counts["pass"] + 1
counts["pass"] += 1
print(counts)
```

最终`"pass"` 对应 2，`"fail"` 对应 0。这类计数与第一章混淆矩阵的四个类别计数具有同样的累加结构。

集合用于去重和成员关系

集合（set）保存互不重复的元素，没有用于本章数据配对的确定位置顺序。例如 `set(["A", "B", "A"])` 只含 A 与 B。空集合必须写 `set()`，`{}` 是空字典。

常用运算包括交集 `a & b`、并集 `a | b` 和差集 `a - b`。若训练用户标识集合与测试用户标识集合交集非空，说明存在跨集合用户重叠；是否违反任务协议，还要看第一章中的使用目标。

```
train_users = {"u1", "u2", "u3"}
test_users = {"u3", "u4"}
overlap = train_users & test_users
print(sorted(overlap))
```

输出 `['u3']`。用 `sorted` 将集合变成稳定排序的列表，便于显示。不要用集合存放需要与标签位置一一对应的特征序列，去重和无位置语义会破坏配对。

列表、字典和集合等可变容器不能直接作为集合元素或字典键，因为它们不具备所需的**可哈希性**（hashability）。字符串、整数通常可以；元组本身不可变，但只有所有元素都可哈希时，元组才可哈希。例如 `(1, 2)` 可以作键，`([1], 2)` 却不可以，因为其中含有不可哈希的列表。可哈希性不能仅按外层是否可变判断。哈希的机制在后续算法中再说明，本章先据此选择合法键类型。

容器	最直接的用途	应避免的误用
列表	有序样本与可修改序列	忘记索引从 0 开始
元组	固定结构的一条记录	误以为内部对象都不可变
字典	命名字段、键到值的映射	把键的存在当成值的存在
集合	去重、交并差、成员判断	用其位置连接特征与标签

2.9 for 循环：遍历、累加与配对

从一项公式到多项计算

`for` 逐项遍历可迭代对象。对列表计算平均数：

```
values = [6.0, 8.0, 10.0]
total = 0.0
for value in values:
    total += value
mean = total / len(values)
print(mean)
```

循环开始前 `total=0.0`。每次取出一个列表元素，绑定到 `value`，执行缩进内部的累加。三个循环后 `total=24.0`，离开循环，再除以 3 得到 8.0。平均行若错误缩进进循环，就会在尚未累加完整时不断计算中间结果。

轮次	当前 value	累加前 total	累加后 total
1	6.0	0.0	6.0
2	8.0	6.0	14.0
3	10.0	14.0	24.0

这个累加器对应数学求和 $\sum_i v_i$。若列表为空，循环一次也不执行，total 仍为 0.0，但平均值除以 0，因此必须另行处理空输入。

range 与边界

range(stop) 提供 0 到 stop−1 的整数序列。完整形式可指定起点、终点和步长，如下面三个示例所示。它不是立即生成一个普通列表，但可在循环中逐项使用；调用 list 可显式转成列表。

```
print(list(range(4)))
print(list(range(1, 5)))
print(list(range(5, 0, -2)))
```

输出为 [0, 1, 2, 3]、[1, 2, 3, 4]、[5, 3, 1]。终点仍被排除，反向步长时只要继续满足方向边界就取下一项。

计算 1 至 n 的和可写 for i in range(1, n + 1)。若误写 range(1, n)，会漏掉 n。不需要位置时，应直接 for value in values，减少不必要索引；需要下标时可以使用 enumerate。

enumerate 与 zip

enumerate(values) 逐项产生“下标、值”对：

```
values = [6, 8, 10]
for index, value in enumerate(values):
    print(index, value)
```

输出三行为 0 6、1 8、2 10。它适合错误定位和逐条报告，不需要手动维护另一个下标变量。

zip(actual, predicted) 把相同位置的两个值配成一对：

```
actual = [6, 8, 10]
predicted = [5, 9, 10]
total = 0.0
for y, y_hat in zip(actual, predicted):
    total += abs(y_hat - y)
print(total / len(actual))
```

绝对误差为 1、1、0，结果为 2/3。循环直接对应 $\sum_i |\hat{y}_i - y_i|$，y 和 y_hat 的名称与公式角色一致。

默认 `zip` 在最短输入结束时停止。若真实值有 3 项、预测只有 2 项，最后一项会被遗漏，程序可能仍给出一个数。这是静默逻辑错误，不一定报错。指标函数应先检查两者等长且非空。本章采用显式长度检查，以保持对常见 Python 3 环境的兼容。

> **例题 2.5：错误代码为什么看似能运行**
>
> 真实值为 `[2, 4, 6]`，预测为 `[3, 5]`。某程序使用 `zip` 累加绝对误差，再除以 `len(actual)`。它输出什么？为什么不能称为该数据的 MAE？

解 `zip` 只产生 $(2,3)$ 和 $(4,5)$ 两对，误差总和为 2，再除以 3 得到 $2/3$。第三个真实值没有对应预测，问题本身缺少完整输入。分子只计算两项，分母却计算三项，既不是完整三条记录的 MAE，也不是两条有效配对的 MAE。正确处理是拒绝长度不匹配的输入，而不是通过更换分母掩盖缺失预测。

break、continue 与嵌套循环

`break` 立即结束当前最近一层循环，`continue` 跳过本轮剩余代码并进入下一轮。例如查找第一个超过 100 的值：

```
values = [82, 91, 105, 88]
first_bad = None
for index, value in enumerate(values):
    if value > 100:
        first_bad = index
        break
print(first_bad)
```

第 3 项 105 在下标 2 出现，记录 2 后结束循环，不检查 88。初始化 `None` 保证即使没有错误，变量仍有明确结果。

忽略负数但保留 0 可以写：

```
clean = []
for value in [3, -1, 0, 5]:
    if value < 0:
        continue
    clean.append(value)
print(clean)
```

输出 `[3, 0, 5]`。若以 `if not value` 代替负数判断，会错误跳过 0，却无法跳过 -1。真假语义不等于任务有效性规则。

嵌套循环适用于遍历二维结构：

```
rows = [[1, 2], [3, 4], [5, 6]]
total = 0
for row in rows:
    for value in row:
        total += value
print(total)
```

外层取三行，内层各取两项，累计为 21。若每行有 d 项、共 n 行，该程序进行 nd 次加法，时间随 nd 增长。理解这些循环是第 4 章向量化改写的前提。

2.10 while 循环与终止条件

`while` 在条件为真时重复执行代码块，适合重复次数尚未确定但有停止条件的过程。

```
value = 1
steps = 0
while value < 20:
    value *= 2
    steps += 1
print(value, steps)
```

每轮开始先检查 `value < 20`，成立才执行。`value` 依次变成 2、4、8、16、32，`steps` 为 5。得到 32 后下次检查不成立，结束；条件不是执行过程中时时检查，所以结果可能越过边界。

轮次	检查前 `value`	条件	执行后 `value`
1	1	真	2
2	2	真	4
3	4	真	8
4	8	真	16
5	16	真	32
下一次	32	假	不执行

若漏写 `value *= 2`，条件永远为真，就产生无限循环。写 `while` 前应回答：哪些变量决定条件；它们每次怎样改变；在什么输入条件下能停止。迭代算法通常还应有最大步数，防止数值条件永远达不到。

一个简单的收敛示意为不断将差距减半：

```
gap = 1.0
steps = 0
while gap > 0.01 and steps < 20:
    gap /= 2
    steps += 1
print(gap, steps)
```

7 步后 `gap` 为 0.0078125，满足容差而结束。最大 20 步是保护条件。该例只演示控制流程，不是某个机器学习算法的完整收敛证明。

当重复次数已知、对象可直接遍历时，优先用 `for`；当下一次是否执行依赖动态状态时，用 `while` 更自然。两者都需明确初始化和终止，循环本身不会判断任务是否已经正确完成。

2.11 函数、参数、返回值与作用域

把重复计算封装为函数

函数（function）为一段有名称、可调用的计算规则。定义采用 `def`：

```
def predict_cost(weight, base=4.0, rate=2.0):
    """Return the predicted cost in yuan."""
    cost = base + rate * weight
    return cost

print(predict_cost(2.5))
print(predict_cost(2.5, base=5.0, rate=3.0))
```

定义函数时并不立即执行内部收费计算，而是创建函数对象并绑定名称。第 6 行调用才把 `weight` 设为 2.5，未提供的 `base`、`rate` 采用默认值，返回 9.0。第 7 行使用关键字参数覆盖默认值，返回 12.5。

定义中的名称称为**形参**（parameter），调用中提供的具体对象称为**实参**（argument）。这里语法中的“参数”与第一章的“模型参数”不是同一个分类问题：函数参数可以传入数据、配置、模型系数甚至另一个函数；它是否属于模型可训练参数，要由学习流程决定。

首行说明字符串称为**文档字符串**（docstring），可以交代单位、返回值和条件。一个函数的说明应让调用者知道怎样提供输入、如何解释输出以及哪些输入不接受。

return 与 print 有不同作用

`return` 把结果交还给调用位置，并立即结束当前函数。`print` 只将内容显示到输出区域，通常返回 `None`。

```
def square_show(x):
    print(x ** 2)


def square_return(x):
    return x ** 2

shown = square_show(3)
returned = square_return(3)
print(shown, returned)
```

第 8 行先打印 9，然后因函数没有 return 而把 shown 设为 None。第 9 行不自动打印，returned 获得 9。最后输出 None 9。若后续还要平均、比较或保存结果，就应返回数据，而不是只打印。

函数可以返回多个量，实际常为一个元组：

```
def limits(values):
    return min(values), max(values)

low, high = limits([3, 8, 5])
print(low, high)
```

返回 (3, 8)，调用者通过解包获得两个值。当前函数要求输入非空，否则 min、max 不能定义结果，后面会加边界检查。

位置参数、关键字参数与默认值

位置参数按顺序匹配，predict_cost(2.5, 5.0, 3.0) 等于把重量、起步价、单价分别传入。关键字参数通过名称匹配，适合有多个含义相近的数值时使用。常规调用中位置参数要先于关键字参数，不能把同一个形参重复赋值。

默认值在函数定义时求值。数值、字符串和 None 常可安全作为默认配置；可变列表默认值会被多次调用共享，容易引发累积：

```
# Avoid this pattern for a fresh list per call.
def collect_bad(value, bucket=[]):
    bucket.append(value)
    return bucket
```

连续调用 collect_bad(1)、collect_bad(2) 后，两次都使用同一个默认列表，第二次结果为 [1, 2]。若每次需要新列表，可使用

```
def collect(value, bucket=None):
    if bucket is None:
        bucket = []
    bucket.append(value)
    return bucket
```

`is None` 检查是否为那个特殊对象，比把 `None` 混入普通数值比较更清楚。若调用者主动传入列表，这个函数会修改该列表；这应当记录在接口说明中。引用与副作用的深入分析放在第 3 章。

局部名称与作用域

作用域（scope）规定名称在什么位置可见。函数内的参数和通常赋值的名称属于该次调用的局部作用域。相同名称在函数内外可以代表不同绑定：

```
x = 10

def add_one(x):
    result = x + 1
    return result

y = add_one(3)
print(x, y)
```

函数内部的 `x` 为 3，返回 4；外部 `x` 仍为 10，输出 `10 4`。局部 `result` 不能在函数之外直接读取。

尽量把函数所需数据作为参数传入，而不是依赖外部可变状态。若函数在内部写 `threshold`，但它既非参数也非局部定义，就要向外寻找名称，程序行为会受外部环境影响。纯计算函数如 MAE 可以只由输入决定结果，便于手算和重复核对。

赋值局部名称和修改传入对象不同。例如 `values = []` 只是使局部名称改指向新列表，`values.append(3)` 却修改了调用者可能持有的原列表。本章指标函数不修改输入，数据清理函数创建新列表，这使实验更容易追踪。

例题 2.6：建立可靠的均值函数

设计 `average(values)`，对非空数值列表返回平均值，空列表返回 `None`。用三个输入核对，并解释分支为什么放在除法之前。

```
def average(values):
    if not values:
        return None
    total = 0.0
    for value in values:
        total += value
    return total / len(values)

print(average([2, 4, 6]))
print(average([0]))
print(average([]))
```

解 输出 4.0、0.0、`None`。`if not values` 检查列表是否为空，`[0]` 仍是非空列表，不会误判成空输入。空输入提前返回，避免除以零，也明确表示平均值未定义。该函数

假定非空输入每项为可进行数值加法的数，不负责解析任意文本。

2.12 列表推导式、排序与函数作为输入

从普通循环到列表推导式

列表推导式（list comprehension）把逐项变换和筛选写成紧凑形式。先看循环：

```
squares = []
for value in [1, 2, 3, 4]:
    squares.append(value ** 2)
```

等价写法为

```
squares = [value ** 2 for value in [1, 2, 3, 4]]
```

结果是 `[1, 4, 9, 16]`。阅读时先看 `for` 从哪里取值，再看前面的表达式怎样变换。加筛选可写 `[v ** 2 for v in values if v >= 0]`，含义是先保留非负项，再平方。

`["pass" if v >= 60 else "fail" for v in values]` 则对每一项选择一种输出，不丢弃元素。这里的条件表达式位于 `for` 前；位于末尾的 `if` 是过滤。两者结果长度可能不同，数据配对时不能混淆。

复杂的多层条件或需要打印中间过程时，用普通循环更容易阅读和排错。推导式减少行数并不自动提高算法效率，也不会把 Python 循环变成 NumPy 向量化。

生成器表达式与 sum

`sum(values)` 计算数值和；`sum(v ** 2 for v in values)` 在求和时逐项产生平方，不必先保存全部平方列表。括号中的形式称为**生成器表达式**，这里可以理解为向 `sum` 按需提供各项。

在 `sum(v >= 60 for v in values)` 中，每次比较得到布尔值，数值求和时 `True` 按 1、`False` 按 0 处理，因此得到及格人数。这是一个有意使用的计数约定，应知道它与 `bool` 的数值行为有关，而不是把真假值本身当成成绩。

表达式 `sum(...)` 遍历一次输入，时间随项数线性增长；若使用列表推导式，还要保存一份结果列表。小实验中差异很小，但理解是否创建中间列表有助于后续数据规模扩大时判断空间成本。

sorted、sort 与排序键

`sorted(values)` 返回新的排序列表，原对象不变；`values.sort()` 在原列表中排序，返回 `None`。这与 `append` 的原地修改语义一致。

```
values = [3, 1, 2]
ordered = sorted(values)
print(values)
print(ordered)
```

输出 `[3, 1, 2]` 和 `[1, 2, 3]`。若排序的是记录，需要说明根据哪个字段：

```
records = [(1.0, 6.0), (3.0, 10.0), (2.0, 8.0)]

def get_weight(row):
    return row[0]

ordered = sorted(records, key=get_weight)
print(ordered)
```

key 接收函数，排序时用它为每条记录计算比较键。本例按重量得到 1.0、2.0、3.0 的顺序，而不是把重量和费用分别排序。

简短的 `lambda row: row[0]` 等价于此处的取重量函数，但没有独立名称。实验 1.1 中的 `lambda row: abs(row[0] - x)` 计算与查询重量的距离。Python 排序是稳定的，键相同的记录保留原来的相对次序；因此可以据此制定明确的距离同分规则。

函数作为参数与第一章代码回读

函数对象也可以作为输入。下面指标函数接收一个预测规则：

```
def constant_predict(x):
    return 8.0


def evaluate_mae(records, predict):
    total = 0.0
    for x, y in records:
        y_hat = predict(x)
        total += abs(y_hat - y)
    return total / len(records)

result = evaluate_mae([(1, 6), (2, 8)], constant_predict)
print(result)
```

第二个参数传入 `constant_predict` 函数本身，不是 `constant_predict(...)` 调用结果。指标函数每次拿到 x 后调用 `predict(x)`，因此不必知道预测规则如何实现。输出为 $(|8-6|+|8-8|)/2 = 1.0$。

实验 1.1 在候选循环内部使用 `lambda x: neighbor_predict(train, x, k)`，把当前 `train` 和 `k` 与预测规则结合起来，并立即执行 MAE 计算。这里没有把多个延迟执行的匿名函数存入列表，所以不会遇到以后执行时才读取循环变量的另一类问题。若后续需要长期保存多个函数，应重新检查绑定语义，第 3 章将讨论相关组织与调试。

学习完本节，可按四个问题回读第一章代码：记录是什么容器；排序键返回什么；切片取哪几项；求和与分母对应哪个数学集合。能回答这些问题，便能独立核对近邻预测和指标计算的主要逻辑。

2.13 逐行执行、错误定位与计算约定

用状态表检查程序

对每个循环记录当前输入、条件判断和累积量，能把“看懂大致意思”转为可验证的执行分析。设有成绩 `[82, 59, 0, 91]`，只统计达到 60 分的成绩平均值：

```
scores = [82, 59, 0, 91]
total = 0
count = 0
for score in scores:
    if score >= 60:
        total += score
        count += 1
if count > 0:
    mean = total / count
else:
    mean = None
print(total, count, mean)
```

条件决定哪些样本进入分子和分母，两者必须一致。

当前成绩	是否纳入	更新后 `total`	更新后 `count`
82	是	82	1
59	否	82	1
0	否	82	1
91	是	173	2

最终输出 `173 2 86.5`。若分母错误使用 `len(scores)`，输出 43.25，实际上将未纳入的两条记录也计入分母。数字看起来合法，但统计意义错误。

例题 2.7：累加器初始化的位置

下面程序想计算所有平方和，却把初始化放在循环内部。预测输出，然后改正。

```
values = [1, 2, 3]
for value in values:
    total = 0
    total += value ** 2
print(total)
```

解 每次循环都先把 `total` 重置为 0，随后只加当前项，所以三轮结束时分别为 1、4、9，最终输出 9。应将 `total = 0` 移到循环之前，得到 $1 + 4 + 9 = 14$。如果原输入为空，错误程序还会在打印时遇到名称未定义，修正后平方和自然为 0，但这与平均数空输入未定义的规则不同。

语法、运行与逻辑错误

语法错误表示程序无法按 Python 规则解析，例如遗漏冒号、括号未配对或缩进结构不合法。**运行时错误**表示语法可以解析，但执行时遇到不能完成的操作，例如除以零、读取缺失键或不同类型相加。**逻辑错误**表示程序运行完成，却实现了错误的规则，例如把平方误差写成普通误差。

读报错时先找到最后的异常类型和说明，再看指向的文件与行号，并检查该行使用的变量。一个错误行可能由前面不正确赋值引起，例如 `scores = scores.append(88)` 本身可执行，直到后面对 `scores` 求长度才失败。需要追踪数据如何来到错误行，而不是只修改报错处。

对逻辑错误，应使用能揭示差异的小输入。检查 MAE 时，不只用全正有符号误差，因为忘写绝对值可能偶然同样正确；应包含高估和低估。检查循环边界时，要包含恰好等于阈值的输入。检查分母时，要让被筛选数量与原数量不同。这样的核对比大量随机打印更能回答具体疑问。

接口约定与有限输入检查

函数先说明接受什么输入、返回什么结果、怎样处理不合法输入。对于 MAE 函数，本章采用约定：两个数值序列长度相同且非空；错误输入主动报 `ValueError`；不修改输入。主动报错的语法为 `raise ValueError(" 说明")`，它会结束正常执行并传播错误。第 3 章讨论调用者怎样捕获与处理。

```
def mean_absolute_error(actual, predicted):
    if len(actual) != len(predicted):
        raise ValueError("lengths differ")
    if not actual:
        raise ValueError("inputs are empty")
    total = 0.0
    for y, y_hat in zip(actual, predicted):
        total += abs(y_hat - y)
    return total / len(actual)
```

前五行在任何平均计算之前核对结构。随后 `zip` 恰好覆盖全部配对，分母与误差项数一致。它仍假定元素是合法有限数值，不是任意混合对象；更完整的数值检查要按实际任务添加。

若希望空输入返回 `None`，也可以设计另一种接口，但调用者就必须处理这一返回值。实验 2.1 的统计函数采用 `None`，实验 2.2 的指标函数采用主动拒绝，两者各自明确而一致；不能在同一函数中偶尔返回 0、偶尔返回 `None`，却不说明规则。

例题 2.8：正负误差抵消

真实值为 `[5, 5]`，预测为 `[7, 3]`。某函数直接平均 `y_hat - y` 并将结果称为 MAE。给出反例计算与修正。

解 有符号误差为 $(2, -2)$，平均为 0；每项预测都错了 2，MAE 应为 $(2+2)/2 = 2$。修

正累加项为 abs(y_hat - y)。平均有符号误差可以描述总体高估或低估倾向，但不能衡量错误大小，应该使用不同名称并解释含义。

2.14 实验 2.1：从原始文本到统计报告

任务与规则

入口为 ch02_basics.py。输入为九条固定文本：

```
raw = [" 82 ", "91", "", " 76", "absent", "88", "0", "105", "-3"]
```

任务是从文本中提取 0–100 范围内的非负整数成绩，保留原顺序，并记录每条被拒绝记录的下标、原始文本和原因。然后给出有效数量、均值、最小值、最大值与及格比例。该输入不是实际学生数据，仅演示文本处理和统计规则。

输入规则明确为：只接受由 ASCII 十进制数字组成的非空文本，可以带两端空白；不接受正负号、小数点或单位；转为整数后还需满足范围。因而"-3" 在文本规则阶段被拒绝，"105" 在范围阶段被拒绝，"0" 被保留。拒绝原因的先后由检查顺序决定，不意味着负值没有范围问题。

清理循环与逐条追踪

```
clean = []
rejected = []
for index, text in enumerate(raw):
    stripped = text.strip()
    digits = stripped.isascii() and stripped.isdecimal()
    if not digits:
        rejected.append((index, text, "invalid text"))
        continue
    value = int(stripped)
    if not 0 <= value <= 100:
        rejected.append((index, text, "out of range"))
        continue
    clean.append(value)
```

strip 只去除两端空白，保留原始 text 用于报告。isdecimal 判断是否为非空十进制数字文本；再结合 isascii 将范围限定到本实验接受的 ASCII 数字，避免把 Unicode 数字形式混入该简化接口。通过文本检查后，int 转换才有明确依据。

两处 continue 把拒绝记录从后续处理排除；最后 append 只有在两次检查均通过时执行。rejected 保存三项元组，数据来源与原因可以逐条定位。

下标	原始文本	处理后文本	结果
0	`" 82 "`	`"82"`	保留 82
1	`"91"`	`"91"`	保留 91
2	`""`	`""`	拒绝：空串不符合数字规则
3	`" 76"`	`"76"`	保留 76
4	`"absent"`	`"absent"`	拒绝：不是数字文本
5	`"88"`	`"88"`	保留 88
6	`"0"`	`"0"`	保留 0
7	`"105"`	`"105"`	拒绝：超过范围
8	`"-3"`	`"-3"`	拒绝：含负号

有效数据为 `[82, 91, 76, 88, 0]`，和为 337，平均 67.4；及格有 4 项，及格比例 0.8。若错误地丢弃 0，平均变成 84.25，及格比例变成 1.0。这会改变报告含义，而不是仅改变程序风格。

统计函数与运行结果

```
def describe(values):
    if not values:
        return None
    count = len(values)
    average = sum(values) / count
    passed = sum(value >= 60 for value in values)
    return {
        "count": count,
        "mean": average,
        "minimum": min(values),
        "maximum": max(values),
        "pass_rate": passed / count,
    }
```

函数只接受已经清理的数值列表。数量、均值和及格比例共享同一个有效总体，字段用字典命名，方便调用者读取。空输入返回 `None`，避免未定义统计量被伪装成正常 0 值。

运行 `python3 ch02_basics.py`，应打印原始记录、有效列表和四条拒绝记录。统计部分为

```
count = 5
mean = 67.4
minimum = 0
maximum = 91
pass_rate = 0.8
empty input: None
```

列表、计数与拒绝下标应完全一致；小数值与手算差异应小于 10^{-9}。完整英文错误原因见脚本和随附输出，不影响这里的规则说明。

修改任务

将`"105"`改成`"100"`，应新增一个有效值，数量 6，和 437，均值 $437/6 \approx 72.833333$，及格比例 5/6，最大值 100。再将`"absent"`改成`"59"`，应在原实验基础上得到数量 6、和 396、均值 66.0、及格比例 4/6。两项修改应分别从原始输入开始，避免混淆对照条件。

将原始列表改成全空串，确认统计返回 `None`。此时没有统计字典，不能继续遍历其字段。完整脚本先判断 `summary is None`，全无效输入时打印 `no valid scores`；只有获得统计字典后才遍历字段。这说明函数正确处理边界后，调用程序也必须遵守它的接口。

最后把清理逻辑整理成 `clean_scores(raw)` 函数，返回 `clean, rejected`，保留原输入不变。习题 2.15 提供核对方案。不要只返回有效值而丢弃拒绝原因，否则报告难以追踪数据变化。

2.15 实验 2.2：预测误差与模型比较

数据、函数与数学对应

入口为 `ch02_metrics.py`。四条真实值为 $(6, 8, 10, 12)$，两个模型的固定预测分别为 A：$(5, 9, 10, 14)$，B：$(6, 8, 10, 16)$。实验不训练模型，只把已有预测变成可解释的评价报告。

先手算 A 的误差为 $(-1, 1, 0, 2)$，MAE 为 1，MSE 为 1.5，RMSE 为 $\sqrt{1.5} \approx 1.224744871$。B 的误差为 $(0, 0, 0, 4)$，MAE 同样为 1，MSE 为 4，RMSE 为 2。两个模型在 MAE 下打平，平方指标更偏好 A。

完整脚本把长度和空输入检查提取为 `validate_pairs`，两个指标函数共用。平方指标函数为

```
def mean_squared_error(actual, predicted):
    validate_pairs(actual, predicted)
    total = 0.0
    for y, y_hat in zip(actual, predicted):
        total += (y_hat - y) ** 2
    return total / len(actual)
```

第 4 行取得一对标签和预测，第 5 行对应 $(\hat{y}_i - y_i)^2$，第 6 行对应 $1/m$。RMSE 不需重新遍历数据，在 MSE 结果上求平方根即可。

字典遍历、最坏误差与输出

两个模型预测保存在字典，按键分别评价：

```
predictions = {"A": [5.0, 9.0, 10.0, 14.0],
               "B": [6.0, 8.0, 10.0, 16.0]}
for name, predicted in predictions.items():
    validate_pairs(actual, predicted)
    errors = [p - y for y, p in zip(actual, predicted)]
    absolute = [abs(error) for error in errors]
    worst_index = absolute.index(max(absolute))
    mae = mean_absolute_error(actual, predicted)
    mse = mean_squared_error(actual, predicted)
    print(name, mae, mse, worst_index)
```

`items()` 提供键值对，`name` 为模型名，`predicted` 为一组预测。`max` 找到最大绝对误差，`index` 返回第一次出现该值的下标；若有并列最大项，当前报告只记录第一项，不能宣称它是唯一最坏项。

运行 `python3 ch02_metrics.py`，主要结果为

```
model: A
errors: [-1.0, 1.0, 0.0, 2.0]
MAE=1.000000 MSE=1.500000 RMSE=1.224745
worst index: 3
model: B
errors: [0.0, 0.0, 0.0, 4.0]
MAE=1.000000 MSE=4.000000 RMSE=2.000000
worst index: 3
```

脚本还核对完全正确预测、单样本预测，并确认空输入和长度不同输入被拒绝。错误路径的捕获使用 `try/except`，完整语义在第 3 章展开，此处只用于让脚本能继续报告“非法输入已拒绝”。

修改任务与比较限制

把 B 最后一项 16.0 改成 14.0，则 B 误差变成 $(0, 0, 0, 2)$，MAE 为 0.5，MSE 为 1，RMSE 为 1。随后恢复原始预测，再给两个模型加入同一条真实值 20，A 预测 21，B 预测 24，分别手算新指标。注意分母由 4 变成 5，不能只追加分子而忘记改变样本数。

删除某个模型最后一项，程序应拒绝不等长输入。为报告所有并列最坏项，可把 `worst_index = absolute.index(max(absolute))` 改为先保存 `maximum = max(absolute)`，再通过 `enumerate` 筛出所有误差等于最大值的下标。对复杂浮点计算是否采用近似并列，应另外明确容差，本例误差为简单整数无需扩大概念。

本实验中的固定预测不证明哪个算法普遍更好。指标比较只能在同一数据与评价

协议下解释。若随后按本数据不断修改模型，这组数据就参与了选择；真实项目仍需保留未参与选择的评价数据。

2.16 代码风格、知识联系与自检

基础程序采用四个空格缩进，一个名称保持一个稳定含义，变量名表达数据角色和必要单位。长表达式可以先求 `error`、再求 `absolute_error`，避免过度紧凑。函数优先返回数据，输出在调用层集中处理，便于重复使用和核对。

容器决定数据怎样组织，循环决定哪些项参与运算，函数决定接口和边界，条件决定有效性和决策规则。它们与第一章直接联系：记录对保证特征标签对齐，长度检查保证指标分母正确，切片帮助理解划分边界，字典帮助组织评价报告，排序键帮助实现近邻查询。

本章没有引入数组广播、类继承、异步程序或大型框架。下一章处理文件、模块、异常和引用，随后 NumPy 把逐项数值运算改写为数组运算。语法越简洁，越需要知道输入形状和运算对象，不能仅凭代码行数判断自己已经理解算法。

自检时应能够预测一个含分支循环程序的输出，指出终端命令与 Python 语句的区别，对空输入写出明确规则，并将一条数学求和式对应到循环中的累加项与分母。最后能在不改变数据的情况下运行前两章合计四个配套实验，再逐项解释修改后的结果。

2.17 分层习题

2.1 **运行。**脚本 `hello.py` 含 `2 + 3` 和 `print(2 + 3)` 两行。运行时输出几次 5？在 Python 交互环境中能否直接输入 `python3 hello.py` 运行？说明如何在含空格目录中正确执行脚本。

2.2 **追踪。**依次执行 `a = 2`、`b = a`、`a += 3`、`b *= a`、`a, b = b, a`。写出每行后的 a, b 与最终输出。

2.3 **计算。**求 `7 / 2`、`7 // 2`、`-7 // 2`、`-7 % 2`、`-2 ** 2` 和 `(-2) ** 2`。为什么 `int(-3.5)` 与 `-7 // 2` 不同？

2.4 **数值。**为什么 `0.1 + 0.2 == 0.3` 可能为假？给出用于核对该计算的容差方法；能否靠格式化保留两位小数让所有后续计算精确？

2.5 **字符串。**将字符串 `" 82,91,0 "` 转为三个整数。若直接拆分文本后交给 `sum`，为什么失败？这里哪些文本检查可因固定输入假设省略，推广后又需补上？

2.6 **条件。**对成绩 0–100，90 及以上为 A，60–89 为 B，其余为 C，范围外为非法。写出分支，核对 $-1, 0, 59, 60, 89, 90, 100, 101$。

2.7 **切片。**给定 `v = [0, 1, 2, 3, 4, 5, 6]`，求 `v[1:6:2]`、`v[-3:]`、`v[::-1]` 和 `v[4:2]`。与 `v[20]` 相比，`v[:20]` 有什么区别？

2.8 **容器。**解释 `[1, 2].append([3, 4])` 与 `extend` 对原列表结构的不同影响；为什么不能把 `append` 返回值赋回原列表名？空字典与空集合分别怎样写？

2.9 **循环。**不使用 `sum`，用循环计算 1 至 n 的整数和。规定 $n \geq 0$，核对 0、1、5；说

明为什么终点要写 $n+1$。

2.10 while。从 `x = 1` 开始，每轮令 `x = 2 * x + 1`，直到 `x >= 20`，统计轮数。给出代码和每轮状态。若输入上界是 1，循环是否执行?

2.11 字典与集合。对 `labels = ["cat", "dog", "cat", "bird", "dog", "cat"]`，用字典统计频数，不预设类别列表。再给出不同标签集合，并说明为什么集合不能保存原位置。

2.12 函数。编写成绩分类函数 `classify`，参数为成绩和默认值 60 的阈值。合法成绩按阈值返回`"pass"` 或`"fail"`，范围外返回 `None`。比较成绩 70 在默认阈值与阈值 80 下的结果。解释 `return` 与 `print` 的区别。

2.13 指标。编写等长非空数值输入上的 MAE 与 MSE 函数，不修改输入。核对真实值 `[1, 3, 5]`、预测 `[2, 1, 5]`，并验证空输入和长度不同被拒绝。

2.14 推导式。将非负项筛选后平方的循环改写成列表推导式。给定 `[-2, 0, 3, -1, 4]`，给出结果。再用一个条件表达式将每项标记为`"valid"` 或`"invalid"`，说明两种结果长度不同的原因。

2.15 实验整理。把实验 2.1 清理部分封装成 `clean_scores(raw)`，返回有效值和拒绝记录，不修改输入。编写调用代码，在全无效输入时显示`"no valid scores"`，否则打印数量与均值。

2.16 综合。训练记录为 `[(1, 5), (2, 7), (4, 11)]`，评价记录为 `[(1.2, 5.5), (3.5, 10)]`。编写只使用训练记录的最近一个邻居预测函数和 MAE 评价，给出预测与误差。实现应检查训练非空；按稳定排序处理距离同分。

2.18 习题参考答案

2.1–2.4：运行、赋值与数值

2.1 只输出一次 5，第一行的表达式没有 `print`，在脚本中不自动显示。交互环境中的 `python3 hello.py` 不是 Python 语句，应先 `exit()` 返回普通终端，再在文件所在目录执行。含空格目录用引号包围，如 `cd "/path/with space"`，也可以给脚本提供带引号的绝对路径。若编辑器运行按钮使用另一解释器，要核对实际环境。

2.2 第 1 行 $a=2$，b 尚未定义；第 2 行 $a=2, b=2$；第 3 行 $a=5, b=2$；第 4 行 $a=5, b=10$；交换后 $a=10, b=5$。整数绑定彼此独立，修改 a 不会自动改写 b 的过去结果。最终打印 `10 5`。

2.3 结果依次为 3.5、3、−4、1、−4、4。向下取整除法将 −3.5 变成 −4，`int(-3.5)` 朝零截断为 −3。幂先于无括号的一元负号，故`-2 ** 2` 是 $-(2^2)$；给底数加括号后才是 $(-2)^2$。

2.4 常见浮点表示不能精确存储 0.1 和 0.2，运算后与 0.3 的存储近似值有微小差异。可以 `math.isclose(0.1 + 0.2, 0.3, rel_tol=1e-9, abs_tol=1e-12)` 核对。格式化只生成显示字符串，不会使原值或后续运算变成精确十进制计算。若问题要求精确货币规则，需选择适当数值表示和舍入协议，而不是任意增加显示位数。

2.5–2.8：文本、边界与容器

2.5 固定输入可写

```
raw = " 82,91,0 "
values = [int(part.strip()) for part in raw.split(",")]
print(values)
```

结果 `[82, 91, 0]`。`split` 产生字符串列表，`sum` 的默认初值是数值 0，不能将 0 与字符串相加；即使文本拼接成功，也不等于数值求和。本题固定输入假定各项都是合法整数；任意输入应检查空项、数字格式和成绩范围，记录拒绝原因。

2.6 先排除非法范围，再从高阈值向低阈值判断：

```
def grade(score):
    if not 0 <= score <= 100:
        return None
    if score >= 90:
        return "A"
    if score >= 60:
        return "B"
    return "C"
```

八个核对结果为 None、C、C、B、B、A、A、None。函数中的 `return` 会立即结束，故两个独立 `if` 在这里可正确工作；若改成多次赋值且不提前返回，就要重新检查覆盖问题。

2.7 结果分别为 `[1, 3, 5]`、`[4, 5, 6]`、`[6, 5, 4, 3, 2, 1, 0]` 和空列表。`v[4:2]` 默认步长为正，从 4 不能在小于 2 的边界内前进，故为空。`v[:20]` 截取到可用末尾并返回完整列表，`v[20]` 单项访问越界会报错。

2.8 `append([3, 4])` 把二项列表当成第三个元素，结果为 `[1, 2, [3, 4]]`；`extend([3, 4])` 逐项加入，结果为 `[1, 2, 3, 4]`。两种方法均修改原列表并返回 None，不能把返回值误当新列表。空字典为 `{}`，空集合为 `set()`。题中用列表字面量调用方法后，如果没有保存这个列表，就不能通过名称继续读取它，因此实际追踪时应先绑定变量。

2.9–2.12：循环、映射与函数

2.9

```
def integer_sum(n):
    if n < 0:
        raise ValueError("n must be nonnegative")
    total = 0
    for i in range(1, n + 1):
        total += i
    return total
```

输入 0 时范围为空，返回 0；输入 1 返回 1；输入 5 返回 15。`range` 不含终点，要

包含 n 必须将终点设为 $n+1$。本函数假定 n 为整数，若接收任意对象，还需另外检查类型。

2.10

```
x = 1
steps = 0
limit = 20
while x < limit:
    x = 2 * x + 1
    steps += 1
print(x, steps)
```

每轮结束 x 为 3、7、15、31，最终 4 轮。上界为 1 时，初始 `x < limit` 就是假，不执行，输出 `1 0`。停止条件应在循环入口核对，不能先假定至少执行一次。

2.11

```
labels = ["cat", "dog", "cat", "bird", "dog", "cat"]
counts = {}
for label in labels:
    counts[label] = counts.get(label, 0) + 1
unique = set(labels)
print(counts)
print(sorted(unique))
```

频数 cat 为 3、dog 为 2、bird 为 1，不同标签排序显示为 `['bird', 'cat', 'dog']`。`get` 为未出现标签提供初始 0；此处 0 确实是合理频数。集合只保留不同元素，不记录同一标签第几次出现，因而不能替代原标签序列与特征配对。

2.12

```
def classify(score, threshold=60):
    if not 0 <= score <= 100:
        return None
    if score >= threshold:
        return "pass"
    return "fail"
```

两个调用依次返回`"pass"` 和`"fail"`。这里还假定阈值为合法数值，若作为外部配置接收，可增加阈值范围检查。`return` 为调用者提供可继续使用的对象；`print` 只显示，并不把显示内容作为返回值交给调用者。

2.13–2.16：计算与综合程序

2.13 两个函数可以共用输入核对：

```
def check_pairs(actual, predicted):
    if len(actual) != len(predicted) or not actual:
        raise ValueError("equal nonempty inputs required")


def mae(actual, predicted):
    check_pairs(actual, predicted)
    return sum(abs(p - y) for y, p in zip(actual, predicted)) / len(actual)


def mse(actual, predicted):
    check_pairs(actual, predicted)
    return sum((p - y) ** 2 for y, p in zip(actual, predicted)) / len(actual)
```

有符号误差为 $(1, -2, 0)$，MAE 为 1，MSE 为 $5/3$。分别以 `([], [])` 和 `([1], [1, 2])` 调用应产生 `ValueError`，而不是返回一个误导性的数。可与实验 2.2 相同的错误捕获方式核对；输入列表应在调用前后保持相同内容。

2.14

```
values = [-2, 0, 3, -1, 4]
squares = [v ** 2 for v in values if v >= 0]
marks = ["valid" if v >= 0 else "invalid" for v in values]
print(squares)
print(marks)
```

平方结果为 `[0, 9, 16]`，长度 3。标记结果依次为 invalid、valid、valid、invalid、valid，长度 5。第一种筛选删除不满足条件的项，第二种对每个原始项都产生一个结果。

2.15 完整可运行参考答案还保存在 `ch02_solutions.py` 中。核心函数如下：

```
def clean_scores(raw):
    clean = []
    rejected = []
    for index, text in enumerate(raw):
        stripped = text.strip()
        if not (stripped.isascii() and stripped.isdecimal()):
            rejected.append((index, text, "invalid text"))
            continue
        value = int(stripped)
        if not 0 <= value <= 100:
            rejected.append((index, text, "out of range"))
            continue
        clean.append(value)
    return clean, rejected
```

调用端先解包结果，再检查是否有有效值，最后决定是否计算均值：

```
values, rejected = clean_scores(["82", "", "0"])
if not values:
    print("no valid scores")
else:
    print(len(values), sum(values) / len(values))
```

当前输入有效值为 `[82, 0]`，数量 2，平均 41.0，拒绝下标 1。若输入两项均无效，则输出规定文本。函数新建结果列表，没有修改 `raw`；可保存调用前副本并比较调用后原列表，作为有限示例核对。

2.16

```
def nearest_one(train, x):
    if not train:
        raise ValueError("training data are empty")
    ordered = sorted(train, key=lambda row: abs(row[0] - x))
    return ordered[0][1]

train = [(1, 5), (2, 7), (4, 11)]
evaluation = [(1.2, 5.5), (3.5, 10)]
errors = []
for x, y in evaluation:
    prediction = nearest_one(train, x)
    errors.append(abs(prediction - y))
    print(x, prediction)
print(sum(errors) / len(errors))
```

对 1.2，距离 0.2、0.8、2.8，选 $(1,5)$，预测 5，绝对误差 0.5。对 3.5，距离 2.5、1.5、0.5，选 $(4,11)$，预测 11，绝对误差 1。平均 0.75。预测函数只接收训练记录与新输入，未读取评价标签；评价列表在本题中已保证非空，通用评价函数仍需加非空检查。距离同分时稳定排序保留原训练顺序。

2.19 本章小结

Python 脚本按控制流程执行，赋值绑定名称，表达式产生值，条件决定分支，循环逐项或按状态重复，函数用明确的输入输出封装计算。列表、元组、字典和集合对应不同的数据组织需求，不能只凭都能装多个元素而互相替换。

数值计算必须保持运算顺序、样本配对、分子分母和单位一致；浮点核对使用合理容差；无效输入和空数据需要明确处理。完成本章后，第一章的教学实验不再只是运行入口，而应成为可以逐行解释、手算核对和有控制地修改的程序。

3 Python 程序组织与调试

先修知识、学习目标与本章位置

本章以第 2 章的函数、列表、字典、条件与循环为基础，不要求软件工程经验。学习后应能把程序拆成职责明确的模块；辨认导入路径和数据路径；安全地读写文本、CSV 与 JSON；解释类、实例、方法、引用和复制；根据报错与最小测试定位错误；组织一次可从头复现的训练与评价。

本章侧重程序结构，预测模型仍采用第 1 章的均值基线和单近邻规则。第 4 章将把列表循环改写成数组计算，第 25 章与第 44 章再扩展到深度学习和较完整的工程项目。

一个计算 MAE 的函数可以只有十行，但一个完整实验还需要知道数据从哪里读取、怎样核对字段、哪个对象保存训练结果、结果写到哪里，以及失败时如何找到原因。若这些工作混在一个长脚本里，修改输入路径就可能影响训练，调整打印格式也可能意外改变计算。程序组织的目的，是使每一步的输入、输出和状态可以分别解释和检查。

本章先把“一个文件中的若干函数”扩展为“几个可以互相导入的文件”，再讨论对象与状态。后半章将这些机制放回机器学习流程：数据读取完成后才允许训练；验证选择结束后才使用最终测试标签计算指标；保存报告时同时记录数据标识和选择规则。数据均为人为构造的教学数据。

3.1 从脚本到模块：代码如何被复用

模块、名称空间与导入

模块（module）是可以被导入的代码组织单元。最常见的形式是一个`.py`文件，例如`metrics.py`。**名称空间**（namespace）保存名称与对象之间的对应关系；不同模块可以各自定义`mean`而不直接覆盖彼此。导入语句让当前模块获得访问另一个模块对象或其中某些名称的途径。

在同一目录保存`metrics.py`：

```
def mae(actual, predicted):
    if len(actual) != len(predicted) or len(actual) == 0:
        raise ValueError("expected equal, nonempty sequences")
    return sum(abs(p - y) for y, p in zip(actual, predicted)) / len(actual)
```

再保存`run.py`：

```
import metrics

score = metrics.mae([5, 5], [7, 3])
print(score)
```

运行`python3 run.py`得到 2.0。`metrics.mae`中的点表示先取得`metrics`所指模块，

再访问其中名为mae的对象。与第 2 章的list.append一样，点号用于属性访问；属性可以是函数，也可以是数据。

常见导入方式的区别如下：

语句	当前文件得到的名称	调用写法
`import metrics`	`metrics`	`metrics.mae(...)`
`import metrics as mt`	`mt`	`mt.mae(...)`
`from metrics import mae`	`mae`	`mae(...)`

别名只改变当前文件使用的名称，不复制模块，也不改变磁盘文件名。from metrics import mae并不会同时定义metrics这个名称。使用from metrics import *会把许多名称引入当前文件，使来源不清楚，本讲义不用这种形式。

导入标准库不需要另外安装。例如import math导入数学功能，from pathlib import Path导入路径类。第三方包则必须存在于当前解释器的环境中。import numpy失败时，应检查第 2 章的虚拟环境与依赖，不应把文件名改成numpy.py来“补齐模块”。

导入时会发生什么

导入不只是查看函数名称。通常第一次导入一个模块时，解释器会执行该模块的顶层语句，包括函数和类的定义、顶层赋值以及没有放入函数的打印和文件操作。def语句创建函数，但函数体等到调用时才执行。若在模块顶层直接写train_model()，导入也会启动训练。

在同一解释器进程中，已导入模块通常会被缓存，普通重复导入不会把全部顶层代码重新执行一次。修改文件后，交互环境可能仍持有旧模块；本书的初学实验采用重新启动脚本的方法，确保从当前文件重新开始，避免把“保存了新代码”误认为“正在执行新代码”。

定义：程序入口与主模块保护

直接运行的文件，其特殊名称__name__通常为"__main__"；被普通导入的文件，其__name__为模块名称。把运行流程放进main()，再以if __name__ == "__main__":保护，可以使文件同时支持直接运行和作为模块导入。

```
def main():
    print("experiment starts")

if __name__ == "__main__":
    main()
```

直接运行时条件成立，打印文字。被导入时函数仍然可用，但该分支不执行。双下划线名称是 Python 预定义机制的一部分，必须按原样拼写。入口保护只控制这一个分支，不会阻止分支之外的顶层语句执行。

例题 3.1：区分导入与直接运行

`helper.py`的内容如下。另一个文件只写`import helper`，会打印哪些内容？直接运行`helper.py`又会打印哪些内容？

```
print("loading")

def main():
    print("running")

if __name__ == "__main__":
    main()
```

解 新进程中首次导入时，第一行是顶层打印，因此输出`loading`；函数体没有被调用，不输出`running`。直接运行时先输出`loading`，随后入口条件成立，再输出`running`。若要求导入保持安静，应把第一行也移入`main`，而不是只在末尾添加保护语句。

查找路径、命名冲突与包的最小概念

模块查找路径是解释器寻找模块的位置集合，可通过`sys.path`查看。直接运行脚本时，脚本所在目录通常位于查找路径前部；标准库目录和当前环境中的第三方包目录也参与查找。它与后面打开数据文件时使用的当前工作目录不是同一个概念。

```
import sys
import metrics

print(sys.executable)
print(metrics.__file__)
```

两个`print`依次输出所用解释器的实际位置和当前导入模块的来源。若目录里有自建`random.py`、`json.py`或`numpy.py`，它可能遮蔽同名库。报错中出现“部分初始化模块”或预期属性不存在时，应核对模块来源，再给自建文件取不冲突的名称。

包（package）用于组织多个模块。例如一个目录中放置`__init__.py`，再包含`metrics.py`和`models.py`，就可以组成普通包。`import package_name.metrics`表示访问包内模块。Python 还支持其他包形式，本章不展开。`pip`安装的**分发包**是安装单位，安装名与导入名不保证相同；不能仅凭安装名猜出所有导入语句。

两个模块相互在顶层导入、又立即访问对方尚未建立的名称，可能形成**循环导入**。本章用单向依赖减少这一问题：入口导入数据、模型与指标模块，模型与指标不反向导入入口。共同的小函数可以放入独立模块，由需要的模块导入。

3.2 文件路径：数据位置与运行位置

绝对路径、相对路径与当前工作目录

绝对路径从文件系统的根位置确定文件，**相对路径**需要一个起点才能解释。`Path("data/parcels.csv")`通常相对于**当前工作目录**（current working directory，CWD），不是自动相对于写出这行代码的文件。编辑器、终端和其他程序启动同一脚本时，当前工作目录可能不同。

设项目位于`/tmp/demo`，入口为其中的`main.py`。从上层目录启动时执行：

```
python3 demo/main.py
```

此时相对数据路径从工作目录解释，两个位置分别是：

```
实际尝试读取：/tmp/data/parcels.csv
项目数据位置：/tmp/demo/data/parcels.csv
```

两者不同，所以项目中即使已有数据，也可能报找不到文件。

`pathlib`是标准库中的路径工具，`Path`对象表示路径并提供相关方法：

```
from pathlib import Path

print(Path.cwd())
base = Path(__file__).resolve().parent
data_path = base / "data" / "parcels.csv"
print(data_path)
print(data_path.is_file())
```

`__file__`在普通脚本中提供当前源文件路径；`resolve()`获得规范化的绝对位置并解析符号链接；`parent`取父目录。路径对象之间的`/`是拼接路径的运算，不是数值除法。`is_file()`检查当前位置是否为文件，但“刚才存在”不能保证稍后读取一定成功，实际打开仍需处理异常。

在交互解释器或某些笔记本中没有`__file__`，不能直接复制这一写法。本章配套实验是磁盘脚本，因而可以使用。符号链接会影响解析后的基准位置；实验未依赖符号链接，数据始终相对于真实脚本目录定位。

例题 3.2：同一脚本为什么换目录就失败

入口与数据的位置分别为：

```
入口：/tmp/demo/main.py
数据：/tmp/demo/data/a.csv
```

入口使用`open("data/a.csv")`读取。分别从项目目录与上层目录启动，解释结果，

并给出不依赖启动目录的写法。

解 从项目目录启动，相对路径指向实际数据；从`/tmp`启动，则指向`/tmp/data/a.csv`，若该文件不存在就失败。修正为先计算`base = Path(__file__).resolve().parent`，再用`base / "data" / "a.csv"`读取。导入能找到同目录模块，并不意味着数据相对路径也会采用该目录。

路径对象的常用操作

`path.name`取得含扩展名的文件名；`path.stem`取得去掉最后扩展名后的名称；`path.suffix`取得最后扩展名，例如`".csv"`；`path.parent`取得父目录。路径存在不等于内容格式正确，扩展名为 CSV 也不保证能读出期望字段。

输出目录需先建立：

```
output = base / "results"
output.mkdir(parents=True, exist_ok=True)
report_path = output / "summary.json"
```

`parents=True`允许创建缺失的上层目录；`exist_ok=True`允许目录已经存在，若同名对象是普通文件仍会失败。本章输出固定保存在项目的结果目录，重复运行会替换同名报告。需保留多组结果时，应使用明确的实验编号建立子目录，并记录每组配置，不能只依赖记忆辨认文件。

Windows 路径的分隔符与 macOS 不同，`Path`会按当前系统解释拼接。Python 字符串中的反斜线可能形成转义，如`"\n "`代表换行；Windows 手写路径可以使用原始字符串`r"C:\data \a .csv"`。本章优先采用`Path`拼接，目录含中文或空格不需要改名。

3.3 文本文件、编码与资源释放

文件对象与 with 语句

文件对象连接程序与打开的文件，记录读取或写入状态。`open`或`Path.open`建立连接后，操作系统资源需要释放。`with`使用**上下文管理器**（context manager）管理进入和退出时的操作；对普通文件，即使代码块抛出异常，也会在退出时关闭文件。

```
from pathlib import Path

path = Path("note.txt")
with path.open("w", encoding="utf-8") as stream:
    stream.write(" 重量\n 费用\n")

with path.open("r", encoding="utf-8") as stream:
    text = stream.read()
print(repr(text))
```

写模式`"w"`会在打开时截断已有文件，不能用它打开需要保留的原始数据。`"r"`读取；`"a"`在末尾追加，重复运行可能重复加入相同记录；`"x"`仅新建，已存在时失败。含

"b"的模式处理二进制字节，本节文本模式处理字符串。

编码（encoding）规定字符与字节之间的转换。显式使用 UTF-8 可减少不同机器默认编码的差异；用错误编码读取可能抛出`UnicodeDecodeError`，也可能得到乱码。不要用`errors="ignore"`无声丢弃字符来假装数据完好。文本中的`"\n "`表示换行，`write`不会自动补换行。

`repr(text)`返回适合观察转义和空白的字符串表示，因此上述输出可看到`' 重量\n 费用 \n '`。普通`print(text)`会实际换行。调试含空格的字段时，`repr`比直接打印更容易辨认空串和只含空白的串。

逐行读取、位置与大文件

`stream.read()`一次读入剩余全部文本，`readline()`读取下一行，`for line in stream`逐行遍历。文件对象有当前位置；一次`read()`读到末尾后，再次调用通常得到空串，不是文件内容突然消失。

```
with path.open("r", encoding="utf-8") as stream:
    for number, line in enumerate(stream, start=1):
        print(number, repr(line.rstrip("\n")))
```

这里`start=1`使显示行号从 1 开始；`rstrip("\n ")`去掉行尾换行，不等同于`strip()`去掉两端所有空白。是否允许去除空白取决于字段定义：成绩数字外的空格常可去除，文本内容中的空格却可能有意义。

若文件大小为 B 字节，一次完整读取通常需要与 B 同量级的内存；逐行迭代只需保留当前行及程序积累的结果。不过若每行都被追加到列表，最后仍会保存全部数据。逐行读取是一种访问方式，不自动等于低内存算法。

> **注意：保存成功与完整保存**
>
> `with`保证文件关闭，不保证一组多文件输出构成不可分割的整体。程序在写一半时终止，结果目录可能只含部分文件。本章把报告写入固定输出目录，适用于小型教学实验；严肃运行可先写临时文件，校验完成后替换目标，并用运行状态区分完成和失败。原始数据不应与输出使用同一路径。

3.4 CSV 与 JSON：给数据建立明确结构

CSV 的行、字段与表头

CSV（comma-separated values，逗号分隔值）是一类表格文本格式，通常每条记录一行、各字段用逗号分隔。字段本身可能包含逗号、引号或换行，需要引用和转义规则。因此不能把任意 CSV 都当成`line.split(",")`。

本章输入文件的开头为：

```
id,split,weight,cost
tr01,train,1.0,6.0
tr02,train,2.0,8.0
```

表头给出字段名称；id是记录标识；split说明所属集合；weight单位为千克；cost单位为元。id只用于追踪，不作为数值特征。读取 CSV 得到的"1.0"仍是文本，需显式转为数值。

```
import csv

with data_path.open("r", encoding="utf-8", newline="") as stream:
    reader = csv.DictReader(stream)
    print(reader.fieldnames)
    for row in reader:
        weight = float(row["weight"])
        print(row["id"], weight)
```

DictReader将每条记录映射为字典；reader.fieldnames给出表头。newline=""让 CSV 模块按自身规则处理换行，尤其避免某些系统上写入多余空行。文本转数值、范围检查、字段缺失检查仍是应用程序的责任。

输入验证：语法正确还不够

输入验证检查数据是否满足程序接口与任务要求。对本章 CSV，依次核对表头、字段数量、标识、集合名称和数值。三种集合都必须非空；标识在整个文件中唯一；重量和费用必须有限且非负。float("nan")和float("inf")可以转换成功，但前者表示非数值，后者表示无穷，均不符合本实验的测量值要求。

```
from math import isfinite

value = float("nan")
print(isfinite(value))
print(isfinite(float("inf")))
print(isfinite(0.0))
```

输出依次为False、False、True。只用value < 0不能排除 NaN，因为 NaN 参与通常的大小比较不符合普通实数规则。正确检查可以写isfinite(value) and value >= 0。

> **例题 3.3：一条可以转换的坏记录**
>
> CSV 中有a01,train,nan,6.0。程序只调用float，然后检查重量是否小于 0。为什么可能放过它？应在哪个步骤拒绝？

解 float("nan")返回特殊浮点值，转换没有抛异常；nan < 0为假，因此“若小于 0 就拒绝”的分支不执行。应在转换后验证有限性，并给出原记录位置。本章严格读取器拒绝整次实验，而不是悄悄删除记录，因为删除会改变已指定的划分和评价样本。

若数据任务明确允许丢弃坏行，也应保存拒绝原因、数量及原标识，并说明对结果的影响。第 2 章成绩清理采用“保留有效项并报告拒绝项”，本章项目采用“输入不合约定就停止”；两种策略适用于不同接口，不能在同一次运行中无说明地混用。

写出 CSV 使用`csv.writer`：

```
with report_path.with_suffix(".csv").open(
    "w", encoding="utf-8", newline=""
) as stream:
    writer = csv.writer(stream)
    writer.writerow(["id", "actual", "predicted"])
    writer.writerow(["te01", 6.8, 6.0])
```

`with_suffix`返回扩展名替换后的新路径，不修改原路径对象。`writerow`接收字段序列，负责 CSV 引用与分隔；`writerows`可以一次接收多条记录。

JSON 的结构与序列化

JSON（JavaScript Object Notation，JavaScript 对象表示法）是一种结构化文本格式，适合保存配置与汇总报告。常见对应关系为：Python 字典对应 JSON 对象，列表对应数组，字符串和数值对应相应值，`None`对应`null`，布尔值对应`true`或`false`。

序列化（serialization）把内存中的结构转换为可保存或传输的表示；**反序列化**恢复结构。JSON 文件保存的是数据，不会把 Python 类的方法一起保存下来。

```
import json

report = {"selected": "nearest", "test_mae": 0.85, "count": 4}
with report_path.open("w", encoding="utf-8") as stream:
    json.dump(report, stream, ensure_ascii=False, indent=2, allow_nan=False)
with report_path.open("r", encoding="utf-8") as stream:
    loaded = json.load(stream)
print(loaded["selected"])
```

`dump`写文件，`load`读文件；带字母`s`的`dumps`与`loads`分别生成字符串和解析字符串。`ensure_ascii=False`便于直接阅读中文，`indent=2`设置缩进，`allow_nan=False`拒绝 NaN 和无穷值，避免写出不符合标准 JSON 数值约定的报告。

JSON 对象的键最终是字符串；Python 元组保存后通常读回列表；路径对象和自定义模型不能默认直接写入 JSON，应先选取必要的字符串、数值、列表与字典。保存报告与保存可继续训练的模型是不同任务。第 25 章再讨论模型检查点。

3.5 异常处理：失败应当保留原因

异常对象与传播过程

异常(exception)表示执行中断正常流程的情况。第 2 章用`raise ValueError(...)`主动拒绝错误输入。本章进一步区分：函数负责发现什么错误，调用者决定能否恢复。若当前函数不捕获异常，它会向调用者传播；一直没有被处理时，程序终止并显示回溯（traceback）。

常见类型包括：`FileNotFoundError`表示找不到文件；`PermissionError`表示操作权限不足；`ValueError`表示对象类型通常可接受但取值不合适；`TypeError`表示操作

或参数类型不合适；KeyError表示字典缺少指定键；RuntimeError可用于不满足运行状态的情况。异常类型用于表达原因，不能只为了消除红色文字而随机更换。

```
try:
    value = float("missing")
except ValueError as exc:
    print("cannot parse:", exc)
else:
    print("parsed:", value)
finally:
    print("attempt finished")
```

try包含可能失败的操作；匹配的except处理异常；只有try正常完成才执行else；finally在正常或异常离开时执行，常用于清理。以上例子先打印无法转换的说明，再打印结束文字，不执行else。文件资源清理优先使用前面的with。

只捕获能够解释或处理的异常

若捕获转换失败后把值设为 0，损坏文本就变成合法测量，后续平均数看起来正常，却已改变数据含义。若使用except Exception: pass忽略所有普通异常，变量可能没有正确赋值，真正错误也失去位置。

更清楚的设计是把错误补充为可定位的信息，再继续抛出：

```
def parse_weight(text, row_number):
    try:
        value = float(text)
    except ValueError as exc:
        raise ValueError(f"row {row_number}: invalid weight") from exc
    if not isfinite(value) or value < 0:
        raise ValueError(f"row {row_number}: invalid weight range")
    return value
```

这里假定已导入isfinite，输入text为CSV 得到的字符串。raise ... from exc保留**异常链**，同时说明具体记录。float转换后的有限性检查放在try之外，使捕获范围与原因更明确。

入口可捕获预期的数据和文件错误，输出简短失败信息并以非零状态退出；调试时保留异常链有助于追踪。拼写错误产生的NameError等程序缺陷应直接暴露并修正，不应被统一解释为“用户数据有误”。

例题 3.4：异常处理也会制造逻辑错误

某程序对每个费用文本执行float，转换失败后except ValueError: value = 0.0，随后把所有值求平均。输入为["6", "missing", "10"]，它给出什么结果？为什么不应默认为正确均值？

解 程序实际计算 $(6 + 0 + 10)/3 = 16/3$。若任务规则为只统计两条有效记录，均值应为 8，且应报告一条拒绝记录；若规则为必须使用完整三条记录，则应停止并补充数

据。设为 0 相当于增加了一条零费用测量，没有数据依据。异常捕获能恢复控制流程，但不能替代任务中的缺失值规则。

3.6 对象、类与实例：保存状态并提供方法

对象不是面向对象语法才出现的事物

Python 中的整数、列表、函数、模块和路径都是**对象**（object）。每个对象具有类型、身份和相关数据或行为。`type(value)`检查类型，`id(value)`返回在对象存活期间用于标识其身份的整数，不用于保存跨运行的稳定编号。

类（class）描述一类对象的结构和行为；按类创建的具体对象称为**实例**（instance）。`Path("data/a.csv")`创建一个路径实例；`path.open()`调用其**方法**（method）。**属性**（attribute）是用点号访问的名称，既可能表示数据，也可能表示方法。理解这些术语后，后续库中的`model.fit(...)`就可以读成“调用模型实例的拟合方法”。

函数适合把输入映射为输出；类适合把一份需要持续保存的状态与操作它的方法放在一起。不是每个函数都需要改成类。MAE 只依赖给定的真实值和预测值，没有训练状态，作为普通函数很自然；模型则需要在`fit`以后保留从训练数据得到的内容。

一个完整的均值模型类

以下简化类假定`y`为非空有限数值列表，`x`与`y`已对齐；完整实验另加输入检查。

```
class MeanRegressor:
    def __init__(self):
        self.mean_ = None

    def fit(self, x, y):
        if len(x) != len(y) or len(y) == 0:
            raise ValueError("expected equal, nonempty data")
        self.mean_ = sum(y) / len(y)
        return self

    def predict(self, x):
        if self.mean_ is None:
            raise RuntimeError("fit must be called before predict")
        return [self.mean_ for _ in x]
```

`class`建立类；调用`MeanRegressor()`创建实例，随后`__init__`初始化实例属性。它是初始化方法，通常不返回实例，不能写`return self`。`self`是接收当前实例的形参名称，按约定如此命名，不是独立的关键字。

调用`model.fit(x, y)`时，Python 自动将`model`作为第一个参数传入，所以定义需要`self`，调用不再手动填写。`self.mean_`保存在该实例上，方法退出后仍可使用；局部变量`mean`若没有被保存或返回，则不能靠另一个方法直接读取。

下划线后缀`mean_`在这里用于提示“训练后得到的属性”，是命名约定，不是 Python

自动限制读写的机制。初始化为None表示尚未拟合，不能用if not self.mean_判断拟合状态，因为合法训练均值也可能为 0。

```
model = MeanRegressor()
returned = model.fit([1, 2, 3], [2, 2, 8])
print(returned is model)
print(model.mean_)
print(model.predict([0, 100]))
```

输出True、4.0 和[4.0, 4.0]。fit返回自身，支持连续调用：

```
model = MeanRegressor().fit(x, y)
```

这一写法假定已定义合法训练输入x与y。predict返回新列表，不修改输入。均值模型不使用输入数值来改变预测，但仍用x确定输出条数，并保持与其他模型相同的调用方式。

实例状态与共享的类属性

```
a = MeanRegressor().fit([1, 2], [6, 8])
b = MeanRegressor().fit([1, 2], [20, 30])
print(a.mean_, b.mean_)
```

两个实例分别保存 7.0 和 25.0。若把可变训练记录写成类体中的records = []，再通过实例向它追加，多个实例可能共用同一个列表。应该在__init__内部写self.records_ = []，使每个实例获得单独容器。

> **例题 3.5：一个模型对象还是两个模型对象**
>
> 令a = MeanRegressor()、b = a。先用a.fit([1], [6])拟合，再用b.fit([1], [10])拟合。此时a.predict([2])是什么？若需独立比较两套训练结果，应怎样创建对象？

解　b = a只增加一个指向同一实例的名称，因此第二次拟合把同一个mean_改为 10，预测为[10.0]。独立结果需要分别调用两次类，例如a = MeanRegressor()和b = MeanRegressor()。把同一个可变模型放入多个字典键，不会自动保存多个历史版本。

接口、组合与本章的面向对象边界

接口（interface）规定外部如何与对象交互，包括方法名称、参数、返回值、状态和错误。两个模型都提供fit(x, y)与predict(x)，入口就能用相同代码比较它们。接口相同不保证内部算法相同，也不保证训练结果可靠。

组合是让一个对象或流程使用其他对象完成工作，例如入口持有数据、模型和评价函数。继承允许以已有类为基础定义新类，本章无需依赖继承就能组织实验。后续 PyTorch 的nn.Module会使用继承，第 25 章再补充其必要语法。当前需掌握实例状态、方法调用和明确接口，不必先学习所有面向对象机制。

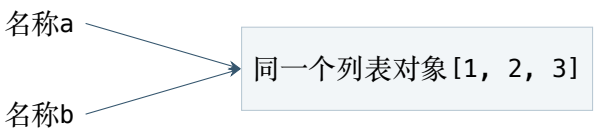
3.7 引用、可变对象与复制

名称绑定与原地修改

第 2 章已说明赋值绑定名称。**引用**（reference）可以理解为程序访问对象的联系；多个名称可以引用同一对象。执行`b = a`不意味着按字节复制`a`中的全部数据。

```
a = [1, 2]
b = a
b.append(3)
print(a, b)
print(a is b)
```

两个列表显示都为`[1, 2, 3]`，身份比较为`True`。`append`修改原对象；若改为`b = [9]`，则只把名称`b`重新绑定到新列表，`a`仍指向原列表。



可变对象（mutable object）允许改变对象内的内容，例如列表、字典和集合；**不可变对象**（immutable object）不能以这种方式改变自身内容，例如整数、字符串和元组。对整数执行`x += 1`会计算新整数并重新绑定名称，不是把所有引用原整数的名称一起改成新值。

`==`比较值是否相等；`is`比较是否为同一个对象。两个独立创建的`[1, 2]`通常值相等但身份不同。小整数或字符串的对象复用属于实现细节，不能用`is`代替数值或文本相等判断。对`None`使用`is None`则表达明确的身份检查。

函数传参为何可能影响调用者

函数形参同样绑定到实参对象。调用`change(values)`不会自动复制列表：

```
def rebind(values):
    values = [99]


def mutate(values):
    values.append(99)

items = [1, 2]
rebind(items)
print(items)
mutate(items)
print(items)
```

第一次输出`[1, 2]`，第二次输出`[1, 2, 99]`。`rebind`只改变局部名称，`mutate`通过引用修改了共享对象。与返回值无关：即使函数没有`return`，原地修改也可能已经

发生。

副作用（side effect）指函数除了返回结果外，对外部可观察状态产生的改变，包括修改传入列表、修改全局字典、写文件和打印。副作用并非都错误，`fit`更新模型状态就是有意设计的副作用；应明确哪些函数修改什么，不能把评价函数也写成会排序或删除输入记录的隐式修改。

浅复制与深复制

浅复制（shallow copy）创建新外层容器，但内部元素仍引用原来的对象。普通列表的`copy()`、`list(a)`和`a[:]`都可以建立浅复制。**深复制**（deep copy）递归复制内部结构，可用标准库`copy.deepcopy`。

```
import copy

rows = [[1, 6], [2, 8]]
shallow = rows.copy()
deep = copy.deepcopy(rows)
shallow[0][1] = 99
shallow.append([3, 10])
print(rows)
print(shallow)
print(deep)
```

输出依次为：

```
[[1, 99], [2, 8]]
[[1, 99], [2, 8], [3, 10]]
[[1, 6], [2, 8]]
```

新外层列表上的追加不影响原外层长度；共享的内层第一行被修改，则两边都能看到。

操作	外层身份	内层可变对象
`b = a`	相同	相同
`b = a.copy()`	不同	仍可能相同
`b = copy.deepcopy(a)`	不同	通常递归得到独立副本

深复制有时间和空间成本，对文件句柄、外部资源及特殊自定义对象也不能理解为“复制整个运行世界”。若本章训练记录是只含数值的元组，保存`tuple(zip(x, y))`已经可以隔离原输入列表的后续增删或数值替换，不需要随意深复制。

例题 3.6：二维列表的重复引用

预测下面代码的结果，并构造两个真正独立的内层列表。

```
rows = [[0, 0]] * 2
rows[0][0] = 7
print(rows)
```

解 列表重复运算复制的是其中的引用，两个位置都指向同一内层列表，输出`[[7, 0], [7, 0]]`。应写`rows = [[0, 0] for _ in range(2)]`，每次循环创建新内层列表。若每一行都用同一个外部列表`row`追加，即使使用循环也仍会共享；关键是何时创建对象。

默认参数与延迟读取的循环变量

第 2 章的可变默认参数问题可以用引用解释。`def collect(value, bucket=[])`在定义时只创建一次默认列表，后续未提供`bucket`的调用会引用它。用`None`作默认值，再在函数体中新建列表，才能实现“每次调用独立容器”。若调用者显式传入一个列表，函数仍会按接口修改它。

保存多个预测函数还可能遇到另一种问题：函数执行时才读取外层名称。

```
functions = []
for k in range(3):
    functions.append(lambda x: x + k)
print([f(10) for f in functions])
```

构建结束后`k`为 2，三个函数随后调用都读取此时的`k`，结果为`[12, 12, 12]`。要在每次创建函数时保存当前数值，可以使用默认参数：

```
functions = []
for k in range(3):
    functions.append(lambda x, k=k: x + k)
print([f(10) for f in functions])
```

结果为`[10, 11, 12]`。左侧`k`是函数形参，右侧`k`在创建时求值。本例保存不可变整数，含义明确；若保存的是可变列表默认值，还需重新检查共享关系。项目比较模型时直接创建不同实例，可以减少保存匿名函数时的这种绑定问题。

3.8 阅读回溯与检查中间状态

从最后一行判断失败类型

假设`run.py`调用`metrics.mae`，而函数内部遗漏了非空检查，可能看到如下回溯：

```
Traceback (most recent call last):
  File "run.py", line 4, in <module>
    print(mae([], []))
  File "metrics.py", line 3, in mae
    return total / len(actual)
ZeroDivisionError: division by zero
```

先读最后一行，确定是除以零；再读最近的调用位置，确认分母`len(actual)`；随后沿调用链向上追踪为什么传入空列表。修正可能位于指标函数的输入检查，也可能

位于前面把数据筛空的代码，应根据接口决定。直接把分母改成`len(actual) + 1`会改变指标定义，不能算修复。

回溯所标行号是实际保存文件中的位置。编辑器里未保存的修改尚未执行；运行了同名旧副本时，也可能出现“修改过却没有变化”。应结合`__file__`、解释器位置和实际命令确认正在运行哪份程序。

有问题意识地打印

调试输出应针对一个假设。若怀疑 CSV 数值没有转换，应检查字段值的类型；若怀疑划分为空，打印每组长度；若怀疑特征与标签错位，打印带标识的前几条对应记录；若怀疑共享列表，打印`a is b`和内部对象的身份关系。

```
print("source:", data_path)
print("counts:", len(train), len(validation), len(test))
print("first train:", train[:2])
print("label type:", type(train[0]["cost"]).__name__)
```

这一片段需在相应变量建立后执行。先检查`train`非空，再访问`train[0]`。只打印最终 MAE 无法分辨数据读取、模型拟合和指标计算分别是否正确。

断点（breakpoint）让程序暂停在指定位置，以便观察变量并逐步执行。Python 内置`breakpoint()`通常进入交互调试器；常见操作包括查看变量、单步执行和继续运行。编辑器也可提供图形断点。对本书小程序，先用小输入、状态表和少量打印足以定位大多数问题；避免一次打印整份大数据掩盖关键记录。

最小可复现示例

最小可复现示例（minimal reproducible example）只保留复现同一个错误所需的输入和步骤。若完整项目报告错误，不必一开始就重读全部数据。可以先把触发记录保存成三行 CSV，直接调用读取器；若读取正常，再用两条训练记录和一个查询调用模型；最后单独计算一个手算可核对的指标。

缩小问题时必须保留触发条件。广播错误需要保持相关形状，引用错误需要保留共享对象，编码问题需要保留原字符。把所有数据换成简单数字后不再出错，并不证明原程序正确，只说明已删掉某个触发因素。

例题 3.7：定位“模型总预测同一个数”

一个程序的所有预测都是 9。给出至少三个可能原因，并分别设计能区分原因的检查。

解 第一种可能是程序确实选中了训练均值模型，检查对象类型、选择记录和训练均值即可。第二种可能是近邻函数在循环外固定使用第一个查询，应输入相距很远的两个查询，并打印每次距离。第三种可能是保存多个候选时反复引用同一个实例，检查候选对象身份及每次拟合后的状态。还可能是输入列被错误读成常数，应按标识打印原字段和转换后的值。常数输出本身不能唯一证明模型没训练。

3.9 断言、最小测试与可靠核对

assert 的用途与限制

断言（assertion）写出开发者预期始终成立的内部条件。例如检查输出条数：

```
assert len(predicted) == len(test)
```

条件为假时会抛出AssertionError。断言适合教学核对与测试，但在启用优化模式的 Python 运行中可能被移除。对外部输入的必要检查仍应使用显式条件判断与raise，不能只依赖断言。

```
from math import isclose
from metrics import mae

assert isclose(mae([5, 5], [7, 3]), 2.0, rel_tol=0, abs_tol=1e-12)
assert mae([0], [0]) == 0
```

isclose使用浮点容差。第一个测试包含一正一负有符号误差，能揭示遗漏绝对值的错误；只用[1, 2]与[2, 3]，错误的有符号平均也会偶然得到 1。核对输入应针对要识别的错误，而非只追求测试数量。

正常、边界和失败路径

对 MAE，应包含等长正常输入、全零误差、空输入、长度不等和非有限数。对模型，应包含预测前未拟合、两次独立实例、距离同分和输入后来被修改。对文件，应包含错误表头、缺字段、多字段、重复标识和未知集合。每项检查都对应一个明确承诺。

捕获预期异常的最小写法为：

```
try:
    mae([], [])
except ValueError:
    pass
else:
    raise AssertionError("empty inputs were not rejected")
```

这里的pass表示测试已经收到所要求的异常，不是项目运行时忽略失败。若没有报错，else让测试失败；若报了其他类型的异常，也应暴露出来，因为那并不符合约定。

单元测试（unit test）核对一个较小组件；**集成测试**（integration test）核对多个组件连接后的行为。分别检查读取器和模型，不能替代检查整条流程是否仍将正确标签交给指标。反过来，只核对最终 MAE 也可能让两个错误彼此抵消。完整项目保留少量两类检查。

使用标准库 unittest

unittest是标准库测试工具，以方法组织测试，并报告哪些检查失败。本章test_project.py包含以下形式：

```
import unittest
from metrics import mae

class MetricTests(unittest.TestCase):
    def test_hand_value(self):
        self.assertAlmostEqual(mae([5, 5], [7, 3]), 2.0)

    def test_empty(self):
        with self.assertRaises(ValueError):
            mae([], [])

if __name__ == "__main__":
    unittest.main()
```

`TestCase`提供核对方法，`test_`开头的方法被识别为测试。`assertAlmostEqual`检查浮点结果，`assertRaises`检查是否抛出指定异常。此处继承仅用于沿用测试工具已定义的功能，无需先设计复杂类层次。命令`python3 test_project.py`会运行测试并打印通过数量。

测试不能证明所有可能输入都正确，但可以固定已经明确的规则，帮助修改代码后发现退步。若把预期值写成与被测函数完全相同的计算，就容易共同犯错；本章优先采用手算小例、独立实现对照和特定失败输入。

3.10 实验 3.1：对象变化与文件往返

入口为配套实验`/ch03_objects.py`，仅依赖 Python 标准库。程序不要求输入个人文件，文件实验在自动建立的临时目录中进行，结束后清理。实验同时核对三个问题：哪些修改会穿过共享引用；怎样在每次调用时创建独立列表；JSON 写入再读回是否保持预期数据。

开始前手工画出`rows`、`alias`、`shallow`、`deep`指向的外层和内层对象。核心操作为：

```
rows = [[1, 6], [2, 8]]
alias = rows
shallow = rows.copy()
deep = copy.deepcopy(rows)
alias.append([3, 10])
shallow[0][1] = 99
```

`alias.append`修改原外层列表，所以`rows`增加一行，`shallow`的长度仍为 2。第二次操作修改共享内层第一行，所以`rows`和`shallow`的费用都变为 99。`deep`保存的原始两行不变。

在项目的配套实验目录运行：

```
python3 ch03_objects.py
```

主要预期输出为：

```
rows: [[1, 99], [2, 8], [3, 10]]
shallow: [[1, 99], [2, 8]]
deep: [[1, 6], [2, 8]]
identity: True False True False
fresh: [1] [2]
captured: [10, 11, 12]
round trip: 3 教学数据
caught: ValueError
all checks passed
```

身份比较依次核对别名与原列表、浅复制与原列表、浅复制第一行与原第一行、深复制第一行与原第一行。结果应精确一致；本实验没有近似数值计算。

文件往返部分使用`TemporaryDirectory`作为上下文管理器。进入时创建临时目录，将字典写入`report.json`再读回；退出时删除本实验临时目录。这是标准库管理临时资源的实例，不能将它替换为随意删除原始数据目录。

修改任务依次进行，每次从原版恢复：把`shallow`改为深复制，预测第一行还会不会互相影响；把默认参数改成`bucket=[]`，连续调用观察累积；去掉`lambda`中的`k=k`，预测三个输出；把 JSON 中的整数改成元组`(1, 2)`，读回后核对类型。最后一个任务应得到列表，说明数据往返不保证保留所有 Python 类型。

3.11 实验 3.2：组织可复现的预测小程序

目录、输入与流程约定

入口为配套实验`/ch03_project/main.py`。**可复现**（reproducible）在本章指：给定同一代码、数据、环境和流程规则，可以从头得到一致的预测和评价结果。它不等同于模型对未来数据一定可靠，也不要求不同机器生成完全相同的耗时或临时路径。

```
ch03_project/
    main.py
    io_utils.py
    models.py
    metrics.py
    test_project.py
    data/parcels.csv
    results/summary.json
    results/predictions.csv
```

`main.py`协调流程；`io_utils.py`负责严格读取与结果写出；`models.py`保存模型类；`metrics.py`提供 MAE；`test_project.py`核对规则。结果文件由运行生成。依赖关系由入口指向各组件，组件不导入入口，导入模块不启动实验。

数据有 12 条，训练、验证、测试各 4 条，划分直接由 CSV 的`split`列指定。本章不重新随机划分，因此没有用一个随机种子代替数据记录的必要。数据如下，重量和

费用均为教学构造值。

集合	重量（千克）	费用（元）
训练	1，2，3，4	6，8，10，12
验证	1.2，2.2，3.2，3.8	6.4，8.4，10.4，11.6
测试	1.4，2.6，3.4，4.5	6.8，9.2，10.8，13.0

候选为训练均值模型和单近邻模型；按最小验证 MAE 选择，同分时取声明顺序靠前者。单近邻距离同分时取训练记录中靠前者。选定后不把验证集合并重训，直接评价已选模型；这样能明确追踪每次拟合到底使用哪些标签。其他项目可以另定重训协议，但必须记录并保持一致。

严格读取与保持记录对齐

读取器先验证`reader.fieldnames`等于预期表头。每条记录转换前检查字段数、非空唯一标识和合法集合名。完整读取器用`reader.line_num`记录当前记录结束的物理行号。CSV 字段可以含引用的换行，因此记录序号不总等于文件行号。对`DictReader`，多出的字段可能对应键`None`，缺少的字段可能对应值`None`；因此完整读取器有如下检查：

```
if None in row or any(v is None for v in row.values()):
    raise ValueError(f"row {line}: wrong field count")
```

通过检查后再解析`float`，验证有限、非负，最后建立规范字典。这一过程先保证数据结构，再保证字段意义，不能只看能否运行除法。完整实现保存在`io_utils.py`；讲义中的短片段用于说明该检查的原因，运行时使用完整入口。

训练输入和标签必须从同一记录顺序提取：

```
def columns(records):
    return ([row["weight"] for row in records],
            [row["cost"] for row in records])
```

这里两次遍历同一个未被修改的记录列表，因此位置对应。更换顺序或筛选时，应对整条记录操作，而非分别排序重量和费用。唯一标识只能揭示某些重叠，不能证明不同标识没有复制内容；真实数据仍需按第 1 章协议检查来源与重复样本。

用相同接口比较两个模型

均值模型的`fit`保存`sum(y) / len(y)`。单近邻模型的`fit`保存`tuple(zip(x, y))`，`predict`逐个查询最近训练记录：

```
return [min(self.records_, key=lambda row: abs(row[0] - q))[1]
        for q in x]
```

这一片段展示方法内部的核心选择规则，假定已完成拟合与输入检查，且差值处

于可表示范围。完整模块还会检查均值、距离和指标结果是否有限；输入各项有限，并不保证求和或相减后仍有限。若范围检查失败，程序停止并要求检查数据尺度，第 4 章将进一步讨论数值异常。`min`在同分时选第一次遇到的记录；`[1]`取得费用。与第 1 章完整排序相比，单个近邻只需扫描找到最小值，无需把所有距离排序。对 n 条训练记录、m 个查询，时间为 $O(mn)$；保存训练记录需要 $O(n)$ 空间，保存输出需要 $O(m)$。这里 O 表示随输入规模增长的量级，不表示实际秒数。

入口中每个候选都创建新实例：

```
factories = {"mean": MeanRegressor, "nearest": NearestRegressor}
models = {}
scores = {}
for name, factory in factories.items():
    model = factory().fit(x_train, y_train)
    models[name] = model
    scores[name] = mae(y_val, model.predict(x_val))
selected = min(scores, key=scores.get)
```

`factory`在这里是类对象，`factory()`创建对应实例。两个字典分别保存模型和验证分数，不把模型对象与浮点分数混在一起。`scores.get`作为比较函数，让`min`按键对应的分数选择；字典的插入顺序对应已声明的候选顺序。

读取器会一次读入文件全部字段，但候选拟合只接收训练标签，候选选择只接收验证指标。文件中存有测试标签不自动构成泄漏，关键是这些标签是否影响训练或选择；本章按代码依赖限制其用途。真实项目也可把最终测试标签交给独立评价程序管理。

手算验证选择与最终评价

训练均值为 $(6+8+10+12)/4 = 9$。在验证集，四个绝对误差为 2.6、0.6、1.4、2.6，因此 MAE 为 1.8。单近邻分别找到重量 1、2、3、4，预测 6、8、10、12，绝对误差均为 0.4，MAE 为 0.4，应选择单近邻。

测试只对选定模型作最终报告，同时保留预先声明的均值基线作比较：

重量	真实费用	单近邻预测	单近邻绝对误差	基线绝对误差
1.4	6.8	6	0.8	2.2
2.6	9.2	10	0.8	0.2
3.4	10.8	10	0.8	1.8
4.5	13.0	12	1.0	4.0

单近邻测试 MAE 为 $(0.8 + 0.8 + 0.8 + 1.0)/4 = 0.85$；均值基线为 $8.2/4 = 2.05$。4.5 超过训练重量范围，单近邻仍返回边界费用 12；这体现近邻方法的外推限制。该数据构造得十分规则，结果不能用于推断真实物流业务表现。

运行、保存与再次运行

在配套实验目录执行：

```
python3 ch03_project/main.py
python3 ch03_project/test_project.py
```

也可从其他目录用入口的绝对路径运行。程序应输出：

```
counts: {'train': 4, 'validation': 4, 'test': 4}
validation mean: 1.800000
validation nearest: 0.400000
selected: nearest
test predictions: [6.0, 10.0, 10.0, 12.0]
test MAE: 0.850000
test baseline MAE: 2.050000
saved: results/summary.json, results/predictions.csv
```

第二个命令应报告 5 项测试通过。报告数字按六位小数显示，内部计算允许约 10^{-12} 的绝对误差。每条预测、集合计数和被选名称应一致，不把打印舍入差异解释为模型改变。

JSON 记录 Python 版本、数据文件的 SHA-256 摘要、集合数量、候选顺序、选择规则、是否重训、验证分数和最终指标。**摘要**（hash digest）把文件内容映射为定长标识，有助于判断两次运行是否使用相同字节；它不证明数据真实、标签正确或划分合理。仅修改 CSV 换行也会改变摘要，但模型数值可能仍相同。

预测 CSV 保存标识、真实值、预测值与逐条绝对误差。再次运行会替换同名结果；代码与数据不变时，关键数值应相同。跨 Python 版本的版本字段自然不同，JSON 字节完全相同不是本章复现要求。需要保留多次结果时，应复制到明确编号目录，并同时保留对应代码与输入。

有控制地修改与故障注入

从原始数据副本开始，每次只改一个因素，运行后恢复原版。

1. 把某重量改为`nan`，应在读取阶段失败并提示记录位置，不生成新的完整成功结果；旧输出若仍存在，也不能当作本次成功证据。
2. 把一条验证记录的标识改成训练标识，应拒绝重复。将集合名称拼成`validaton`，应拒绝未知集合。
3. 从其他工作目录启动入口，数值应与原版一致；若改成裸相对数据路径，应能复现路径错误，再恢复正确定位。
4. 将第一条训练费用 6 改为 10，先手算两候选验证分数，再运行。不能根据测试分数把该修改说成真实改进。
5. 单独对重量 100 作预测，应得到 12；该输入没有真实标签时，不并入 MAE。解释为什么程序能给出数值并不保证外推可靠。

第 4 项的均值变为 10，验证 MAE 为 1.8；单近邻验证误差为 3.6、0.4、0.4、0.4，MAE 为 1.2，仍选单近邻。修改后最终测试误差变成 3.2、0.8、0.8、1.0，MAE 为 1.45。该手算用于核对流程传播，不能用最终测试数值倒推应怎样修改训练数据。

3.12 知识联系与自检

程序中的三类联系需要分别理解。模块依赖说明代码从哪里取得；对象引用说明哪些名称共享状态；数据依赖说明哪些记录影响模型和选择。正确的导入结构不能自动防止标签泄漏，独立的对象也不能自动防止读错文件。

第 2 章函数的输入输出约定在本章扩展为模块接口和模型方法；列表切片的浅复制与本章的内部共享相连；第 1 章的训练、验证、测试在入口中表现为不同的数据传递。第 4 章增加数组以后，除这些要求外，还必须核对轴、形状和共享内存。

自检时应能回答：为什么脚本可导入却读不到数据；为什么`b = a`不是模型快照；为什么捕获错误后填 0 会改变任务；为什么空输入应由显式检查拒绝；为什么一个测试应包含正负有符号误差；为什么相同种子不能代替保存数据版本。回答应举出具体语句或输入，而非只复述术语。

3.13 分层习题

以下编号为 3.1–3.14。代码题先写预期状态，再运行核对；文件题使用自己建立的临时数据，不修改原始教学数据。

3.1 **概念**。区分脚本、模块、包和第三方依赖；说明`from metrics import mae`后为什么不能直接推断名称`metrics`可用。

3.2 **执行分析**。在例题 3.1 中将第一行打印移入`main`，直接运行和首次导入各输出什么？重复导入为什么通常不再次执行顶层语句？

3.3 **路径**。工作目录为`/tmp`，源文件在项目目录中。对以下两种定位写出实际指向的位置。

```
源文件：/tmp/project/main.py
路径甲：Path("data/a.csv")
路径乙：Path(__file__).resolve().parent / "data" / "a.csv"
```

3.4 **文件**。解释`"r"`、`"w"`、`"a"`的区别，为什么打开原始数据时不能误用`"w"`？`with`解决什么问题，不能保证什么？

3.5 **输入验证**。实现`parse_nonnegative(text)`，接受可转为有限非负浮点数的文本，其他情况抛`ValueError`。核对`"0"`、`"nan"`、`"inf"`、`"-1"`与`"missing"`。

3.6 **引用**。假定已导入`copy`，执行以下语句后写出四个变量的值。

```
a = [[1, 2], [3, 4]]
b = a
c = a.copy()
d = copy.deepcopy(a)
b.append([5, 6])
c[0][0] = 9
```

3.7 **副作用**。比较函数内部`values = sorted(values)`与`values.sort()`对调用者列表的影响。设计一个输入和检查区分两者。

3.8 **类与状态**。为什么均值模型未拟合标记用`None`，而不使用 0？若`a`和`b`引用同一模型，连续拟合会有什么后果？给出独立创建的写法。

3.9 **绑定**。三个函数在循环中通过`lambda x: x + k`保存，循环结束再调用得到什么？改为保存当前整数`k`，并解释默认参数求值时机。

3.10 **异常与测试**。有人用`except Exception: return 0`包装整个 MAE 函数，称“任何输入都不报错”。指出这怎样破坏接口，并写一个检查空输入必须报错的最小测试。

3.11 **项目手算**。实验 3.2 将第一条训练费用改为 10，其他不变。计算两候选验证 MAE、选定模型和测试 MAE，说明哪些结果必须重新生成。

3.12 **JSON**。字典`{"pair": (1, 2), "missing": None}`经过 JSON 写入与读回后，各值是什么类型？为何不应直接保存模型实例？

3.13 **故障定位**。项目报`KeyError: 'cost'`。给出应依次检查的三个具体位置；为什么不应直接改成`row.get("cost", 0)`？

3.14 **综合实验**。从项目目录与另一目录分别运行实验 3.2，再运行其测试。核对预测文件、数据摘要和指标；列出报告中哪些字段用于复现、哪些条件仍不足以证明真实泛化。

3.14 习题参考答案

3.1–3.4：组织、路径与文件

3.1 脚本强调作为入口执行的文件；模块强调可导入的代码单元，同一文件可以兼具两种用途；包组织多个模块；第三方依赖是需要安装到解释器环境中的外部代码。`from metrics import mae`只把指定名称绑定到当前名称空间，不同时绑定`metrics`；需用`import metrics`才能按模块前缀访问。

3.2 移入后直接运行输出`loading`与`running`，首次导入不输出文字。普通重复导入通常复用进程中已缓存模块；重启解释器后会重新建立模块。入口保护控制是否调用`main`，缓存则控制普通重复导入是否重新初始化，两个机制不同。

3.3 前者按当前工作目录解释为`/tmp/data/a.csv`；后者以源文件父目录为起点，指向`/tmp/project/data/a.csv`。`resolve`解析符号链接，本题假定没有符号链接影响。两者不能仅从路径字符串都含`data`就视为相同。

3.4 `"r"`读取且文件应存在；`"w"`写入并在打开时截断原内容；`"a"`在末尾追加，重

复运行可能重复记录。with保证按上下文管理规则退出，普通文件会被关闭；不能保证写入内容符合任务，也不能保证多个文件在故障时全部完整写入。

3.5–3.9：验证、引用与对象

3.5 先转换，再检查有限与非负；不能以真假判断代替非负检查。

```
import math

def parse_nonnegative(text):
    try:
        value = float(text)
    except ValueError as exc:
        raise ValueError("not a number") from exc
    if not math.isfinite(value) or value < 0:
        raise ValueError("expected finite nonnegative number")
    return value
```

"0"返回 0.0，其余四项抛ValueError。本题接口接受文本，不承诺任意 Python 对象都被统一转为同一错误。配套ch03_solutions.py核对这些输入。

3.6 四个名称的值为：

```
a, b: [[9, 2], [3, 4], [5, 6]]
c:    [[9, 2], [3, 4]]
d:    [[1, 2], [3, 4]]
```

追加通过共享外层作用于a和b；第一项修改通过共享内层作用于a、b、c；d在两次操作前已递归复制。

3.7 函数内values = sorted(values)建立新排序列表并重绑局部名称，调用者列表不变；values.sort()原地排序，调用者看到变化。以items = [3, 1, 2]分别传入，调用后核对原列表与[3, 1, 2]是否相等即可。若原列表本来已排序，这个输入就无法揭示两者行为差异。

3.8 均值 0 是合法训练结果，真假判断会把它误当成未拟合。None明确标记缺少训练状态。同一模型被两个名称引用时，后一次拟合覆盖前次状态；独立写a = MeanRegressor()与b = MeanRegressor()，再分别拟合。若用字典保存比较结果，也应每次创建新实例。

3.9 循环结束k=2，原版三个函数对 10 都返回 12。改成lambda x, k=k: x + k后，创建时分别保存 0、1、2，对 10 返回 10、11、12。这里默认值在函数对象创建时求值，不是在后续每次调用时重新读取循环变量。

3.10–3.14：失败路径与复现

3.10 返回 0 把输入错误、程序缺陷和真正零误差合并为一个无法区分的结果，可能让错误方案看起来最好。可使用本章的try/except/else最小测试，或在unittest.TestCase方法内写：

```
with self.assertRaises(ValueError):
    mae([], [])
```

测试中捕获预期异常是核对失败路径；实际评价中忽略异常并给出零分则改变接口含义。

3.11 训练费用变为 10、8、10、12，均值 10。验证均值误差 3.6、1.6、0.4、1.6，总和 7.2，MAE 为 1.8。单近邻验证误差 3.6、0.4、0.4、0.4，总和 4.8，MAE 为 1.2，选单近邻。测试预测为 10、10、10、12，误差 3.2、0.8、0.8、1.0，MAE 为 1.45。数据摘要、验证分数、选择报告和逐条预测应一起更新。用测试分数选择是否保留该修改会违反原协议。

3.12 `pair`读回为列表，`missing`读回为`None`。JSON 保留常用数据结构，不保存任意类的方法、运行资源或全部类型身份。模型实例应选取必要配置与训练状态按明确格式保存；本章只保存结果报告，不承诺可从报告恢复模型。

3.13 先检查原 CSV 表头是否真有`cost`，包括两端空格与大小写；再检查读取器构造字典时是否换名或漏字段；最后检查报错行变量是否仍是预期记录，而不是另一种字典。默认 0 会把缺字段变成测量值，掩盖格式错误，并可能改变训练目标或评价标签。应在读取边界拒绝不合约定的数据。

3.14 两种启动位置应给出相同集合数量、模型选择、预测与关键指标；数据未变时摘要一致。记录 Python 版本、代码版本或副本、数据内容、划分、候选顺序和同分规则，才能解释复现条件。本项目还需保留主源及配套代码的对应版本。即使这些条件满足，也只能复现教学实验，不能保证未来分布、标签质量和真实业务代价与该数据相同。

3.15 本章小结

模块给代码划分名称和依赖边界，路径给文件确定位置，数据格式与验证确定输入含义。类把训练状态和预测方法连接起来；引用决定状态是否共享，复制决定哪些层次独立。调试通过回溯、中间状态和最小输入寻找原因，测试把已明确的规则固定为可重复检查。

实验 3.2 将这些机制落实为可从头运行的预测流程。结果是否可复现，要检查代码、数据、环境和选择协议；评价是否可信，还要回到第 1 章对信息可用性、划分和泛化条件的要求。

4 NumPy 与向量化计算

先修知识、学习目标与本章位置

本章需要第 2 章的循环、列表和函数，以及第 3 章的导入、对象、引用与调试。矩阵运算只用到按行列取值、乘积求和和平均数，所需含义在本章补充，第 6 章再系统复习线性代数。

学习后应能解释数组的类型、轴和形状；区分索引、切片、视图与复制；逐轴核对广播和矩阵乘法；用归约与向量化替代逐样本循环；控制随机数并检查浮点结果；从头完成一个无预处理泄漏的数值实验。

第 2 章用循环逐条计算预测、有符号误差与平均误差，这种写法直接反映计算过程。数据量增加以后，可以把规则相同的操作交给专门的数值库一次完成。**NumPy**（Numerical Python）是 Python 科学计算中的基础数组库，提供多维数组及其运算。NumPy 数组也会遇到逻辑错误：程序能运算，只说明类型与形状在规则上兼容，不说明每一行仍对应正确样本。

本章把数组理解为带有形状与数据类型的数值容器。先逐项说明它与列表的不同，再建立逐元素运算、归约、广播和矩阵乘法的联系。所有维度推导都回到元素下标，避免只凭代码短或输出形状合理就判断正确。

4.1 安装、导入与数组的用途

让解释器与依赖处于同一环境

NumPy 是第三方依赖，需安装到实际运行脚本的 Python 环境中。按第 2 章建立并激活虚拟环境后，可在普通终端执行：

```
python3 -m pip install numpy
python3 -c "import numpy as np; print(np.__version__)"
```

Windows 按第 2 章将`python3`换为相应启动命令。`-m pip`让所选解释器运行包安装工具，减少把包装到另一个解释器的情况。若已安装且能导入，无需重复安装。网络不可用时可使用已经提供 NumPy 的环境，本书实验不依赖下载数据。

代码中采用惯用别名：

```
import numpy as np

values = np.array([1.0, 2.0, 3.0])
print(values)
print(type(values))
```

输出数值数组和其类型`numpy.ndarray`。`np`只是`numpy`的当前名称，不是 Python 内置关键字；`ndarray`表示多维数组类型。后文代码框未重复导入时，均以此前已执行`import numpy as np`为前提。

配套实验在 Python 3.12 与 NumPy 2.3.5 环境实际核对。基础接口不依赖特定硬件；

复现时应记录自己的版本，而不是只写“已安装 Python”。安装命令不固定后续所有版本行为，项目要长期复现时应同时保留已核对的依赖版本。

列表运算与数组运算不同

```
items = [1, 2, 3]
array = np.array(items)
print(items * 2)
print(array * 2)
print(items + items)
print(array + array)
```

列表乘 2 得到`[1, 2, 3, 1, 2, 3]`，表示序列重复；数组乘 2 得到`[2 4 6]`，表示每个元素乘 2。列表相加连接序列，数组相加执行逐位置加法。NumPy 显示数组时通常不在数字间添加逗号，显示格式不意味着它变成另一种数学对象。

向量化（vectorization）在本章指把逐元素或逐样本的规则表达为数组级操作，使主要循环由数值库执行。它没有消除底层循环，而是减少 Python 层逐项调度，并利用面向数组的实现。列表推导式虽然简短，仍然主要是 Python 逐项执行，不等于 NumPy 向量化。

NumPy 不自动把所有工作放到 **GPU**（graphics processing unit，图形处理器）上，本章标准数组在 **CPU**（central processing unit，中央处理器）上计算。深度学习中的设备与张量将在第 25 章讲解；数组代码正确与使用哪种硬件是两个问题。

4.2 数组形状、轴与统一的数据约定

shape、ndim、size 和 dtype

形状（shape）给出每个**轴**（axis）的长度；**维数**在数组语境中常指轴的个数，用`ndim`表示；`size`表示总元素数量；`dtype`表示元素数据类型。四者承担不同工作。

```
X = np.array([[1.0, 2.0, 3.0], [4.0, 5.0, 6.0]])
print(X.shape)
print(X.ndim)
print(X.size)
print(X.dtype)
```

分别得到`(2, 3)`、2、6 与`float64`。`shape`是元组；`(3,)`表示单轴长度 3，逗号不可省略；零维数组形状为`()`。若形状为 $(s_0, \dots, s_{r-1})$，总元素数为各长度的乘积，轴数为 r。对矩阵形状 (n, d)，`len(X)`只给出第一轴长度 n，不等于总元素数 nd。

创建方式	形状	结构含义
`np.array(3.0)`	`()`	零维数组，含一个数
`np.array([1, 2, 3])`	`(3,)`	一维数组，没有独立行轴与列轴
`np.array([[1, 2, 3]])`	`(1, 3)`	二维数组，一行三列
`np.array([[1], [2], [3]])`	`(3, 1)`	二维数组，三行一列

同样三个数，可以有不同形状。数学中的列向量常写成 3×1，NumPy 中的长度 3 一维数组却不自动带有“列”方向。`w.T`对一维数组仍为一维，必须明确增加轴才能得到二维列数组。

样本轴与特征轴

本书对普通表格数值数据采用

$$X \in \mathbb{R}^{n \times d}, \qquad X_{ij} = \text{第 } i \text{ 个样本的第 } j \text{ 个特征}.$$

数学下标在推导中通常从 1 开始，Python 索引从 0 开始，程序位置`X[0, 0]`对应数学 X_{11}。n 为样本数，d 为每个样本的特征数，`X.shape == (n, d)`。轴 0 沿样本变化，轴 1 沿特征变化。

例如两件包裹分别有三个已数值化特征，数组可以为

$$X = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}.$$

第一行是一个样本，不是一个特征的全部测量。若把矩阵转置为 3×2，数值总数不变，但样本与特征的解释互换。形状必须与语义一起写，不能只写“这是一个二维数组”。

> **注意：数组维数与特征维数**
>
> 100 个样本、每个样本有 20 个特征，可表示为形状`(100, 20)`的二维数组；特征空间则是 20 维。说“数据是 20 维”与说“数组有两个轴”并不矛盾，它们描述不同事物。后续一批图像可能有样本、高度、宽度、通道等轴，仍需逐轴解释。

reshape、增加轴与转置

`reshape`按指定顺序重新组织元素，要求总元素数量不变：

```
a = np.arange(6)
print(a.reshape(2, 3))
print(a.reshape(3, 2))
print(a.reshape(-1, 1).shape)
```

`arange(6)`给出 0 至 5；默认按行顺序填入。前两次分别得到两行三列和三行两列；`-1`表示由其他轴长度推算该轴，最后形状为`(6, 1)`，最多只能有一个待推算轴。没有依据地`reshape`可能让一个样本的特征散落到多行，即使元素数正确仍破坏数据含义。

增加长度为 1 的轴可用None或np.newaxis：

```
w = np.array([1.0, 2.0, 3.0])
print(w[:, None].shape)
print(w[None, :].shape)
print(w.T.shape)
```

依次为(3, 1)、(1, 3)和(3,)。二维数组X.T交换两个轴；reshape(3, 2)与X.T一般不是同一运算。对前面的 X，转置为 $\left(\begin{smallmatrix}1&4\\2&5\\3&6\end{smallmatrix}\right)$，而直接重排为 $\left(\begin{smallmatrix}1&2\\3&4\\5&6\end{smallmatrix}\right)$。

reshape可能返回共享数据的视图，也可能因为内存布局而复制，不能一律当成独立副本。需要独立数据时显式copy()。squeeze可去除长度为 1 的轴，但在单样本批次中不加选择地挤压，可能把样本轴也去掉；优先明确指定要处理的轴。

4.3 数据类型、数组创建与数值范围

同一 dtype 与固定宽度

普通数值数组的元素共用一个dtype。float64通常以 64 位表示浮点数，float32通常以 32 位表示；int64表示固定宽度整数；bool表示布尔值。位数影响可表示范围、精度和内存，不表示小数点后固定保存 64 或 32 位。

```
counts = np.array([1, 2, 3], dtype=np.int64)
weights = counts.astype(np.float64)
print(weights.dtype)
print(weights.itemsize, weights.nbytes)
```

astype在这里转换并建立新数组；itemsize为每元素字节数，nbytes为元素数据所占字节数。三个float64元素共 24 字节，不含 Python 对象元数据等额外成本。约百万个float64元素需要约 800 万字节，仅是数组数据本身的量级估算。

整数数组里写小数不会自动把整个数组改为浮点类型：

```
a = np.array([1, 2], dtype=np.int64)
a[0] = 1.9
print(a)
```

结果为[1 2]，小数部分被截去。用于平均、标准化与模型系数的数组应从开始就明确采用合适浮点类型，而不是等发现小数丢失后再转换。

Python 整数可按需要扩大表示，但 NumPy 固定宽度整数可能溢出。例如两个uint8数组[250]和[10]相加，结果按其范围回绕为[4]。图像像素常见 0–255 整数，直接做差或平方可能在计算前就出错。应先把输入转换成浮点，再相减或平方；在错误计算以后才转浮点，不能恢复已经丢失的信息。

配套数值接口先把实数输入转换为独立的float64数组，再执行差值与平方。以下辅助函数也会拒绝布尔、复数、字符串和一般对象数组；不能无说明地丢弃复数虚部或把文本解释为特征。

```
def as_float_array(values, name):
    array = np.asarray(values)
    if array.dtype.kind not in "iuf":
        raise ValueError(f"{name}: expected real numeric values")
    with np.errstate(over="raise", invalid="raise"):
        array = array.astype(np.float64)
    if not np.isfinite(array).all():
        raise ValueError(f"{name}: expected finite values")
    return array
```

`dtype.kind`中的`i`、`u`、`f`分别表示有符号整数、无符号整数和浮点类型。`astype`在这里生成副本，转换不修改调用者数组。`np.errstate`的局部异常检查将在数值稳定性一节说明。`float64`也不能精确保存任意大的整数；本实验输入是小尺度实数，不用于需要精确保留超大整数标识的任务。

规则数组与对象数组

`np.array([[1, 2], [3, 4]])`有规则矩形结构；`[[1, 2], [3]]`各行长度不等，不能直接作为常规二维数值数组。混入字符串可能使数组转为字符串类型，后续加减不再具有原数值意义。

`dtype=object`可以装入一般 Python 对象，但会回到逐对象操作，也带来第 3 章的内部可变对象共享问题。本章聚焦规则数值数组，不用对象数组绕开结构错误。数组的`copy()`复制数值缓冲区；若数组元素本身是对象引用，它不自动深复制这些对象。

zeros、ones、arange 与 linspace

```
print(np.zeros((2, 3)))
print(np.ones((1, 3)))
print(np.arange(1, 7, 2))
print(np.linspace(0.0, 1.0, 5))
```

前两者创建指定形状的全 0、全 1 数组；`arange`按步长生成并通常不含终点，得到 1、3、5；`linspace`指定点数，默认包含两端，得到 0、0.25、0.5、0.75、1。需要固定数量的等距浮点点时，`linspace`比依赖浮点步长恰好落在终点更清楚。

`np.full((n,), value)`创建指定值填充的数组，适合均值基线。`np.empty`分配数组但不把元素初始化为约定数值，其内容不能在完整赋值之前使用。`zeros_like(X)`沿用`X`的形状和通常的数据类型；若`X`是整数，不能假定新数组就适合存标准化小数。

4.4 索引、切片、布尔筛选与共享内存

二维索引与保留轴

```
X = np.array([[1., 2., 3.], [4., 5., 6.]])
print(X[1, 2])
print(X[0].shape)
print(X[:, 1].shape)
print(X[:, 1:2].shape)
print(X[0:1, :].shape)
```

依次为 6.0、(3,)、(2,)、(2, 1)和(1, 3)。单个整数索引选定该轴的一项并去除该轴；切片即使只选一项也保留该轴。二维写法X[i, j]直接说明两个轴的索引，通常比连续X[i][j]更容易核对。

> **例题 4.1：同一列的两种形状**
>
> X.shape == (5, 3)。预测X[:, 0]、X[:, 0:1]、X[0, :]、X[0:1, :]的形状，并说明哪些仍是二维数组。

解 依次为(5,)、(5, 1)、(3,)、(1, 3)。第二、四项仍有两个轴。若模型接口要求 $n \times d$ 输入，一个样本也应保留为(1, d)；若指标要求一维标签，就应有意取成(n,)，而非随意保留或去除轴。

基本切片的视图与显式复制

视图（view）是共享同一底层数据的数组表示，可以有不同形状、偏移和步长。数值数组的基本切片通常给出视图，与第 2 章列表切片建立新外层列表的行为不同。

```
a = np.array([[1., 2.], [3., 4.]])
view = a[:, :1]
independent = a[:, :1].copy()
view[0, 0] = 99
independent[1, 0] = -1
print(a)
print(independent)
print(np.shares_memory(a, view))
```

原数组变为[[99., 2.], [3., 4.]]；独立副本为[[1.], [-1.]]；共享检查为真。view is a虽为假，两者仍共享数据；身份不同不能推出数据互相独立。

a = np.asarray(source)用于取得数组表示；若输入已经是兼容类型的数组，它可能直接返回同一对象。需要保护调用者输入时，不应只写asarray就假定复制完成。np.array(source, copy=True)或转换后的copy()可以明确请求独立数值缓冲区。

视图可避免复制大块数据，但也会延长底层数据的存活。例如从很大的数组取得一个小切片，若仍引用原缓冲区，保留小切片可能继续占着大块内存。是否复制要根据共享语义与内存用途决定，不能一律认为“视图总更好”。

整数数组索引与布尔筛选

高级索引（advanced indexing）包括整数数组和布尔数组等形式。读取时通常产生新数组：

```
a = np.array([[1., 2.], [3., 4.], [5., 6.]])
picked = a[[0, 2]]
picked[0, 0] = 99
mask = a[:, 0] >= 3
print(mask)
print(a[mask])
print(a[0, 0])
```

布尔掩码为`[False True True]`，筛选后保留后两行；原`a[0, 0]`仍为 1.0。**掩码**（mask）在这里是决定保留哪些位置的布尔数组。多个条件使用`&`、`|`与`~`并分别加括号，例如`(a[:, 0] >= 3) & (a[:, 1] < 6)`。Python 的`and`不是逐元素数组逻辑操作。

直接赋值`a[mask] = 0`会修改原数组，因为它以原数组为赋值目标；不能根据“读取高级索引得到副本”推断这句也不修改原数组。`a[mask][:, 0] = 0`则先得到临时副本，再改副本，往往不改变`a`。避免依赖连续索引的隐含中间对象，直接指定清楚的赋值位置。

`X[[0, 1], [1, 2]]`按位置配对，取`X[0, 1]`和`X[1, 2]`，不是取两行两列的全部交叉组合。若要子矩阵，可以写`X[np.ix_([0, 1], [1, 2])]`。复杂索引先在小矩阵上核对实际值，再用于数据集。

4.5 逐元素运算、归约与轴的消去

逐元素运算与通用函数

逐元素运算（elementwise operation）按对应位置处理数值，例如`a + b`、`a * b`、`a ** 2`、`np.abs(a)`、`np.sqrt(a)`与`np.exp(a)`。形状相同时，每个输出位置只对应相同位置的输入；形状不同时需按下一节的广播规则处理。

NumPy 的许多数值函数属于**通用函数**（universal function，ufunc），以数组为单位实现逐元素操作。`math.sqrt`主要面向标量，不能把一个普通多元素数组直接当作标量；数组平方根用`np.sqrt`。函数名相似不意味着接口相同。

对真实值和预测均为`(n,)`的非空有限数组，可计算：

```
y = np.array([2., 4., 6.])
predicted = np.array([3., 4., 4.])
errors = predicted - y
mae_value = np.mean(np.abs(errors))
mse_value = np.mean(errors ** 2)
print(errors, mae_value, mse_value)
```

有符号误差为 1、0、−2，MAE 为 1，MSE 为 5/3，与第 1 章手算一致。若输入含非有限值，平均也可能成为 NaN 或无穷，因此指标入口仍需验证，不能只把循环替换

为一行表达式。

sum、mean 与 axis 的含义

归约（reduction）沿一个或多个轴汇总数值，从而消去相应轴。对 $X \in \mathbb{R}^{n\times d}$，沿轴 0 求和得到每个特征在所有样本上的总和：

$$s_j = \sum_{i=1}^{n} X_{ij}, \qquad \mathbf{s} \text{ 的数组形状为} (d,).$$

沿轴 1 求和则得到每个样本的特征总和：

$$t_i = \sum_{j=1}^{d} X_{ij}, \qquad \mathbf{t} \text{ 的数组形状为} (n,).$$

`axis`指出被汇总、消去的轴。不能把`axis=0`机械记成“每行”，它沿样本轴把各行合并，剩下列对应的结果。

```
X = np.array([[1., 2., 3.], [4., 5., 6.]])
print(X.sum(axis=0))
print(X.sum(axis=1))
print(X.mean(axis=0))
print(X.mean())
```

依次为`[5. 7. 9.]`、`[6. 15.]`、`[2.5 3.5 4.5]`和 3.5。未指定轴时，默认对全部元素汇总。不同物理单位的特征直接求总和通常没有任务意义，此处只是解释数组规则。

keepdims、布尔归约与位置

`keepdims=True`将被汇总轴保留为长度 1，而不是完全移除：

```
column_mean = X.mean(axis=0, keepdims=True)
row_mean = X.mean(axis=1, keepdims=True)
print(column_mean.shape)
print(row_mean.shape)
```

形状为`(1, 3)`和`(2, 1)`。该参数使结果更容易沿预期方向广播回原数组；它不是多计算几个均值，而是保留轴信息。

`np.isfinite(X)`返回逐位置布尔数组；`.all()`判断是否全部为真；`.any()`判断是否至少一项为真。`if X`对普通多元素数组会因真假含义不明确而报错，应根据意图写`X.size == 0`、`mask.any()`或`mask.all()`。

`argmax`与`argmin`返回位置，不返回最大或最小值。`X.argmax(axis=1)`给出每行最大值的列下标；默认不指定轴时按展平顺序给出位置，同分时通常返回首先遇到者。若用该位置关联其他字段，应明确它属于哪个轴，不能把局部列下标当成样本编号。

例题 4.2：按列中心化的手算

对 $X=\left(\begin{smallmatrix}1&2&3\\4&5&6\end{smallmatrix}\right)$，先求每列均值，再从每个元素减去所属列均值。写出结果、代码和结果均值。

解　列均值为 $(2.5, 3.5, 4.5)$。两行分别减去同一均值行，得到

$$X_c=\begin{pmatrix}-1.5&-1.5&-1.5\\1.5&1.5&1.5\end{pmatrix}.$$

代码为`Xc = X - X.mean(axis=0, keepdims=True)`。`Xc.mean(axis=0)`应在浮点容差内为零。若改为`axis=1`，就减去每行自身均值，得到两行均为 $(-1, 0, 1)$，这是另一种操作，不能因同样叫“减均值”就混用。

4.6　广播：不同形状怎样逐元素配对

从标量到逐列操作

`X + 1`把同一个标量加到每个元素上，不需要先手工创建一个全 1 数组。对形状`(n, d)`的`X`，加上形状`(d,)`的`offset`，常用于给每个特征加不同常数。这个规则称为广播（broadcasting）。

广播规定下标怎样对应，不必真的把较小输入复制成大数组。但是运算的输出和某些中间量仍可能很大。输入不复制与整个表达式不消耗内存是不同判断。

规则：从右向左比较各轴

对逐元素运算，将各数组的形状从右侧对齐。每一对轴长度必须相同，或其中一个为 1；缺少的前导轴按长度 1 理解。兼容时，长度 1 的轴可在逻辑上重复使用，输出保留对应的非 1 长度；相同长度则保持该长度。存在一对不相等且都不为 1 的轴，就不能按该广播规则运算。

本章实例轴长均为正。长度为 0 的数组另需按接口处理，不能先用广播绕过非空验证。

对`(2, 3)`与`(3,)`，先把后者理解为`(1, 3)`：最后轴 3 与 3 相同，前轴 2 与 1 兼容，输出`(2, 3)`。对`(2, 3)`与`(2,)`，右侧 3 与 2 冲突，因此失败。若要每个样本加一个值，应把后者明确变为`(2, 1)`。

输入形状	是否兼容	输出形状
(5, 3)与(3,)	是	(5, 3)
(5, 3)与(5, 1)	是	(5, 3)
(5, 1)与(1, 3)	是	(5, 3)
(5, 3)与(5,)	否	末轴 3 与 5 冲突
(2, 1, 3)与(4, 3)	是	(2, 4, 3)

广播只认识轴的位置和长度，不认识“这个轴是样本还是特征”。若 $n = d$，给(n, d)加(n,)可以运行，但会沿最后特征轴配对，可能偏离逐样本加偏移的意图。测试时刻意使用不同的样本数和特征数，有助于让此类错误暴露。

用下标解释扩展方向

设 $X \in \mathbb{R}^{n\times d}$，每个特征有偏移 a_j，则

$$Z_{ij} = X_{ij} + a_j.$$

这里 a 不依赖样本下标 i，因此用a.shape == (d,)或(1, d)。若每个样本有偏移 b_i，则

$$U_{ij} = X_{ij} + b_i,$$

此时 b 不依赖特征下标 j，应显式写成(n, 1)。

```
X = np.array([[1., 2., 3.], [4., 5., 6.]])
feature_offset = np.array([10., 20., 30.])
sample_offset = np.array([100., 200.])
print(X + feature_offset)
print(X + sample_offset[:, None])
```

第一个结果为两行 $(11, 22, 33)$ 和 $(14, 25, 36)$；第二个结果为 $(101, 102, 103)$ 和 $(204, 205, 206)$。看下标中哪个符号缺失，比只背“横着广播”“竖着广播”更能推广到多轴数据。

静默错误：标签向量与预测列数组

例题 4.3：零误差为何变成 4/3

真实值与预测定义如下。两者数值看似相同，计算均方误差为什么不是 0?

```
y = np.array([1., 2., 3.])
p = np.array([[1.], [2.], [3.]])
print(np.mean((p - y) ** 2))
```

解 p.shape == (3, 1),y.shape == (3,)按广播变为(1, 3)。相减的输出为(3, 3)

，元素是 $p_i - y_j$，而非只计算 $p_i - y_i$：

$$p - y = \begin{pmatrix} 0 & -1 & -2 \\ 1 & 0 & -1 \\ 2 & 1 & 0 \end{pmatrix}.$$

九个平方的和为 12，平均得到 $12/9 = 4/3$。想要的有符号误差只对应主对角线上的三个 0；其余六项比较了不同样本。

若接口规定一维标签和预测，应先检查`p.shape == y.shape`，本例有意转换为`p[:, 0]`后再计算。若接口采用二维列数组，则应将`y`转换成`y[:, None]`。无条件`reshape(-1)`可能把多输出预测也压成一条长向量，掩盖原结构，因此形状调整必须有语义依据。

更一般地，若 $p_i = y_i$，却把所有两两差都计算出来，则

$$\begin{aligned} \frac{1}{n^2}\sum_{i=1}^{n}\sum_{j=1}^{n}(y_i - y_j)^2 &= \frac{2}{n}\sum_{i=1}^{n} y_i^2 - 2\left(\frac{1}{n}\sum_{i=1}^{n} y_i\right)^2 \\ &= 2\left[\frac{1}{n}\sum_{i=1}^{n}(y_i - \bar{y})^2\right]. \end{aligned}$$

第一步展开平方，并分别对 i,j 求和；第二步使用 $\bar{y} = n^{-1}\sum_i y_i$。错误结果等于以分母 n 定义的数据方差的两倍，本例方差为 $2/3$。这个推导也说明：只有全体标签相同时，该错误才可能偶然返回 0。

4.7 矩阵乘法与维度核对

从一个样本的加权和到批量预测

第 1 章的直线模型是 $\hat{y} = wx + b$。多个特征时，用每个特征各自的权重做加权和：

$$\hat{y}_i = \sum_{j=1}^{d} X_{ij} w_j + b.$$

X_{ij} 是第 i 个样本的第 j 个特征；w_j 是该特征权重；b 是所有样本共用的偏置。若目标单位为元，w_j 的单位应为元除以对应特征单位，各乘积才具有相同单位。

矩阵乘法（matrix multiplication）把上述“沿特征求乘积和”写成

$$\hat{\mathbf{y}} = X\mathbf{w} + b\mathbf{1}_n.$$

数学中 $\mathbf{w} \in \mathbb{R}^d$，$\mathbf{1}_n$ 表示 n 个 1；NumPy 采用`X.shape == (n, d)`、`w.shape == (d,)`时，`X @ w`得到`(n,)`，再把标量`b`广播到每个样本。符号`@`表示矩阵乘法，`*`仍是逐元素乘法。

例题 4.4：乘积和与逐元素乘积

设

$$X = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}, \quad \mathbf{w} = \begin{pmatrix} 2 \\ -1 \end{pmatrix}, \quad b = 4.$$

计算批量预测；比较`X @ w`、`X * w`与`(X * w).sum(axis=1)`。

解 三行加权和分别为 $2-2=0$、$6-4=2$、$10-6=4$，加偏置得到 $(4,6,8)$。`X @ w` 为长度 3 数组 $(0,2,4)$；`X * w`广播后为

$$\begin{pmatrix} 2 & -2 \\ 6 & -4 \\ 10 & -6 \end{pmatrix},$$

形状为`(3, 2)`。随后沿特征轴 1 求和才得到 $(0,2,4)$。两个表达式在实数运算中相同，浮点求和顺序不同可能带来微小差异。

```
X = np.array([[1., 2.], [3., 4.], [5., 6.]])
w = np.array([2., -1.])
print(X @ w + 4.0)
print((X * w).sum(axis=1) + 4.0)
```

两个矩阵的乘法与单轴特例

若 $A \in \mathbb{R}^{m\times d}$、$B \in \mathbb{R}^{d\times k}$，乘积 $C = AB \in \mathbb{R}^{m\times k}$ 定义为

$$C_{i\ell} = \sum_{j=1}^{d} A_{ij} B_{j\ell}.$$

内侧长度 d 必须相等，结果保留外侧长度 m, k。代码`A @ B`按此规则运算。一般 $AB \neq BA$；即使两个方向都能计算，也可能代表不同含义。

左、右操作数形状	@结果形状	含义
`(n, d)`与`(d,)`	`(n,)`	每行与同一权重做内积
`(n, d)`与`(d, 1)`	`(n, 1)`	保留二维列结果
`(n, d)`与`(d, k)`	`(n, k)`	每个样本产生 k 个输出
`(d,)`与`(d,)`	`()`	两个一维数组内积

一维数组的@具有专门的升轴再去轴规则，不必把`(d,)`强行解释成既是行又是列。多轴`matmul`将最后两个轴作为矩阵轴、前导轴作为批次轴并按规则广播；本章实验主要采用一维与二维，后续批量神经网络再展开多轴乘法。

`np.dot`在一维、二维常见情形与熟悉的内积、矩阵乘法有关，但高维规则与`matmul`并不完全相同。本书矩阵乘法优先写`@`，减少在尚未核对高维语义时混用名称。

带形状的计算记录

对每个表达式同时写出输入和输出，比报错以后随机转置更有效。例如：

$$
\begin{aligned}
X : (n, d), \quad W : (d, k), \quad b : (k,), \\
S = XW : (n, k), \\
Z = S + b : (n, k), \\
\operatorname{mean}(Z, \text{axis} = 0) : (k,).
\end{aligned}
$$

W 的每一列对应一个输出的权重；b 的每项对应一个输出的偏置。若 b 表示每个样本的偏置，则应采用`(n, 1)`，语义已经改变。线性代数公式与数组形状相互核对，第 6 章将用同一习惯检查投影、最小二乘与矩阵微分。

4.8 随机数、固定种子与复现条件

生成器、状态与种子

计算机中的常用随机数是**伪随机数**（pseudorandom numbers）：由算法和内部状态生成，统计表现类似所需分布，但给定相同条件可以重建序列。**种子**（seed）用于初始化状态，不是每次抽样都重新计算一次的固定结果。

```
rng = np.random.default_rng(2026)
a = rng.normal(size=(2, 3))
b = rng.normal(size=(2, 3))
print(a.shape, b.shape)
```

`default_rng`创建随机数生成器对象；`normal`默认从均值 0、标准差 1 的正态分布抽样。同一个生成器连续调用会推进状态，`a`与`b`通常不同。每次都重新创建同种子生成器，则会反复得到序列起点，可能把本应独立的扰动变成重复数据。

常用方法包括`rng.integers(low, high, size=...)`生成下界包含、上界通常不包含的整数；`rng.uniform`生成均匀抽样；`rng.permutation(n)`返回 0 至 $n-1$ 的随机排列。分布含义在第 8 章系统说明，本章只用它们产生可核对的教学输入。

配对打乱必须使用同一个排列

```
X = np.array([[1., 10.], [2., 20.], [3., 30.]])
y = np.array([100., 200., 300.])
rng = np.random.default_rng(2026)
order = rng.permutation(len(X))
X_shuffled, y_shuffled = X[order], y[order]
print(X_shuffled[:, 0] * 100 == y_shuffled)
```

三个布尔结果都应为真，因为两边使用同一个排列。若分别打乱特征和标签，两

次随机调用会推进状态，可能采用不同排列并破坏对应关系。同一种子并不意味着同一个生成器的两次调用得到相同结果。

打乱不自动意味着合理划分。时间任务、用户分组任务仍需第 1 章的约束；随机抽样会影响哪些样本进入各集合，应保存最终索引或标识。只有保存种子而未保存原始行顺序，输入顺序一变就可能得到另一组划分。

相同种子的适用范围

固定生成器类型、实现版本、种子和调用顺序，可重复生成相同序列；NumPy 生成器不承诺所有未来版本和环境下完整随机流永远不变。跨环境复现应记录 NumPy 版本、生成器类型以及最终数据或索引。不同算法调用可能消耗不同数量随机数，因此在前面增加一个随机抽样会影响后续结果。

可把划分与噪声生成分别交给独立生成器，减少无关修改的相互影响。对大型项目还可能有多个进程、不同库和硬件中的随机状态，第 25 章与第 44 章再展开。本章实验 4.2 采用固定训练、验证、测试数据，随机种子只用于数值性能对照输入，不把它当成划分记录。

4.9 数值稳定性与浮点核对

范围、舍入与非有限结果

数学实数不受计算机位数限制，浮点数则只能表示有限范围内离散的近似数。有限输入经过运算仍可能溢出，例如大数平方或大指数；极小正数也可能下溢到 0。输出警告不一定使程序立即停止，后续计算可能继续传播无穷或 NaN。

`np.isfinite(result).all()`检查结果是否全部有限。若问题本来不允许无穷，应在适当边界检查并停止，而非用`nan_to_num`无说明地替换成 0。缺失值处理是数据规则，数值溢出是计算问题，不能靠同一种替换掩盖。

`np.errstate`可以在局部改变浮点异常的处理方式：

```
with np.errstate(over="raise", invalid="raise", divide="raise"):
    result = np.exp(np.array([1000.]))
```

这段故意演示失败路径，在常见`float64`环境中会抛`FloatingPointError`。它帮助尽早发现失效位置，但不自动修复计算方法。不要把故意失败片段直接加入必须从头成功完成的实验入口。

先减最大值再取指数

某些模型要把一组实数分数转成非负且和为 1 的数。设 $z_1, \dots, z_k$ 为有限分数，直接定义

$$p_j = \frac{\exp(z_j)}{\sum_{\ell=1}^{k} \exp(z_\ell)}.$$

这里先把它作为数值归一化规则；第 13 章和第 30 章再讲其模型意义。分数为 1000、1001、1002 时，直接取指数可能溢出。令 $M = \max_j z_j$，分子分母同时除以 $\exp(M)$，得

到实数意义下等价的表达式

$$p_j = \frac{\exp(z_j - M)}{\sum_{\ell=1}^{k} \exp(z_\ell - M)}.$$

所有指数输入都不大于 0，最大项指数为 1，分母至少为 1，可避免正向指数溢出。非常小的项仍可能下溢到 0，但不会令所有项同时为 0。

例题 4.5：稳定归一化与按行计算

一批数据有两行分数 $(1000, 1001, 1002)$ 和 $(-1000, -1000, -1000)$。写出稳定计算，说明每个中间数组形状，并给出近似结果。

```
z = np.array([[1000., 1001., 1002.], [-1000., -1000., -1000.]])
shifted = z - z.max(axis=1, keepdims=True)
exp_values = np.exp(shifted)
p = exp_values / exp_values.sum(axis=1, keepdims=True)
print(p)
print(p.sum(axis=1))
```

解 行最大值形状`(2, 1)`，广播减法后`shifted`为`(2, 3)`，两行分别为 $(-2, -1, 0)$ 和 $(0, 0, 0)$。指数数组也为`(2, 3)`，行分母为`(2, 1)`。结果近似为

$$\begin{pmatrix} 0.090031 & 0.244728 & 0.665241 \\ 1/3 & 1/3 & 1/3 \end{pmatrix},$$

各行和为 1。归一化得到和为 1 的数，不自动证明它们是经过校准的真实发生概率。若输入整行含无穷或 NaN，以上有限性前提不成立。

接近数相减与运算顺序

两个很大且非常接近的数相减，会放大输入近似误差对差值的影响。后文常用恒等式

$$\|\mathbf{x} - \mathbf{q}\|^2 = \|\mathbf{x}\|^2 + \|\mathbf{q}\|^2 - 2\mathbf{x}^{\mathrm{T}}\mathbf{q}$$

计算距离，但右侧可能在大数之间发生抵消，得到微小负数，甚至丢失应有的小距离。将负值截到 0 能修正部分舍入表现，却不能恢复已经损失的精度。对规模小且近邻差异细的教学数据，本章完整实验优先直接计算差后平方。这种形式减少抵消风险，却仍会在差值或平方过大时溢出，必须另外检查。

求和顺序也影响浮点结果。数学加法满足结合律，浮点近似加法不保证严格满足。循环、向量化和底层矩阵库可能采用不同顺序，所以应比较合理误差，而不要求所有位完全相同。若差异远大于容差，应先检查类型、形状和公式，再讨论舍入。

绝对容差、相对容差与先查形状

常用近似检查可写为

$$|a - b| \le \mathrm{atol} + \mathrm{rtol}|b|.$$

atol是绝对容差，适合接近 0 的量；rtol是相对容差，随参考量 b 的尺度变化。np.allclose按类似规则比较数组，它还可能广播，因此检查前应先确认形状相同。

```
if loop_result.shape != vector_result.shape:
    raise ValueError("result shapes differ")
np.testing.assert_allclose(
    loop_result, vector_result, rtol=1e-12, atol=1e-12
)
```

np.testing.assert_allclose失败时会给出差异信息，便于实验核对。上述容差适用于本章小尺度float64计算，并非所有模型通用精度标准。float32、长求和或病态问题可能需要不同容差，容差也不能大到掩盖公式错误。含 NaN 的数据首先应按任务规则拒绝，不把“两个地方都 NaN”当成正确一致。np.testing.assert_allclose默认允许相同位置的 NaN 通过核对；需要明确拒绝时，可先检查np.isfinite或设置equal_nan=False。

4.10 从逐样本循环到向量化

保留一个可手算的循环实现

对批量线性预测，先写直接对应公式的循环：

```
def predict_loop(X, w, b):
    result = []
    for row in X:
        value = b
        for feature, weight in zip(row, w):
            value += feature * weight
        result.append(value)
    return np.array(result)
```

该教学核心假定X为非空有限二维数组、每行长度等于len(w)、w为一维、b为有限标量；通用接口应先验证。zip仍会截短不等长输入，因此数组并未让第 2 章的配对问题消失。

计算顺序为对每个样本建立一个累加器，依次乘每个特征权重，最后保存预测。时间量级为 $O(nd)$，结果空间为 $O(n)$。循环实现保留为可独立对照的参考，不必为了代码短而立即删除。

先去掉内层循环，再去掉外层循环

第一步让 NumPy 完成一行的逐元素乘积与归约：

```
result = []
for row in X:
    result.append(np.sum(row * w) + b)
result = np.array(result)
```

第二步让全部样本一起计算：

```
result = np.sum(X * w, axis=1) + b
```

形状为(n, d)的X乘(d,)的w，先广播得到每样本每特征贡献，再沿轴 1 归约。第三步利用矩阵乘法表达同一加权和：

```
result = X @ w + b
```

两种批量表达式都保留 $O(nd)$ 算术量级；速度改进主要来自执行方式和内存访问，而不是从 nd 次计算变成只算一次。X * w通常建立 $n \times d$ 的中间结果；矩阵乘法可避免显式存储这份乘积，并可能调用优化数值实现。

评价向量化的正确顺序

先用例题 4.4 的三行数据核对三个实现值与形状，再生成较多数据比较误差，最后计时。**基准测试**（benchmark）是在规定输入与环境下测量性能；首次调用可能受到初始化、缓存等影响，因此应先预运行，再重复测量，报告中位数或其他明确统计量。

time.perf_counter()提供适合测量间隔的计时值。计时时不要让一个实现同时读取文件、另一个只做计算；不要让一个实现包含打印、另一个没有。多线程数值库、后台负载、数据形状和规模都会影响实际耗时，本章不把固定加速倍数作为通过条件。

np.vectorize提供便于把标量函数应用到数组的接口，但主要仍采用 Python 层循环，不能仅凭名称认为它等同于高效数值向量化。对小输入、复杂分支或逐文件处理，普通循环往往更清楚；对共享运算规则的大批数值，数组操作更自然。

4.11 成对距离：广播的用途与内存代价

从一个查询到全部查询

设训练数组 $X \in \mathbb{R}^{n \times d}$，查询数组 $Q \in \mathbb{R}^{m \times d}$。第 a 个查询到第 i 个训练样本的平方欧氏距离为

$$D_{ai} = \sum_{j=1}^{d} (Q_{aj} - X_{ij})^2, \qquad D \in \mathbb{R}^{m \times n}.$$

欧氏距离（Euclidean distance）是各坐标差的平方和再开平方；比较最近顺序时，平方根是单调的，可以直接比较平方距离，节省一次开方。各特征量纲不同会影响距离，下一节实验先用训练统计量缩放。

构造Q[:, None, :]，形状为(m, 1, d)；构造X[None, :, :]，形状为(1, n, d)。广播后得到全部三重下标的差：

```
delta = Q[:, None, :] - X[None, :, :]
distance2 = np.sum(delta ** 2, axis=2)
```

delta[a, i, j]对应 $Q_{aj} - X_{ij}$，沿轴 2 归约去掉特征，留下查询和训练样本两个轴。若误写axis=1，就把训练样本轴求和，输出(m, d)，不能用于逐查询找训练邻居。

例题 4.6：三维中间数组的手算

训练样本为 $(0,0),(2,0),(0,2)$，查询为 $(1,0),(0,3)$。写出平方距离矩阵和最近样本下标。同分时取训练顺序靠前者。

解 第一个查询到三点的平方距离为 1、1、5，第二个为 9、13、1。因此

$$D = \begin{pmatrix} 1 & 1 & 5 \\ 9 & 13 & 1 \end{pmatrix}.$$

两个查询对应最近下标 0、2，使用 Python 从 0 开始的编号。中间`delta`形状为`(2, 3, 2)`，归约后为`(2, 3)`。`distance2.argmin(axis=1)`得到每行最小距离的训练下标，不应沿轴 0 找“每个训练样本最近的查询”。

k 个近邻、排序与索引标签

若每个查询取距离最小的 k 条训练标签平均，可写：

```
order = np.argsort(distance2, axis=1, kind="stable")[:, :k]
predicted = y_train[order].mean(axis=1)
```

`argsort`返回排序后的下标，`kind="stable"`使距离同分时保持训练原顺序；`order.shape == (m, k)`。`y_train`形状为`(n,)`，用二维整数下标取得`(m, k)`标签数组，沿近邻轴 1 平均，得到`(m,)`预测。

本章采用完整排序，距离计算需 $O(mnd)$，排序通常需 $O(mn\log n)$ 时间，距离矩阵需 $O(mn)$ 空间，显式差数组需 $O(mnd)$ 空间。完整近邻算法、部分选择与检索结构在第 15 章讨论；本章重点在于公式与轴对应。

批次计算与更省内存的恒等式

若 $m = n = 10000$、$d = 100$，`delta`含 10^{10} 个元素，单独一份`float64`数据约需 80 GB，平方等中间结果还会增加峰值。输入数组本身不算大，并不表示成对广播输出也能放入内存。

批次（batch）把查询分成若干小组处理，组内向量化，组间循环。若每组最多 B 个查询，差数组由 mnd 缩为 Bnd，全部距离运算的总量级不变。对于只需要预测的任务，每组完成后可只保留其预测，不存全部距离矩阵。

```
blocks = []
for start in range(0, len(Q), block_size):
    part = Q[start:start + block_size]
    blocks.append(predict_vectorized(X, y_train, part, k))
predicted = np.concatenate(blocks)
```

此片段假定查询非空、批大小为正整数，完整答案补充边界检查。`concatenate`沿已有轴连接预测，不额外增加批次轴；`stack`则增加新轴，适用于等形数组的另一种组织目的。

也可按恒等式写出距离矩阵：

```
distance2 = (
    np.sum(Q ** 2, axis=1, keepdims=True)
    + np.sum(X ** 2, axis=1)[None, :]
    - 2 * (Q @ X.T)
)
```

三项分别为`(m, 1)`、`(1, n)`和`(m, n)`，可避免显式`(m, n, d)`差数组。但仍需存储 $m \times n$ 结果，并有前述大数抵消风险。优化应同时考虑计算量、峰值内存与数值条件，不能只比较代码行数。

4.12 实验 4.1：形状、共享内存与稳定计算

入口为配套实验`/ch04_arrays.py`，依赖 NumPy，无需下载数据。实验使用 2 行 3 列的数组，依次检查形状属性、按轴归约、按列中心化、视图修改、高级索引副本、错误广播和稳定归一化。每一步采用前面已手算的数值，因此可先预测再执行。

```
python3 ch04_arrays.py
```

关键输出为：

```
shape ndim size: (2, 3) 2 6
column shapes: (2,) (2, 1)
sum axis=0: [5. 7. 9.]
sum axis=1: [ 6. 15.]
shares: True False
wrong residual shape: (3, 3)
wrong MSE: 1.3333333333333333
correct MSE: 0.0
same seed same draws: True
all checks passed
```

中心化结果为两行全 -1.5 和全 1.5；通过视图修改后原数组第一项成为 99，高级索引副本上的修改不再影响原数组。稳定归一化输出应与例题 4.5 一致，各行和与 1 之差不超过 10^{-12}。

脚本使用`np.set_printoptions`统一小数显示，这只改变数组的显示方式，不改变计算精度。`suppress=True`减少小数显示为科学计数法，并不会把小数截为 0。屏幕显示 0 可能只是精度设置下的舍入，应通过差值或更高显示精度判断。

修改任务如下：把列切片`a[:, :2]`改成`a[:, :2].copy()`，核对原数组不再受影响；把行分母中的`keepdims=True`去掉，分析为什么`(2, 3)`除`(2,)`失败；把归一化改成`axis=0`，说明此时哪一方向和为 1；把预测列数组改为一维，核对错误 MSE 归零。每次修改只处理一个因素并恢复原版，避免前面的原地修改影响后续手算。

4.13 实验 4.2：训练缩放、批量近邻与性能对照

任务、数据与假设

入口为配套实验`/ch04_numeric.py`，只依赖 Python 标准库和 NumPy，CPU 可运行。任务是以 3 个数值特征预测连续目标，比较 $k = 1$ 与 $k = 3$ 的近邻平均。第一特征可理解为一个主要数值量，第二特征用于演示不同尺度，第三特征恒为 1，用于检查零标准差处理；三者与目标均是教学构造，不声称真实因果关系。

训练数据为

$$X_{\text{train}} = \begin{pmatrix} 0 & 10 & 1 \\ 1 & 20 & 1 \\ 2 & 10 & 1 \\ 3 & 20 & 1 \end{pmatrix}, \qquad \mathbf{y}_{\text{train}} = (4, 6, 8, 10).$$

验证与测试数据如下：

集合	特征 1	特征 2	特征 3	目标
验证	0.2	11	1	4.4
验证	1.2	19	1	6.4
验证	2.2	11	1	8.4
验证	2.8	19	1	9.6
测试	0.4	12	1	4.8
测试	1.6	12	1	7.2
测试	2.4	12	1	8.8
测试	3.5	22	1	11.0

数组形状都为`(4, 3)`，标签为`(4,)`。这些集合预先固定，不在本实验中重新随机划分。候选按验证 MAE 选择，同分时取声明顺序靠前的 k；同距离近邻按训练顺序；不合并验证集重训。流程与实验 3.2 一致，但数组计算和预处理多了一层形状要求。

只从训练数组估计标准化参数

标准化（standardization）在本节指对每个特征减训练均值，再除以训练标准差。它把尺度不同的特征转换成更可比较的无量纲数值，但不自动去除相关性，也不保证任何算法都会变好。对第 j 列定义

$$\mu_j = \frac{1}{n} \sum_{i=1}^{n} X_{ij},$$

$$s_j = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (X_{ij} - \mu_j)^2},$$

$$Z_{ij} = \frac{X_{ij} - \mu_j}{\tilde{s}_j}, \qquad \tilde{s}_j = \begin{cases} s_j, & s_j > 0, \\ 1, & s_j = 0. \end{cases}$$

这里采用分母 n，作为本实验确定的缩放规则，对应`std(ddof=0)`。`ddof`为分母自由度修正量，设为 1 时分母变为 $n-1$，统计估计含义在第 8 章讨论。不要在训练和预测中采用不同约定。

训练列均值为 $(1.5, 15, 1)$；第一列平方离差为 2.25、0.25、0.25、2.25，均值 1.25，因此标准差 $\sqrt{1.25} \approx 1.118034$；第二列标准差 5；第三列标准差 0，使用替代尺度 1。训练后第三列标准化值全为 0。

```
mean = X_train.mean(axis=0, keepdims=True)
std = X_train.std(axis=0, ddof=0, keepdims=True)
scale = np.where(std == 0, 1.0, std)
Z_train = (X_train - mean) / scale
Z_val = (X_val - mean) / scale
Z_test = (X_test - mean) / scale
```

`mean`、`std`和`scale`形状均为`(1, 3)`，广播到各集合。`np.where(condition, a, b)`逐位置选择两个输入中的值；此处在除法之前建立安全尺度。若先计算含零除的表达式，再放入`where`，可能已经触发无效除法，因为普通函数参数通常会先求值。

均值与尺度只从训练数据估计，验证和测试复用它们。若分别给各集合减自身均值、除自身标准差，单个新样本就无法按训练规则定位，评价条件也发生变化。若在全部数据上估计，再切出集合，则把后续输入信息用于训练预处理，形成第 1 章所述泄漏。

手算一个查询的距离

第一条验证查询为 $(0.2, 11, 1)$。同一训练缩放下，查询到第 i 条训练点的平方距离可以直接写为

$$D_i = \frac{(0.2 - X_{i1})^2}{1.25} + \frac{(11 - X_{i2})^2}{25} + (1 - X_{i3})^2.$$

同一均值在相减中抵消，但尺度仍影响各坐标的权重。四个距离依次为 0.072、3.752、2.632、9.512，排序下标为 0、2、1、3。

$k = 1$ 预测 4，对真实 4.4 的绝对误差 0.4；$k = 3$ 平均前三条标签 $(4 + 8 + 6)/3 = 6$，误差 1.6。这里第二特征的尺度被缩到与第一特征较接近；若直接用原数值距离，第二特征差异会获得不同权重。

训练常数列在当前各集合中均为 1，因此距离贡献为 0。若某个查询在该列变为 $q_j \neq 1$，它到每条训练记录都会增加相同的 $(q_j - 1)^2$：距离数值改变，但对这个查询的近邻排序不变。该变化仍应检查来源，不能从训练列恒定推断未来也必然恒定。第 10 章再讨论常数特征删除及完整预处理流程。

循环与向量化实现的对应

完整入口保留predict_loop和predict_vectorized。前者三层循环计算查询、训练样本和特征；后者用前节广播形成距离矩阵，再稳定排序取得 k 个标签平均。两者采用同一缩放结果和同分规则，避免把算法条件差异误认为数值实现差异。

向量化接口在计算前检查训练与查询均为非空有限二维数组，特征数相同，训练标签恰为(n,)且有限，k 为 1 至 n 的整数。布尔值虽然在 Python 中与整数有关系，本接口不将True当作合法 k。两个实现共用输入验证，先调用前面的as_float_array，避免整数差值回绕。缩放、距离、标签平均和指标计算均在局部np.errstate检查下执行；有限输入若使中间量溢出，会抛FloatingPointError，不会继续把全无穷距离按同分规则选为某个正常预测。遇到这类输入应检查尺度与算法，不能用全局关闭警告代替修复。这些检查不能因底层矩阵运算可能报错就省略，因为有些错误会广播成功。

评价函数同样检查预测与标签均为等形一维非空有限数组：

```
def checked_mae(y, p):
    y = as_float_array(y, "y")
    p = as_float_array(p, "p")
    if y.ndim != 1 or y.size == 0 or p.shape != y.shape:
        raise ValueError("expected paired nonempty 1D arrays")
    with np.errstate(over="raise", invalid="raise", divide="raise"):
        return float(np.mean(np.abs(p - y)))
```

输入先转浮点，才能避免无符号整数减法回绕；例如uint8真实值 255、预测 0 的绝对误差应为 255，直接在原类型中相减却可能得到 1。末尾的float只把 NumPy 数值转换为普通 Python 浮点，便于 JSON 保存，不会提高精度。

运行与预期结果

在配套实验目录运行：

```
python3 ch04_numeric.py
python3 ch04_solutions.py
```

主要输出应为：

```
mean: [[1.5, 15.0, 1.0]]
scale: [[1.118033988749895, 5.0, 1.0]]
validation k=1: 0.400000
validation k=3: 1.466667
selected k: 1
test predictions: [4.0, 8.0, 8.0, 10.0]
test MAE: 0.850000
test baseline MAE: 2.050000
```

$k = 3$ 的完整逐条结果如下，必须按每条查询的距离排序分别取得近邻，不能从第一条查询类推其他样本。

验证行	真实值	$k = 1$ 预测	$k = 3$ 预测	$k = 3$ 绝对误差
1	4.4	4	6	1.6
2	6.4	6	8	1.6
3	8.4	8	22/3	16/15
4	9.6	10	8	1.6

表中 $k = 3$ 误差总和为 88/15，平均 $22/15 \approx 1.466667$，因此按最小验证 MAE 选择 $k = 1$。

所选 $k = 1$ 测试误差为 0.8、0.8、0.8、1.0，平均 0.85。预先声明的训练均值为 7，测试 MAE 为 2.05。不要因这两个最终分数恰与实验 3.2 相同，就认为两份数据和内部距离相同；本实验的训练输入有 3 个特征，最近记录的选择依据也不同。

性能对照与保存结果

同一入口另用种子 2026 生成`(12000, 12)`的正态数组和长度 12 的权重，比较`linear_loop`与`X @ w + 4.0`。先核对输出形状与数值，采用`rtol=1e-12`、`atol=1e-12`，再预运行并各计时 3 次，报告耗时中位数。最大绝对差通常处于 10^{-14} 量级或更小，实际值随数值库和环境变化。

计时不是验收正确性的条件；在小规模、不同线程或机器负载下，速度比会变化。实验不下载数据、不训练大型网络，也不要求 GPU。数值对照回答的是相同公式的两种实现是否一致，不能回答哪种模型泛化更好。

结果目录`ch04_results`包含`summary.json`与`arrays.npz`。JSON 保存版本、种子、均值、尺度、验证指标、选择、测试指标和计时；NPZ 是 NumPy 保存多个命名数组的格式，保留数据类型和形状，适合本实验数值输入与预测。

```
with np.load("ch04_results/arrays.npz", allow_pickle=False) as saved:
    train = saved["train"]
    predictions = saved["predictions"]
    print(train.shape, predictions.shape)
```

这个相对路径示例假定工作目录为配套实验目录。`allow_pickle=False`表示按普通数值数组读取，不启用 Python 对象序列化支持。实际入口仍以自身路径定位输出，避免启动目录影响结果。

JSON 中的计时自然可能每次不同，随机生成数组也应结合版本和调用顺序理解。训练、验证和测试是固定数组，近邻结果应在上述容差内一致。输出文件使用同名覆盖，若上一轮失败而旧文件仍存在，应检查本轮退出状态与日志，不靠“结果文件存在”判断运行成功。

修改任务与结果解释

1. 把标签改为`y_train[:, None]`传给向量化函数，接口应主动拒绝；不要让`mean(axis=1)`在增加的多余轴上偶然返回数值。
2. 删除标准化，比较第一条查询的邻居顺序，解释各坐标差对距离的权重；这是新的预处理方案，应重新完成验证选择，不能直接挑测试指标好的版本。
3. 把第三列全改为 5 并保持各集合一致，均值与尺度记录应相应改变，但该列标准化仍为 0，预测应保持一致。
4. 将最近邻预测按查询分块，批大小分别为 1、2 和大于查询数的 10，核对与完整批量一致。配套答案给出完整实现。
5. 只为数值核对，将 X 和权重改为`float32`，观察差异并使用与精度相称的容差；保留原版`float64`结果供比较，不把不同类型混作同一计时条件。

4.14 常见错误、知识联系与自检

NumPy 程序常见问题可以按信息缺失的位置定位。类型错误要先看`dtype`和转换发生在运算前还是后；形状错误要逐轴核对样本、特征和输出含义；结果被意外修改要检查视图、别名与原地赋值；随机结果变化要检查生成器状态、调用顺序和数据行顺序；数值异常要检查尺度、有限性和运算形式。

“没有报错”只排除了某些不兼容情况。`(n, 1)`与`(n,)`相减会成功，输入输出同为方阵时错误转置也可能成功。因此核对包括：能手算的小输入、不同轴长、预测与标签的严格等形检查、原输入是否改变，以及与独立循环实现的比较。

本章把第 2 章的累加器写成归约，把嵌套循环写成广播与矩阵乘法，把第 3 章引用问题扩展为共享内存。第 5 章会用 pandas 整理带字段名的表格，第 6 章会进一步解释矩阵几何；表格字段名、数组位置和数学符号需要明确对应，不能转换一次就丢掉特征顺序。

自检应能逐步说明：为什么`X.mean(axis=0)`得到每列均值；为什么一维`w.T`没有变成列数组；为什么数组切片后修改可能影响原数据；为什么固定种子不足以保证任意修改后的随机流一致；为什么把距离计算写成一行仍可能用尽内存；为什么标准化参数必须只由训练集确定。

4.15 分层习题

以下编号为 4.1–4.16。数组题同时写值与形状，复杂度题区分输入、输出与中间量。

4.1 **概念。**解释`shape`、`ndim`、`size`和`dtype`。200 个样本、每个样本 10 个特征，数组维数与特征空间维数分别是多少?

4.2 **形状。**令`a = np.arange(12).reshape(3, 4)`，写出`a[1]`、`a[:, 1]`、`a[:, 1:2]`、`a[1:2]`、`a.T`的形状；比较`a.T`和`a.reshape(4, 3)`的第一行。

4.3 **类型。**整数数组`[1, 2]`第一项赋 1.9 后是什么？为什么对像素整数先平方再转浮点不能保证正确?

4.4 **共享**。令`a = np.arange(6).reshape(2, 3)`，`b = a[:, :2]`，`c = a[[0, 1]]`。依次修改`b[0, 0] = 99`、`c[1, 1] = -1`，写出`a`，并说明身份比较能否替代共享内存检查。

4.5 **归约**。对 $A = \left(\begin{smallmatrix}1&2&3\\4&5&6\end{smallmatrix}\right)$，求沿轴 0、轴 1 的和与均值，并写出加`keepdims=True`后的形状。

4.6 **广播**。判断`(4, 3)`与`(3,)`、`(4, 1)`与`(3,)`、`(4, 3)`与`(4,)`、`(2, 1, 3)`与`(5, 3)`能否广播并给出输出形状。

4.7 **静默错误**。重算例题 4.3 的差矩阵与 MSE。若标签为 0、0、0，这一错误为何不容易被发现？怎样设计形状检查？

4.8 **矩阵乘法**。令 $A = \left(\begin{smallmatrix}1&2&3\\4&5&6\end{smallmatrix}\right)$，$w = (1, 0, -1)$，求`A @ w`、`A * w`及其形状。若`w`改为`(3, 1)`，第一项结果怎样变化？

4.9 **公式到代码**。设 $X : (n, d)$，$W : (d, k)$，$b : (k,)$，写出每样本每输出的预测公式与代码；为什么不能任意把`X @ W`换成`X * W`？

4.10 **随机性**。同一生成器连续调用两次`normal`与两次分别新建同种子生成器有什么区别？为特征和标签分别生成随机排列为什么危险？

4.11 **数值稳定**。对分数 1000、1000、1000 写稳定归一化，解释分母为何不会因全部指数下溢成为 0。输入含 NaN 时能否沿用结论？

4.12 **距离手算**。完成例题 4.6，并写出广播中间形状、归约轴和最近下标。若训练标签为 0、4、8，两个查询的单近邻预测是什么？

4.13 **标准化**。训练第一特征为 0、2、4，第二特征均为 1。按分母 3 求均值与尺度，写出训练标准化结果；预测样本 $(6, 1)$ 应该用哪组参数？

4.14 **复杂度**。查询 1000 条、训练 2000 条、每条 50 特征，直接差数组有多少元素？`float64`仅数据约占多少字节？批大小 20 后最大差数组多大？

4.15 **编程**。实现按查询分块的近邻预测，检查批大小与空输入；批大小 1、2、10 应与完整批量预测一致。说明分块降低了什么成本、没降低什么量级。

4.16 **综合实验**。运行实验 4.2 及答案核对。记录自己环境中的最大数值差与耗时；说明为什么速度不同不必然意味着代码错误，并列出保证算法比较公平的三个条件。

4.16 习题参考答案

4.1–4.5：结构、类型与归约

4.1 `shape`是各轴长度的元组，`ndim`是轴数，`size`是元素总数，`dtype`是元素类型。样本数组形状`(200, 10)`，轴数 2，特征空间维数 10，元素数 2000。该约定需要同时说明行对应样本、列对应特征。

4.2 依次为`(4,)`、`(3,)`、`(3, 1)`、`(1, 4)`、`(4, 3)`。`a.T`第一行为 0、4、8；`a.reshape(4, 3)`第一行为 0、1、2。转置交换轴，重排按指定遍历顺序重新组织；它们不因输出形状相同而相等。

4.3 仍为整数`[1, 2]`，1.9 被转换为整数 1。固定宽度整数计算可能先溢出或截

断，之后转为浮点只能转换已错误的数。正确顺序为先转浮点，再相减或平方，例如`a.astype(np.float64) ** 2`。

4.4 最后a为`[[99, 1, 2], [3, 4, 5]]`。b是基本切片视图，修改传到原数据；c来自高级索引副本，修改不影响a。`b is a`为假仍可能共享内存，因此身份比较不能替代`np.shares_memory`。

4.5 轴 0 和为 5、7、9，均值 2.5、3.5、4.5；轴 1 和为 6、15，均值 2、5。默认形状分别为`(3,)`与`(2,)`；保留轴后分别为`(1, 3)`与`(2, 1)`。归约消去的是指定轴，保留轴只改变形状而非数值。

4.6–4.9：广播与乘法

4.6 四组依次为兼容`(4, 3)`、兼容`(4, 3)`、不兼容、兼容`(2, 5, 3)`。第三组最右轴 3 与 4 冲突；第四组将后者补成`(1, 5, 3)`，前两轴分别按 2 与 1、1 与 5 扩展。

4.7 差矩阵为例题中的 3 行 3 列，平方和 12，平均 4/3。全零标签时所有两两差都为 0，错误结果也为 0，因此只用常数标签测试不足。接口先检查`y.ndim == 1`、`p.shape == y.shape`和非空有限性，再计算；不通过时拒绝，而非自动广播。

4.8 `A @ w`为`[-2, -2]`，形状`(2,)`；`A * w`为`[[1, 0, -3], [4, 0, -6]]`，形状`(2, 3)`。权重改成`(3, 1)`后，矩阵乘法结果为`[[-2], [-2]]`，形状`(2, 1)`。数值排列方向变化会影响后续标签配对，不能只看两个 -2。

4.9 $\widehat{Y}_{i\ell} = \sum_j X_{ij} W_{j\ell} + b_\ell$，代码`Y_hat = X @ W + b`，结果`(n, k)`。逐元素乘法没有沿 j 求和，而且一般`(n, d)`与`(d, k)`本身无法广播；即使某些轴长巧合，意义仍不同。

4.10–4.14：随机、稳定性与规模

4.10 同一生成器两次调用推进状态，取得连续但通常不同的数；两次新建相同种子生成器会从各自序列起点开始，在固定条件下取得相同抽样。特征与标签必须使用同一个排列；分开生成两次排列通常破坏原配对。还需保存原数据顺序、生成器版本与最终划分标识。

4.11 减最大值 1000 后为 0、0、0，指数为 1、1、1，结果各为 1/3。一般有限分数中至少一个最大项减去自身为 0，故至少一项指数为 1，分母至少为 1。含 NaN 时最大值与减法可能传播 NaN，有限性假设已被破坏，应先拒绝或按明确的数据规则处理。

4.12 差数组形状`(2, 3, 2)`，沿轴 2 求平方和得到`[[1, 1, 5], [9, 13, 1]]`。沿轴 1 取最小位置得到 0、2，同分取靠前记录，预测 0、8。用`argmin(axis=0)`会回答另一个问题，得到每个训练点最接近的查询编号。

4.13 均值为 $(2, 1)$，第一列方差 $(4 + 0 + 4)/3 = 8/3$，标准差 $\sqrt{8/3}$，第二列尺度按规则设为 1。标准化后第一列为 $(-\sqrt{3/2}, 0, \sqrt{3/2})$，第二列全 0。新样本仍用训练均值与尺度，得到 $(4/\sqrt{8/3}, 0) = (\sqrt{6}, 0)$，不能给单样本另估均值使其变为 0。

4.14 元素数为 $1000 \times 2000 \times 50 = 10^8$，每元素 8 字节，共 8×10^8 字节，约 0.8 GB，不含平方中间量等额外占用。批大小 20 时为 $20 \times 2000 \times 50 = 2 \times 10^6$ 元素，约

1600 万字节，即 16 MB。这里使用十进制 GB 与 MB，二进制单位换算数值不同。

4.15–4.16：编程与实验

4.15 复用完整的`predict_vectorized`，按查询切片处理。核心答案为：

```
def predict_blocked(train, target, query, k, block_size=2):
    if (not isinstance(block_size, int) or isinstance(block_size, bool)
            or block_size < 1):
        raise ValueError("block_size must be positive")
    if len(query) == 0:
        raise ValueError("empty query")
    blocks = []
    for start in range(0, len(query), block_size):
        part = query[start:start + block_size]
        blocks.append(predict_vectorized(train, target, part, k))
    return np.concatenate(blocks)
```

本答案接口假定`query`已经是数组；组内函数继续验证二维、有限、特征数与标签。批大小与 k 一样拒绝布尔值，不把`True`当成 1。答案脚本用训练点 $(0,1),(2,1),(4,1)$、标签 0、4、8 与查询 $(1,1),(3,1),(5,1)$，同分取靠前，预测 0、4、8；三个批大小应完全一致。分块降低峰值中间内存，不降低全部距离计算的 $O(mnd)$ 量级。

4.16 正确结果先满足形状一致、无非有限数以及指定容差，再比较耗时。环境中的数值库、线程数、硬件、后台负载与计时噪声都可能改变速度。公平比较应使用相同数值输入与类型、同一预测规则和同分约定、相同计时范围；先预运行、重复测量，并排除读取和打印。若改了预处理或 k，就已在比较不同算法条件，需重新解释评价协议。

4.17 本章小结

NumPy 把数值内容、类型和形状放在同一数组结构中。逐元素运算描述位置上的变换，归约消去指定轴，广播在兼容轴间复用输入，矩阵乘法执行内侧下标上的乘积求和。理解每个轴的意义，是判断这些规则是否实现正确计算的前提。

向量化提高执行效率的同时，也可能建立很大的中间结果。可靠的数值程序还要管理共享内存、随机状态和浮点误差；完整实验则继续遵守训练统计量只从训练数据估计、验证负责选择、测试负责最终评价的流程。

5 pandas、数据探索与可视化

先修知识、学习目标与本章位置

本章使用第 2 章的容器、函数与条件，第 3 章的文件路径、CSV 与异常，以及第 4 章的数组、布尔筛选和轴。统计部分从均值与排序开始，补充方差、分位数和相关系数的含义，不要求已经学完概率统计。

学习后应能读取带字段名的表格，区分位置与标签，检查类型、缺失和重复；完成筛选、分组和受约束合并；用统计量与图形回答具体问题；形成说明数据范围、处理规则、证据与局限的数据探索报告。第 10 章会把这里的处理规则纳入训练流程，第 41 章再用于完整项目。

同一列里出现数字、空白和文字，两个文件中的记录顺序不一致，某个区域的平均费用明显较高，这些现象都发生在训练模型以前。若直接把表格转成数组，程序可能成功执行，却已经把缺失理解成零、把两条不同记录拼在一起，或者把重复记录计数两次。数据处理首先要明确每行代表什么、每列怎样测得，以及哪些记录可以互相配对。

pandas 是处理带标签表格的 Python 库。它适合读取、整理、汇总与检查结构化数据；第 4 章的 NumPy 更侧重规则形状上的数值运算。**探索性数据分析**（exploratory data analysis，EDA）指通过数据质量检查、描述统计和可视化认识数据，提出或修正问题。EDA 可以发现值得进一步检验的关系，但图形中的规律不自动成为因果解释或可靠预测结果。

本章使用人为构造的包裹记录。连续目标是费用，二元字段是是否延迟。数据故意包含空白、非法文本、一条完全重复记录与一条大重量记录。其规模足以手算，却不能代表真实配送业务。

5.1 从 CSV 到带标签的表格

安装、入口与数据字典

按第 2 章建立虚拟环境后，在终端安装本章依赖：

```
python3 -m pip install pandas matplotlib numpy
python3 -c "import pandas as pd; print(pd.__version__)"
```

pandas 负责表格，**Matplotlib** 负责绘图；惯用导入别名分别为`pd`与`plt`。别名不属于 Python 关键字。配套依赖文件记录本批实际运行版本，安装命令本身不保证以后安装的是相同版本。

第 3 章已经说明 **CSV**（comma-separated values，逗号分隔值）是一种文本表格格式。文件保存字段文本，不保存可靠的“这一列一定是整数”约束。首行一般是字段名，之后每行是记录；含逗号或换行的字段需要正确引用，不能用简单字符串切分替代 CSV 读取器。

数据字典说明字段的业务含义、单位、允许值和使用时点。先写字典，再判断类

型与处理规则。

字段	含义与单位	约束及用途
id	包裹标识，文本	非空、唯一；不是数值特征
split	预先固定的划分	train、val、test
area	区域代码，文本	A 或 B，合并区域说明
weight_kg	重量，kg	有限非负；允许缺失待处理
distance_km	距离，km	有限非负
late	是否延迟，0 或 1	分类任务的示例标签
fee_yuan	最终费用，元	连续目标；有限非负

本章不把late用于预测费用的输入：是否延迟往往要在配送后才知道。一个字段是否可用，取决于实际预测时点，不取决于它在 CSV 中是否已经存在。第 10 章会进一步讨论这种时间上的数据泄漏。

实验文件位于配套实验/ch05_data/。parcels.csv共 13 条原始记录，其中t03被完全重复一次；去重后训练 8 条、验证 2 条、测试 2 条。areas.csv为区域代码与区域名称的一一对应表。

```
from pathlib import Path
import numpy as np
import pandas as pd

root = Path(__file__).resolve().parent
raw = pd.read_csv(root / "ch05_data" / "parcels.csv",
                  dtype="string", keep_default_na=False)
print(raw.shape)
print(raw.head(3))
print(raw.dtypes)
```

本框在配套脚本中执行；交互环境没有__file__时，应显式设置实际的数据目录。head(3)只查看前三行，不证明其余行也合法。dtype="string"让各字段先保留文本含义，keep_default_na=False关闭默认缺失文本识别，空字段暂为""，随后按数据字典转换。否则文本“NA”可能被默认识别成缺失，而它也可能恰是一个合法区域代码。

Series、DataFrame 与索引

Series 是一维带标签的数据序列；**DataFrame** 是二维带行索引和列名的表格，各列可以有不同类型。索引（index）是用于标识轴位置的标签集合。默认行索引常为 0、1、2，但这不意味着索引永远等于当前位置，也不意味着索引自动具有唯一业务含义。

```
df = pd.DataFrame({"id": ["p1", "p2", "p3"],
                   "weight": [1.0, 2.0, 4.0],
                   "area": ["A", "B", "A"]},
                  index=[10, 20, 30])
print(df["weight"].shape)
print(df[["weight"]].shape)
print(df.columns.tolist())
```

单个列名得到Series，形状(3,)；列名列表得到DataFrame，形状(3, 1)。与第 4 章的一维数组、列数组区别相似，但表格还携带标签。读取shape只得到轴长，不能代替读取字段名称与顺序。

标识“0012”不能仅因含数字而转换成整数 12，否则会丢失前导零。区域代码 1、2、3 也不必是可计算距离的数值：代码之差不表示区域之间的物理距离。**数值变量、类别变量与标识字段**由含义区分，不能完全由文件中的字符或自动推断类型决定。

5.2 按标签、位置与条件选择记录

loc 与 iloc 回答不同问题

loc按标签选行列，iloc按从零开始的位置选行列。

```
print(df.loc[20, "weight"])
print(df.iloc[1, 1])
print(df.loc[10:20, ["id", "weight"]])
print(df.iloc[0:2, [0, 1]])
```

前两项都为 2.0，但依据不同。当前有序唯一整数索引下，loc[10:20]包含结束标签 20；iloc[0:2]遵守 Python 半开位置区间，不含位置 2。不要把标签切片的此例推广成任意无序或重复索引都能无歧义切片。

条件筛选先为每行生成一个布尔值，再取满足条件的记录。多个逐元素条件用&、|与~组合，并给每个比较加括号：

```
mask = (df["weight"] >= 2) & (df["area"] == "A")
selected = df.loc[mask, ["id", "weight"]].copy()
print(selected)
df.loc[df["id"] == "p2", "weight"] = 2.5
```

筛选得到p3与重量 4。最后一行在原表上明确赋值。and要判断整个对象的真假，不能替代 Series 逐行逻辑。连续写df[mask]["weight"] = 0会使赋值对象与原表的关系不清楚；本书统一用一次loc定位原表赋值。独立处理子表时用copy()表达意图，避免依赖不同 pandas 版本的复制细节。

标签对齐：有用的规则也可能掩盖错配

pandas 中的许多运算先按索引标签对齐，再逐位置计算；NumPy 主要按形状和位置计算。下面的两条 Series 顺序不同：

```
a = pd.Series([10, 20], index=["a", "b"])
b = pd.Series([1, 2], index=["b", "a"])
print(a + b)
print(a.to_numpy() + b.to_numpy())
```

Series 相加得到标签 a 为 12、b 为 21；去掉标签后的位置相加为 11、22。对齐并不是重新按数值排序，而是找相同标签。“a”处取 10 与 2，“b”处取 20 与 1。若标签只出现在一边，许多对齐运算会产生缺失值，不会自动认为缺失的一项是 0。

例题 5.1：排序后直接拼接标签

特征表按重量排序后，标识顺序为`p2,p1,p3`；标签表仍为`p1,p2,p3`，目标为 8、6、12。直接把目标数组按位置赋给特征表，会得到什么对应？如何正确配对？

解 错误对应为`p2:8,p1:6,p3:12`，前两条目标交换。正确对应应为`p2:6,p1:8,p3:12`。若双方仍保留可信的同一唯一索引，可以按标签对齐；若索引已经重置，应按显式`id`合并，并检查唯一性与未匹配记录。给两个表分别`reset_index(drop=True)`只能制造相同的位置编号，不能恢复原样本关系。

最终向 NumPy 转换时，要显式固定字段顺序并检查有限值：

```
feature_names = ["weight", "distance"]
example = pd.DataFrame({"distance": [10., 20.],
                        "weight": [1., 2.]})
X = example[feature_names].to_numpy(dtype=np.float64)
if X.ndim != 2 or not np.isfinite(X).all():
    raise ValueError("invalid feature matrix")
print(X)
```

结果为两行`[1,10]`、`[2,20]`。选择列表规定了特征顺序，而不是沿用原表偶然的列顺序。训练和预测使用相同名称、单位与顺序，才对应第 4 章的同一个数值输入空间。

5.3 类型转换与缺失值：保留失败原因

缺失、非法、零与无穷不同

缺失值表示本应存在的信息未获得或未记录；数值零表示确实测得零；非法文本表示字段不符合约定；正负无穷表示超出有限数范围或某些计算的极限结果。它们不能混为一种“空数据”。

pandas 可能用`np.nan`、`pd.NA`或日期缺失值表示不同类型的缺失。判断应使用`isna()`和`notna()`，不要用`x == np.nan`。采用可空整数类型`"Int64"`可以在整数列中保留缺失，和 NumPy 普通`int64`不同；本章重量、距离与费用最终采用浮点类型。

```
text = pd.Series(["1", "", "bad", "0"], dtype="string")
original_missing = text.eq("")
number = pd.to_numeric(text.mask(original_missing),
                       errors="coerce")
invalid = number.isna() & ~original_missing
print(original_missing.tolist())
print(invalid.tolist())
```

原缺失位置为第二项，转换失败位置为第三项。`mask`把条件成立的位置替换为缺失；`errors="coerce"`把不能转换的内容转成缺失。若只在转换后统计`isna()`，两种原因都会显示为缺失，无法判断是采集遗漏还是文件中写入了“bad”。完整脚本还用`np.isfinite`拒绝非有限文本数值，并在审计表中保存标识、字段、原文本与原因。

审计记录在这里指足以追踪数据变化的处理记录，不是复杂的管理系统。原文件保持不改，派生表记录转换结果，另一张表列出异常原因。这样在发现错误处理时，可以回到原文本重做，而不是继续在已丢失信息的表上猜测。

统计函数通常跳过缺失

```
s = pd.Series([1.0, np.nan, 3.0])
print(s.mean(), s.count(), s.size)
print(s.isna().mean())
print(s.sum(min_count=1))
```

均值 2.0 来自两个有效值，`count()`为 2，`size`为 3，缺失比例为 1/3。均值的分母不能默认为原始行数。全缺失序列的默认求和可能得到 0，`sum(min_count=1)`要求至少一个有效值，否则返回缺失；全缺失均值也不能作为有效训练参数。

删除所有带缺失的行会改变样本范围。若重量缺失主要发生在某类包裹，删除后剩余数据可能更少包含这一类。`dropna(subset=[...])`可以只要求当前分析所需字段有效；相关分析只删除两个参与字段不完整的行，不必因无关字段缺失而删除整行。

例题 5.2：均值填补并没有补回观测

一列观测为 1、缺失、3。用有效值均值 2 填补，比较填补前后的均值、样本方差和有效观测数量。

解 两个有效观测的均值为 2，样本方差为 $[(1-2)^2+(3-2)^2]/(2-1)=2$。填补后数值列为 1、2、3，均值仍为 2，若机械地按三条计算样本方差则为 1。填入的 2 由原数据计算而来，并非新增独立测量；有效原始观测仍只有 2 条。均值未变不说明分布、相关性或不确定性未变。

本章只演示一种透明规则：从训练集有效重量计算中位数，填补各集合，同时保留缺失指示列。是否适合实际问题要通过训练与验证流程判断。不能为使图形好看而提前把所有缺失改成零，也不能用全部集合的中位数再声称测试集完全隔离。

```
train = clean.loc[clean["split"] == "train"].copy()
median = train["weight_kg"].median()
if pd.isna(median):
    raise ValueError("no valid training weight")
prepared = clean.copy()
prepared["weight_missing"] = prepared["weight_kg"].isna().astype(int)
prepared["weight_kg"] = prepared["weight_kg"].fillna(median)
```

本框的clean来自实验 5.1 的load_tables()。缺失指示为 1 表示该数原来缺失或转换后无效；原因仍由审计表分别保存。第 10 章会讨论如何让这种“拟合参数，再复用参数”的过程嵌入模型评价。

5.4 重复记录与关键约束

完全重复和标识冲突

duplicated()检查参与比较的列值是否与已有记录重复；drop_duplicates()默认保留第一条。完全重复行可能来自重复导出，也可能恰好是合法的两次独立观测，去重前必须结合标识与采集含义。

本章数据字典规定一条id只对应一个包裹；完全相同的t03是重复导出，删除一条合理。若同一id出现费用 10 与 11，则是标识冲突。仅按id去重并留第一条会任意决定“真值”，应报告冲突并中止，等待核对来源。

```
duplicate_count = int(raw.duplicated().sum())
clean = raw.drop_duplicates().copy()
if clean["id"].eq("").any() or clean["id"].duplicated().any():
    raise ValueError("empty or conflicting id")
if not clean["split"].isin(["train", "val", "test"]).all():
    raise ValueError("unknown split")
```

检查放在数值转换前，便于辨认原文本冲突。本章先对字段去首尾空白；这是一项明确的规范化规则，并非任意文本都能随意删除空格。重复记录若分别进入训练与测试，会使评价样本不再独立于训练过程；不能等到训练后才检查这一问题。

从局部合法到整体一致

字段合法还不足以保证表格可信。应检查行数变化、必需字段、标识唯一性、允许类别、取值范围和样本配对；跨表还要检查关系是否满足一对一或多对一。每一步记录输入行数、输出行数和变化原因，才有机会发现无意丢行或扩行。

对于大型文件，全量读取可能超过内存。read_csv(chunksize=...)可以逐块读入；但“各块内无重复”不能推出全文件无重复，跨块标识需要另外维护。分组均值也不能直接平均各块均值，应同时累计总和与有效数量。本章使用小数据，目标是先把完整规则说明清楚。

5.5 分组、聚合与合并

分组是先拆开，再汇总

分组（grouping）按一个或多个字段把记录分成若干组，聚合（aggregation）把每组的多条记录归纳为一个或几个统计量。区域平均费用、每类标签数量、每日记录数都属于这类问题。

```
train = clean.loc[clean["split"] == "train"].copy()
grouped = train.groupby("area", dropna=False, sort=True).agg(
    rows=("id", "size"), weights=("weight_kg", "count"),
    mean_fee=("fee_yuan", "mean"), late_rate=("late", "mean"))
print(grouped)
```

`agg`采用命名聚合：输出列名后给出“输入列、汇总规则”。`size`数记录，`count`数该字段的非缺失值。`dropna=False`保留分组键缺失的组，避免无意忽略；完整实验仍通过数据字典约束区域。`sort=True`明确规定输出组顺序，不依赖默认显示顺序。

二元标签均值等于正类比例：若组内 m 条有效 0/1 标签中有 k 条为 1，则均值为 k/m。标签若包含 2 或非法值，这一解释立即失效。写出组内记录数与有效值数，有助于判断均值来自多少证据。

> **例题 5.3：组均值怎样合成总体均值**
>
> 区域 A 有 4 条费用，均值 11.5；区域 B 也有 4 条，均值 20.5。求整体均值。若 A 只剩 1 条且均值仍为 11.5，B 仍有 4 条，整体均值又是多少？

解 第一种为 $(4\times 11.5+4\times 20.5)/8=16$。第二种为 $(1\times 11.5+4\times 20.5)/5=18.7$，不是两组均值的简单平均 16。一般有

$$\bar{x}=\frac{\sum_g n_g\bar{x}_g}{\sum_g n_g},$$

其中 n_g 必须是该统计量实际使用的有效值数。若汇总的是重量，而某组有缺失，不能把总行数误作有效重量数。

按键合并，核对关系与未匹配项

键（key）是连接两张表时用于确定对应关系的字段。包裹表有多条 A 区域记录，区域表中 A 只应出现一次，关系为多对一（many-to-one）。左连接保留左表记录，并查找右表对应说明。

```
joined = clean.merge(lookup, on="area", how="left",
                     validate="many_to_one", indicator=True)
if not joined["_merge"].eq("both").all():
    raise ValueError("unmatched area")
joined = joined.drop(columns="_merge")
```

`validate`检查右表键唯一性；`indicator`增加`_merge`列，指出记录只在左侧、只在右侧或两侧均有。内连接`how="inner"`只保留两边都匹配的键，可能悄悄丢失未知

区域；因此本章先左连接，再显式拒绝未匹配项。

> **例题 5.4：重复键为什么使行数增加**
>
> 左表有两条键 A 的包裹，右表有两条键 A 的区域说明。对键 A 进行普通合并，会产生多少行？这会怎样影响统计？

解 左侧每条 A 都与右侧两条 A 配对，共 $2\times2=4$ 行。这是多对多连接的组合结果，不能理解为新增了两条独立包裹。若 A 被重复而 B 没有，A 在整体均值中的权重会无意增加。`validate="many_to_one"`会拒绝右侧重复键，避免把异常扩行继续送入训练。

合并前还应拒绝缺失键；pandas 的合并可能让两侧缺失键互相匹配，不能照搬其他数据库对空值的直觉。若时间变化的区域表每区有多个生效日期，就需要“区域加日期”等复合规则，不能简单去掉重复键来强行满足多对一。

上下追加同结构记录可以使用拼接函数：

```
combined = pd.concat([part1, part2], ignore_index=True)
```

其中两个输入是已读取的同结构表。它与按键补充字段的`merge`不同，拼接时仍需检查字段集合、类型、单位与跨文件重复标识。

5.6 描述统计：中心、离散与分位数

描述的对象与分母

描述统计归纳已观测数据的数量、中心位置、离散程度与分布形态。它首先回答“这批数据是什么样”，不直接回答更大总体是否也如此。`describe()`可以提供计数、均值、标准差、分位数和极值，但其输出仍需结合单位、有效数量与缺失规则解释。

对 m 个有效实数 $x_1,\dots,x_m$，均值为

$$\bar{x}=\frac{1}{m}\sum_{i=1}^{m}x_i.$$

均值与原变量单位相同，所有数值都参与计算，因此极端值会影响它。**中位数**（median）为排序后居中的值；有效数量为偶数时，本章取中间两值的平均。中位数更关注次序，不能反映尾部大数到底有多大。

描述离散程度可用**方差**与**标准差**。本章明确区分两种分母：

$$v_0=\frac{1}{m}\sum_{i}(x_i-\bar{x})^2,$$

$$s^2=\frac{1}{m-1}\sum_{i}(x_i-\bar{x})^2,\qquad m\ge 2.$$

v_0 描述这批数的平均平方离差；s^2 是常见的样本方差，作为总体方差估计的条件在第 8 章讨论。方差单位是原单位的平方，标准差为方差平方根，回到原单位。pandas 的`std()`通常默认`ddof=1`，NumPy 的`std()`默认`ddof=0`；同一列得到不同结果时，应先核对分母，不立即归因于浮点误差。

分位数需要约定插值方法

分位数（quantile）描述排序数据中某个比例位置上的值。不同有限样本定义可能给出不同结果，本章采用 pandas 的线性插值约定。对升序 $x_{(1)}, \dots, x_{(m)}$ 与比例 $p \in [0,1]$，先用零起点位置

$$h = (m-1)p, \qquad j = \lfloor h \rfloor, \qquad \alpha = h - j.$$

若 $j < m-1$，取 $q_p = (1-\alpha)x_{(j+1)} + \alpha x_{(j+2)}$；若 $j = m-1$，取最后一项。第一四分位数 $Q_1 = q_{0.25}$、中位数 $Q_2 = q_{0.5}$、第三四分位数 $Q_3 = q_{0.75}$。

> **例题 5.5：大重量记录怎样影响统计**
>
> 训练集中有效重量按升序为 1、2、3、5、6、20 kg。按上述约定计算均值、中位数、样本方差、四分位数和四分位距，并给出基于四分位距的异常标记。

解 总和 37，均值 $37/6 \approx 6.166667$ kg；中位数 $(3+5)/2 = 4$ kg。平方和 475，因此平方离差和为

$$\sum_i (x_i - \bar{x})^2 = 475 - 6(37/6)^2 = \frac{1481}{6}.$$

样本方差为 $1481/30 \approx 49.366667\ \text{kg}^2$，分母 6 的方差为 $1481/36 \approx 41.138889\ \text{kg}^2$。

四分位位置分别为 1.25 和 3.75，故 $Q_1 = 2 + 0.25(3-2) = 2.25$ kg，$Q_3 = 5 + 0.75(6-5) = 5.75$ kg。**四分位距**（interquartile range，IQR）为 $Q_3 - Q_1 = 3.5$ kg。常见探索规则将

$$[Q_1 - 1.5\,\text{IQR},\ Q_3 + 1.5\,\text{IQR}] = [-3, 11]\ \text{kg}$$

作为围栏，20 kg 超出上围栏，被标记为待检查记录。下围栏为负数不允许真实重量为负，它只是统计规则计算出的界限；物理合法性仍按数据字典检查。

异常值（outlier）指相对某种参考分布或规则显得特殊的记录，不一定是错误值。20 kg 可能是合法重包裹，不能仅凭围栏就删除；若真实任务要处理重包裹，删掉它反而会掩盖模型最需要面对的情况。本章保留`t08`，在报告中单独说明。

```
weight = train["weight_kg"].dropna()
q1, median, q3 = weight.quantile([.25, .5, .75]).tolist()
iqr = q3 - q1
flag = (train["weight_kg"] < q1 - 1.5 * iqr) | (
        train["weight_kg"] > q3 + 1.5 * iqr)
print(train.loc[flag, ["id", "weight_kg"]])
print(weight.mean(), weight.std(ddof=1))
```

缺失值不参与围栏估计，也不会因“没有被标记”就成为正常观测。报告中要同时列有效数量、缺失数量和被标记数量。IQR 为 0 时围栏可能退化，离散或极小样本不能机械套用连续大样本的解释。

5.7 相关性、标签分布与比较边界

相关系数测量线性共同变化

对同一批 m 个完整配对 (x_i, y_i)，**Pearson 相关系数**定义为

$$r = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_i (x_i - \bar{x})^2}\sqrt{\sum_i (y_i - \bar{y})^2}}.$$

本式要求两列均不是常数，并按相同样本配对。分子为同向偏离的乘积总和，分母把两列离差尺度消去，使 r 无量纲。两列在均值同侧时乘积为正，反侧时为负。第 6 章会把它解释为两个中心化向量夹角的余弦，从而说明 $-1 \le r \le 1$。

训练集中重量有效的 6 条记录费用为 6、8、10、14、16、44，满足 $y = 2x + 4$。故 $y_i - \bar{y} = 2(x_i - \bar{x})$，代入分子分母得到 $r = 1$。这个结果反映教学构造中的精确线性关系，不反映真实业务必然达到的预测精度。

```
pairs = train[["weight_kg", "fee_yuan"]].dropna()
r = pairs["weight_kg"].corr(pairs["fee_yuan"])
print(len(pairs), r)
print(train["late"].value_counts(dropna=False))
print(train["late"].value_counts(normalize=True, dropna=False))
```

输出有效配对数 6 与接近 1 的相关系数。二元标签 0、1 各 4 条，比例各 0.5。`normalize=True`把频数除以相应总数；若允许缺失标签，应明确分母是否包含缺失，不能把“已标注样本中的比例”称为全部样本比例。

> **例题 5.6：零相关不等于没有关系**
>
> 令 $x = (-2, -1, 0, 1, 2)$，$y = (4, 1, 0, 1, 4)$。计算相关系数的分子，并说明散点图能补充什么信息。

解 $\bar{x} = 0$、$\bar{y} = 2$，乘积和为 $(-2)2 + (-1)(-1) + 0(-2) + 1(-1) + 2 \cdot 2 = 0$。两列方差非零，因此 $r = 0$；但每个点严格满足 $y = x^2$。散点图能显示 U 形关系，线性相关系数无法概括这一非线性结构。

相关还可能来自共同原因、选择过程、分组构成或少数极端值。区域 A/B 的平均费用不同，但两组重量构成也不同；只有 8 条构造数据，不能把区域差解释为区域造成的费用差。可以提出进一步核对重量、距离和业务规则的需求，不能从这张表直接给出因果结论。

分类任务需要检查各类数量与比例。若 1000 条标签中 990 条为 0，始终预测 0 也有 99% 准确率；“高准确率”可能只来自类别构成。标签缺失、标注规则变化与重复样本都可能使分布失真。第 14 章会进一步讨论适合不平衡任务的评价指标。

5.8 Matplotlib：图形对象与四种基本图

Figure 和 Axes 各负责什么

`Figure`是整张画布，`Axes`是画布中带坐标轴的一块绘图区。`plt.subplots()`同时创建它们。一个 Figure 可以包含多个 Axes，多个图应分别添加轴标、单位和标题。

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(5, 3.5), layout="constrained")
ax.scatter([1, 2, 3], [6, 8, 10], color="#35566E")
ax.set(xlabel="Weight (kg)", ylabel="Fee (yuan)",
       title="Three teaching observations")
fig.savefig("scatter.png", dpi=180)
plt.close(fig)
```

`figsize`单位为英寸，`dpi`为每英寸像素数；增大 DPI 提高栅格输出像素数量，不增加数据证据。`layout="constrained"`为轴标与标题留出空间；仍应打开输出检查，不能把自动布局当作不裁切的保证。`close`释放画布，避免批量绘图时不断积累。

本章配套图使用英文轴标以减少不同系统中文字体差异，含义与单位在正文说明；主讲义中的示意图仍用既有宋体和主题色。需要中文图时应设置可用中文字体并核对缺字，不能仅改变字符串而忽略渲染警告。

散点图、直方图、柱形图与曲线

散点图（scatter plot）把每个配对样本画成一个点，适合查看两列关系、聚集、非线性与特殊点。它不连接样本，不暗示时间连续性。点重叠时可调整透明度，但不能因为看见一个点就断言只有一条记录。

直方图（histogram）把数值轴划分成区间，统计落入各区间的数量。分箱边界会影响外观；应记录边界，不能通过反复调箱宽只展示最符合预期的形状。训练重量按 0、5、10、15、20、25 kg 分箱，半开区间依次为 $[0,5)$、$[5,10)$、$[10,15)$、$[15,20)$，最后一箱 $[20,25]$ 包含右端点，频数为 3、2、0、0、1。

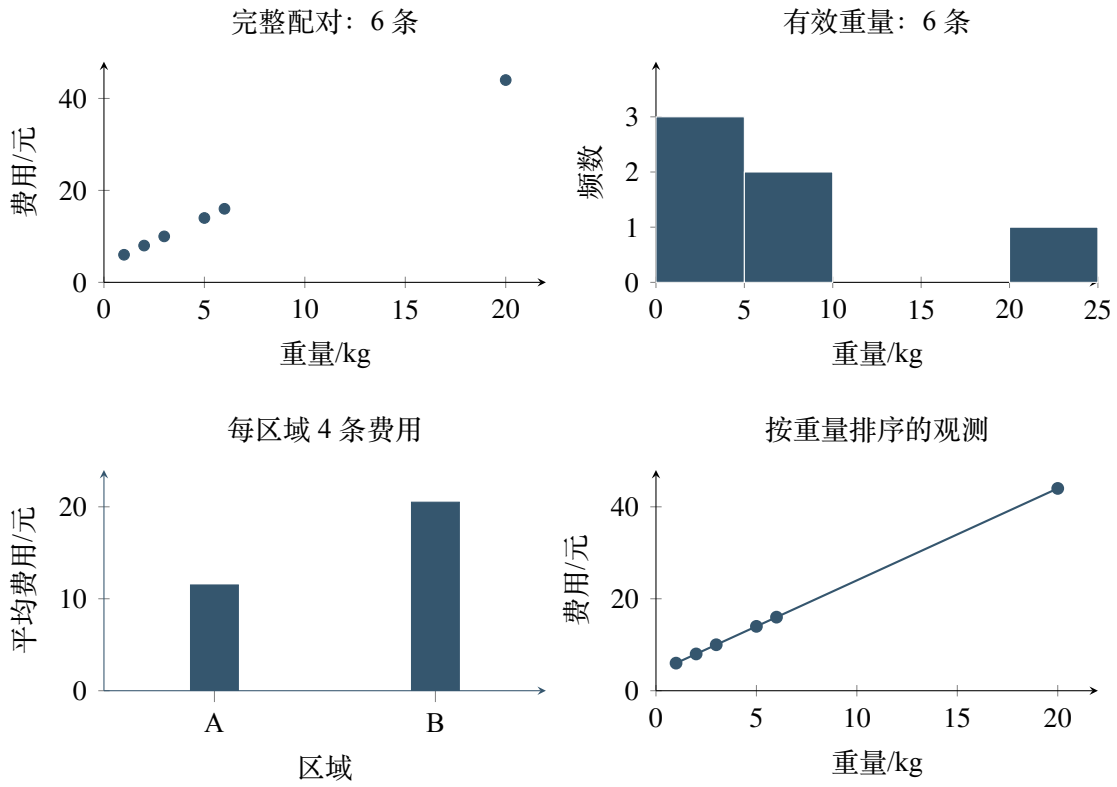
柱形图（bar chart）比较离散类别或已汇总统计量，横轴 A/B 不是连续数值区间。费用均值柱形图应同时报告各组数量。本章两组各 4 条，均值分别 11.5 与 20.5 元；它只描述这 8 条数据。

曲线图（line plot）按给定顺序连接点，适合显示时间或有意义的有序变化。`plot`不会自动替数据排序。按重量排序后连接费用点，可以辅助观察随重量的变化，但不能把横轴当作时间，也不能把连线当作已拟合模型。

```
fig, axes = plt.subplots(2, 2, figsize=(9, 6.5),
                         layout="constrained")
axes[0, 0].scatter(pairs["weight_kg"], pairs["fee_yuan"])
axes[0, 1].hist(weight, bins=[0, 5, 10, 15, 20, 25])
axes[1, 0].bar(grouped.index, grouped["mean_fee"])
ordered = pairs.sort_values("weight_kg")
axes[1, 1].plot(ordered["weight_kg"], ordered["fee_yuan"], "o-")
plt.close(fig)
```

此框只展示各图调用，对外保存前还要像完整脚本一样设置全部轴标与标题。`axes`

形状为`(2, 2)`，位置由两维索引指定；只有一幅图时返回的对象结构可能不同，不能照抄二维下标。



四图仅使用训练集。散点和曲线含 6 个完整配对，直方图使用 6 个有效重量，区域柱图使用 8 条有效费用。曲线的连线按重量排序，横轴不是时间；它也不表示已经训练出一个可外推模型。两条重量缺失记录仍计入区域费用统计，所以各图样本数不能含混地统一写成 8。

频数、密度与可比坐标

直方图的`density=True`把柱高换成概率密度估计，使柱形面积之和为 1，而非柱高之和为 1。若第 j 箱宽 h_j、频数 c_j，共有 m 条纳入分箱的记录，则密度高为 $c_j/(mh_j)$。单位为横轴单位的倒数。

例题 5.7：密度图为什么不是概率柱高

6 个重量按宽 5 kg 的箱分组，频数 3、2、0、0、1。求第一箱和第二箱的密度高，以及第一箱的经验比例。

解 密度高分别为 $3/(6\times5)=1/10\ \text{kg}^{-1}$ 与 $2/(6\times5)=1/15\ \text{kg}^{-1}$。第一箱面积为 $(1/10)\times5=1/2$，等于该箱的样本比例。箱宽改变时高也会变，不能把密度高理解为箱内比例，第二箱并非占 10%。若指定箱边界没有覆盖全部数据，还要说明分母实际涵盖哪些记录。

多图比较应尽量用相同分箱与坐标范围，图上标明样本数。柱形图一般从零起画，否则会夸大比例差异；散点图局部放大可以使用不同范围，但必须说明。对数坐标能

展开数量级差异，却要求正数或另作变换，并改变距离的视觉含义。本章不以删除尾部、截轴或隐藏缺失来改善图形外观。

5.9 实验 5.1：表格清理与可核对汇总

入口为配套实验`/ch05_tables.py`，使用 NumPy 和 pandas，CPU 可运行。程序从脚本位置读取两个 CSV，按第 3 章的入口保护运行；普通导入不会启动实验或写结果。

```
python3 ch05_tables.py
```

流程按如下顺序执行：核对字段与非空输入；保留文本并去首尾空白；删除完全重复行并拒绝标识冲突；转换数值并记录原缺失/转换失败；检查范围与标签；受约束地合并区域说明；仅取训练行作分组汇总。先审计再统计，避免把原始错误隐去。

完整训练表如下，横线表示数值缺失。

标识	区域	重量/kg	距离/km	延迟	费用/元
t01	A	1	10	0	6
t02	A	2	20	0	8
t03	B	3	10	1	10
t04	B	—	20	1	12
t05	A	5	10	0	14
t06	B	6	20	0	16
t07	A	—	10	1	18
t08	B	20	20	1	44

验证标识`v01,v02`的重量为 7、8，费用 18、20；测试标识`s01,s02`的重量为 9、10，费用 22、24。它们只参与固定字段合法性检查；本章不使用其目标值选择清理、填补或删点规则。

关键输出应满足：原始 13 行、唯一 12 行、完全重复 1 行；训练/验证/测试数量 8/2/2；审计表含`t04:missing`与`t07:invalid`；A/B 各 4 条，各有 3 个有效重量，费用均值 11.5/20.5，延迟比例 0.25/0.75。标签对齐演示为`a:12,b:21`。

修改任务每次从原版独立开始：把重复`t03`的费用改 11，应触发冲突错误；把区域表的 B 重复一次，应触发合并关系错误；把`s01`区域改 C，应触发未匹配错误；把延迟值改 2，应触发标签范围错误。不要捕获异常后继续打印旧报告。配套答案在临时目录做冲突核对，不修改原始数据。

5.10 实验 5.2：统计、图形与数据探索报告

问题与输出

入口为配套实验`/ch05_eda.py`，在实验 5.1 基础上增加 Matplotlib。它选择无需窗口的`Agg`后端并保存图片，适合在终端或自动运行环境使用。

```
python3 ch05_eda.py
python3 ch05_solutions.py
```

脚本在`ch05_results/`生成审计表、分组表、填补演示表、`summary.json`、`overview.png`与`report.md`。摘要保存依赖版本与输入文件 SHA-256，便于辨认所分析的文件字节；摘要不能证明数据真实或质量充分。

本实验回答两个预先提出的问题：有效重量与费用如何共同变化？区域平均费用差是否足以解释为区域作用？统计先使用未填补的训练表，报告原始缺失情况；填补演示另存派生表，避免把合成数值冒充观测。

应得到训练重量有效 6 条、均值 37/6、中位数 4、四分位数 2.25 与 5.75、围栏 $[-3, 11]$；只标记`t08`。重量费用完整配对 6 条，相关系数在浮点容差内为 1。直方频数 3、2、0、0、1；延迟标签各 4 条。离散计数与标识应完全一致，浮点统计允许约 10^{-9} 绝对差异。

报告从问题走向证据与局限

一份数据探索报告至少说明：问题与分析单位、数据来源与范围、质量问题及处理、统计与图形证据、能够支持的结论、不能支持的结论和下一步核对需求。仅贴`describe()`和几张图，读者无法判断样本为何变少，也无法知道结论基于原始值还是填补值。

> **报告示例：本教学数据能够支持什么**
>
> 原始表 13 行，去除 1 条完全重复后 12 行，其中训练 8 行。训练重量有 6 个有效值；`t04`原缺失、`t07`为非法文本，分别保留审计原因。有效重量均值 6.166667 kg、中位数 4 kg；20 kg 超出 IQR 围栏，但无录入错误证据，保留并注明。
>
> 6 个完整配对呈精确正线性关系，这是教学构造的性质。区域 A/B 费用均值为 11.5/20.5 元，每组 4 条；两组重量组成不同，不能把均值差解释为区域因果效应。训练标签两类各 4 条，不能据此推断真实业务也平衡。
>
> 填补演示只用训练中位数 4 kg，并保留缺失指示。数据少、缺失原因未知、尾部仅 1 条，不能据此证明预测泛化、识别缺失机制或估计真实业务风险。后续需要核对采集流程并扩充数据。

报告中“相关系数为 1”要同时列配对数；“有异常”要指出采用何规则；“已清理”要列出删改数量；“区域差异”要指出可能混杂因素。解释的力度应与证据相称，不把描述性结果自动写成模型选择依据。

修改任务：只改变测试费用为 999，训练统计与四幅训练图应不变；把`t08`重量由

20 改 8，重新计算分位数、围栏和相关系数，费用 44 仍保留，原线性关系不再成立；在原版上只比较`ddof=0`与 1，说明有效分母为何分别为 6 与 5；另画相同数据的两种分箱，记录外观变化但不改变原始记录。

5.11 流程成本、常见误区与知识联系

对 n 行、d 字段的表格，读取与逐字段检查至少要接触 $O(nd)$ 个单元。均值和计数可单遍累计；排序型分位数的直接做法需要 $O(n \log n)$ 排序。分组与合并常用哈希或排序，实际成本取决于实现、键类型与输出规模；多对多连接的输出本身可能接近乘积规模。不能只因一行`merge`就认为成本恒定。

EDA 容易出错的地方包括：把自动类型推断当作数据字典；把零当缺失；把转换失败与原缺失混在一起；把`count`当总行数；先分别排序再按位置配对；按键去重时隐去冲突；让重复键扩行；看全部测试标签后反复调处理规则；从少量散点推断因果关系。

本章把第 3 章的文件读写与审计衔接起来，把第 4 章的布尔筛选扩展到带标签表格。相关系数中的离差乘积将在第 6 章变成内积；方差、协方差与抽样波动将在第 8 章系统解释。第 10 章会区分一次固定的数据合法性核对和从数据估计的预处理参数，第 14 章会解释为何频繁依据测试结果作选择会损害最终评价。

自检时应能说明：一行数据对应什么实体；标签与位置何时不同；每个统计量有多少有效记录；每次删改为何合理；合并行数为何变化；图的横纵轴、单位、分箱与样本范围是什么；哪些结论只是对本批样本的描述。

5.12 分层习题

以下编号为 5.1–5.12；统计题使用本章线性分位数约定，实验题从原版独立修改。

5.1 **概念**。区分 CSV、Series、DataFrame、行索引与业务标识。为什么“0012”不宜自动转成整数？

5.2 **索引**。给定索引 10、20、30，比较`loc[10:20]`与`iloc[0:2]`。两条索引分别为 a/b 与 b/a 的 Series 相加为何不等于转数组后相加？

5.3 **筛选**。写出选择区域 A 且重量至少 2 kg 的代码；明确是在原表赋值还是独立子表处理，解释为何不用 Python 的`and`。

5.4 **缺失**。文本列为“1”、空白、“bad”、“0”。分别标出原缺失、非法文本与有效零，写出保留失败原因的转换顺序。

5.5 **统计**。对 1、2、3、5、6、20 计算均值、中位数、两种分母的方差及四分位数。删除 20 后重算均值与中位数，说明信息变化。

5.6 **分组**。区域 A 有 4 行，其中 3 个有效重量；B 也如此。`size`与重量`count`分别是多少？若 A/B 费用均值 11.5/20.5，组数变为 1/4，总均值是多少？

5.7 **合并**。左表键 A 出现 2 次、B 出现 1 次，右表 A 出现 2 次、B 出现 1 次。普通内连接有多少行？若右表漏 B，左连接与内连接分别留下什么？

5.8 **相关**。完成例题 5.6，解释为何零相关不能推出独立。常数列的相关系数为什么不能按普通公式计算?

5.9 **图形**。6 条重量的分箱频数为 3、2、0、0、1，箱宽 5 kg。写出各箱密度与面积；解释柱形图和直方图的区别。

5.10 **边界**。为什么不能直接删除全部缺失行或所有超出 IQR 围栏的行？列出还需核对的三类信息。

5.11 **编程**。运行两个实验及答案脚本；改变测试标签，核对训练汇总不变；构造右表重复键与冲突标识，确认主动失败。

5.12 **综合报告**。用本章数据写一份含问题、范围、质量、两项证据、结论与局限的报告。不得把相关性或区域均值差改写成因果结论。

5.13 习题参考答案

5.1–5.4：结构、选择与转换

5.1 CSV 是文本文件格式；Series 是一维带标签序列；DataFrame 是带行列标签的二维表；行索引用于定位，不自动等于包裹身份；业务标识由数据字典规定唯一性。“0012”作为标识时前导零属于信息，转成 12 会改变表示，并可能与另一条“12”发生冲突。

5.2 在给定有序唯一索引下，二者都取前两行，但前者按标签且包含 20，后者按位置且不含位置 2。设数值分别 10/20 与 1/2，按标签得到 a 为 12、b 为 21；去标签按位置为 11、22。排序或重置索引后应重新核对对应依据。

5.3 先分别比较，再逐行取交集；独立子表用复制明确用途。

```
mask = (df["area"] == "A") & (df["weight"] >= 2)
selected = df.loc[mask, ["id", "weight"]].copy()
# 若目的改为修改原表，则用下面的明确定位赋值：
df.loc[mask, "weight"] = 0
```

Series 真假不是一个整体标量，逐行组合需要&。

5.4 先保留文本，再规范化本例首尾空白，记录`text.eq("")`；随后按规则转换，原非空但转换失败的是非法文本。零为有效数值，不是缺失。对文本无穷还应检查有限性。转换后同时保存原文本与原因，避免用一个 NaN 丢掉来源。

5.5–5.8：统计、合并与相关

5.5 均值 37/6，中位数 4，平方离差和 1481/6，方差分母 6 时 1481/36，分母 5 时 1481/30。$Q_1 = 2.25$、$Q_3 = 5.75$。删除 20 后剩 1、2、3、5、6，均值 17/5 = 3.4，中位数 3。变化说明尾部影响很大，却不能证明尾部本应删除；样本范围已改变。

5.6 每组`size`=4、重量`count`=3。费用都有效时，组数 1/4 对应总均值 $(11.5 + 4 \times 20.5)/5 = 18.7$；重量均值合成时要用重量有效数量。不能把不同字段的有效分母混用。

5.7 A 贡献 $2 \times 2 = 4$ 行，B 贡献 1 行，总 5 行。漏 B 时，左连接仍保留左表的 B

行，右侧说明缺失；内连接只剩 4 条 A 配对。约束多对一会拒绝 A 重复；未匹配 B 另由`indicator`检查，两个检查职责不同。

5.8 离差乘积和 0，两列离差平方和均非零，故 $r = 0$；但 $y = x^2$ 由 x 确定，存在明确非线性关系。独立是概率分布关系，在第 8 章定义，不能由一个线性统计量为零推出。常数列离差范数为 0，分母为 0，相关未定义；不应自动填成 0。

5.9–5.12：图形、程序与解释

5.9 密度依次为 $1/10, 1/15, 0, 0, 1/30\ \mathrm{kg}^{-1}$，面积依次为 $1/2, 1/3, 0, 0, 1/6$，和为 1。直方图的横轴是数值区间，柱宽具有含义；柱形图横轴是离散类别，通常通过柱高比较某项汇总。

5.10 需核对缺失发生的采集过程与群体差异，极端记录的原始凭据、单位与业务合理性，以及处理后样本范围是否仍符合预测目标。缺失可能不是随机发生，合法尾部也可能正是任务重点。处理规则应可追溯，并在后续训练流程中验证。

5.11 参考`ch05_solutions.py`。先从`load_tables()`取得清理表，再复制表并只改`split=="test"`行的费用；两次训练分组结果使用 pandas 的表格相等核对函数比较。重复键应为`MergeError`，冲突标识应为`ValueError`，不能只比较最终均值是否碰巧相同。

5.12 参考实验生成的`report.md`。至少列出 13 到 12 行的去重变化、训练 8 行与重量有效 6 条、缺失两种原因；证据可选相关系数 1 及 6 个配对、A/B 均值 11.5/20.5 且各 4 条。结论限于构造训练样本；局限包含小样本、缺失原因未知与尾部不足，不以测试标签调探索规则。

5.14 本章小结

表格处理同时管理数值、类型、标签与记录关系。合理流程先明确数据字典，再保留原始信息、核对字段、追踪缺失与重复、约束合并关系，最后进行统计与图形解释。每个统计量都应能回到对应记录、有效分母与处理规则。

探索报告把问题、证据和解释连接起来。描述性关系可以指导后续调查与建模，但因果判断、泛化评价和预处理参数选择需要各自的条件。图形清晰与代码运行成功，都不能替代这些条件。

6 线性代数复习

先修知识、学习目标与本章位置

本章复习已学过的线性代数，使用第 4 章的数组、矩阵乘法和形状检查。推导只要求基本代数、平方和与一元导数；矩阵微分从分量展开说明，第 7 章再系统讨论梯度和链式法则。

学习后应能解释内积、范数、距离与余弦；把矩阵理解为线性变换；用秩、列空间和零空间判断解的存在与唯一性；从投影推导最小二乘；辨认特征值、正定性、SVD 和条件数；推导平方损失的梯度并核对维度。本章为第 11 章线性回归、第 21 章 PCA、第 23 章反向传播与第 34 章低秩适配准备共同语言。

第 4 章说明了`X @ w`怎样计算，第 5 章说明了怎样把字段转成数值列。现在需要解释：矩阵的列代表哪些可组合的方向？当目标不在这些方向张成的空间中，拟合实际上在做什么？两列非常接近时，为什么预测可能稳定，参数却剧烈变化？这些问题需要把代数式、几何图像与数值计算放在一起。

本章只讨论有限维实向量与实矩阵。复数情况下内积需要共轭转置，本章不沿用实数公式直接处理复数。除说明数组实现外，向量均约定为列向量；数学上的转置不等同于 NumPy 一维数组的`.T`。

6.1 向量、坐标与统一维度约定

向量不仅是一列数字

向量（vector）可以表示带方向的几何量，也可以表示多个特征、模型参数或一批预测。坐标依赖所选基与字段顺序。包裹的重量和距离构成 $\mathbf{x} = (x_1, x_2)^{\mathrm{T}} \in \mathbb{R}^2$；两个分量单位不同，不应仅因放在同一向量中就认为其大小可直接比较。

$\mathbb{R}^d$ 表示由 d 个实数坐标组成的向量空间，d 为该空间的维数。若有 n 个样本、每样本 d 特征，则约定

$$X = \begin{pmatrix} \mathbf{x}_1^{\mathrm{T}} \\ \vdots \\ \mathbf{x}_n^{\mathrm{T}} \end{pmatrix} \in \mathbb{R}^{n \times d}, \qquad \mathbf{w} \in \mathbb{R}^d, \quad \mathbf{y} \in \mathbb{R}^n.$$

X 的行是样本，列是特征；$\mathbf{w}$ 为每个特征的系数；$X\mathbf{w}$ 为每个样本的线性得分或预测。本章用大写字母表示矩阵，粗体小写表示向量，普通小写表示标量。

线性组合是若干向量按系数缩放后求和，例如 $a_1\mathbf{v}_1 + a_2\mathbf{v}_2$。系数可正可负，可为零。张成空间（span）是所有这类组合组成的集合；它包含零向量，并对加法与数乘封闭。

基与坐标变化

线性无关表示只有所有系数都为 0 时，线性组合才等于零；若存在非全零系数组合为零，则线性相关。一组能张成空间的线性无关向量称为**基**（basis）。基的数量等于空间维数。

例如 $\mathbf{e}_1 = (1,0)^{\mathrm{T}}$、$\mathbf{e}_2 = (0,1)^{\mathrm{T}}$ 是 $\mathbb{R}^2$ 标准基。向量 $(3,2)^{\mathrm{T}} = 3\mathbf{e}_1 + 2\mathbf{e}_2$。换用 $\mathbf{b}_1 = (1,1)^{\mathrm{T}}$、$\mathbf{b}_2 = (1,-1)^{\mathrm{T}}$，同一向量为 $(5/2)\mathbf{b}_1 + (1/2)\mathbf{b}_2$。物体未变，坐标变了；不能拿不同基下的两组坐标直接相减。

第 5 章的列顺序是坐标约定的一部分。若训练时 X 列为重量、距离，预测时换成距离、重量，仍然可能得到同样形状的乘积，却已经把系数配给了错误特征。维度检查是必要条件，字段语义检查同样必要。

6.2 内积、范数、距离与余弦相似度

内积描述同向程度

两个 d 维实向量的**内积**（inner product，点积）定义为

$$\langle \mathbf{a}, \mathbf{b} \rangle = \mathbf{a}^{\mathrm{T}}\mathbf{b} = \sum_{j=1}^{d} a_j b_j \in \mathbb{R}.$$

内积对两个输入分别线性，交换输入不改变结果。对非零向量，它还满足几何关系

$$\mathbf{a}^{\mathrm{T}}\mathbf{b} = \|\mathbf{a}\|_2 \|\mathbf{b}\|_2 \cos\theta,$$

其中 θ 为夹角。内积为 0 的向量称为**正交**（orthogonal）；零向量与所有向量内积都为 0，但零向量没有确定方向或夹角。

平方和为何能产生距离和角度约束？对 $\mathbf{b} \neq 0$，任意实数 t 都有 $\|\mathbf{a} - t\mathbf{b}\|_2^2 \geq 0$。取 $t = (\mathbf{a}^{\mathrm{T}}\mathbf{b})/\|\mathbf{b}\|_2^2$，展开得到

$$\|\mathbf{a}\|_2^2 - \frac{(\mathbf{a}^{\mathrm{T}}\mathbf{b})^2}{\|\mathbf{b}\|_2^2} \geq 0.$$

于是

$$|\mathbf{a}^{\mathrm{T}}\mathbf{b}| \leq \|\mathbf{a}\|_2 \|\mathbf{b}\|_2,$$

这就是实向量的 **Cauchy–Schwarz 不等式**。它保证余弦落在 $[-1,1]$ 内；等号对应向量共线，包括相反方向。

范数测量大小，距离测量差

范数（norm）是符合非负性、零向量判别、绝对齐次性与三角不等式的大小度量；范数为 0 当且仅当向量为零。常见的 L_1、L_2 与 L_∞ 范数为

$$\|\mathbf{x}\|_1 = \sum_j |x_j|, \qquad \|\mathbf{x}\|_2 = \sqrt{\sum_j x_j^2}, \qquad \|\mathbf{x}\|_\infty = \max_j |x_j|.$$

齐次性指 $\|c\mathbf{x}\| = |c|\|\mathbf{x}\|$；三角不等式指 $\|\mathbf{a}+\mathbf{b}\| \le \|\mathbf{a}\|+\|\mathbf{b}\|$。由范数得到距离 $d(\mathbf{a},\mathbf{b}) = \|\mathbf{a}-\mathbf{b}\|$。平方欧氏距离 $\|\mathbf{a}-\mathbf{b}\|_2^2$ 便于优化和排序，但一般不满足距离的三角不等式，例如 0、1、2 之间有 $4 > 1 + 1$。

余弦相似度（cosine similarity）在两向量都非零时定义为

$$\cos(\mathbf{a},\mathbf{b}) = \frac{\mathbf{a}^{\mathrm{T}}\mathbf{b}}{\|\mathbf{a}\|_2\|\mathbf{b}\|_2}.$$

它关注方向，正比例缩放不改变结果，但负比例会翻转方向。对零向量应约定拒绝、单独处理或按应用定义特殊结果；随意给分母加小量会改变数学定义，也可能改变近零向量之间的排序。

例题 6.1：三种比较标准

取 $\mathbf{a} = (1,2)^{\mathrm{T}}$、$\mathbf{b} = (2,0)^{\mathrm{T}}$。求内积、两向量的三种范数、欧氏距离和余弦。比较 $\mathbf{a}$ 与 $10\mathbf{a}$ 的方向和距离。

解 内积为 2。$\mathbf{a}$ 的范数依次为 3、$\sqrt{5}$、2；$\mathbf{b}$ 的范数均为 2。差向量 $(-1,2)^{\mathrm{T}}$ 的欧氏范数为 $\sqrt{5}$，余弦为 $2/(2\sqrt{5}) = 1/\sqrt{5}$。$\mathbf{a}$ 与 $10\mathbf{a}$ 余弦为 1，欧氏距离为 $9\sqrt{5}$；方向相同不代表大小接近。

单位化 $\tilde{\mathbf{a}} = \mathbf{a}/\|\mathbf{a}\|_2$ 后，对两个非零向量有

$$\|\tilde{\mathbf{a}} - \tilde{\mathbf{b}}\|_2^2 = 2 - 2\cos(\mathbf{a},\mathbf{b}).$$

这是展开平方、利用单位范数得到的关系。仅在同样单位化条件下，按余弦从大到小与按欧氏距离从小到大排序一致；原向量直接比较时不保证一致。

第 5 章 Pearson 相关系数就是两个中心化数据向量的余弦。先减各自均值，再单位化，能消去平移与正尺度变化。原始向量余弦不减均值，因此两者不能互换。特征尺度变化也会改变原始欧氏距离，这正是第 4 章标准化需要明确的几何影响。

6.3 矩阵与线性变换

按行看得分，按列看组合

矩阵 $A \in \mathbb{R}^{m\times d}$ 定义映射 $T(\mathbf{x}) = A\mathbf{x} \in \mathbb{R}^m$。**线性变换**满足

$$T(\alpha\mathbf{x} + \beta\mathbf{z}) = \alpha T(\mathbf{x}) + \beta T(\mathbf{z}).$$

因此必有 $T(\mathbf{0}) = \mathbf{0}$。$A\mathbf{x} + \mathbf{b}$ 若 $\mathbf{b} \neq 0$，称为**仿射变换**（affine transformation），包含平移，不严格属于线性变换。

记 A 的第 j 列为 $\mathbf{a}_j$，第 i 行为 $\mathbf{r}_i^{\mathrm{T}}$，同一乘积可解释为

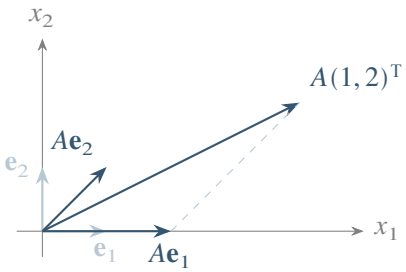
$$A\mathbf{x} = \sum_{j=1}^{d} x_j\mathbf{a}_j, \qquad (A\mathbf{x})_i = \mathbf{r}_i^{\mathrm{T}}\mathbf{x}.$$

前者按列解释输出由哪些方向组合，后者按行解释每个输出怎样求得分。对数据矩阵 X 与系数 $\mathbf{w}$，前者说明预测在特征列张成的空间中，后者说明每样本预测是特征与参数的内积。

例题 6.2：基向量决定变换

令 $A = \left(\begin{smallmatrix} 2 & 1 \\ 0 & 1 \end{smallmatrix}\right)$。求 $A\mathbf{e}_1$、$A\mathbf{e}_2$ 与 $A(1,2)^{\mathrm{T}}$，并解释矩阵两列的作用。

解 $A\mathbf{e}_1 = (2,0)^{\mathrm{T}}$、$A\mathbf{e}_2 = (1,1)^{\mathrm{T}}$，分别是两列。输出为 $1(2,0)^{\mathrm{T}} + 2(1,1)^{\mathrm{T}} = (4,2)^{\mathrm{T}}$。知道每个基向量变到哪里，就能由线性性确定任意输入的结果。



图中使用同一平面坐标展示输入基与变换后向量，虚线表示第二列方向的两倍。一般矩阵也可以在不同维度空间之间映射，不能都画成平面内拉伸。

矩阵乘法是变换的复合

若 $B : \mathbb{R}^d \to \mathbb{R}^k$，$A : \mathbb{R}^k \to \mathbb{R}^m$，则先 B 后 A 为 $AB : \mathbb{R}^d \to \mathbb{R}^m$。维度为 $(m \times k)(k \times d) = (m \times d)$。乘法通常不交换，因为顺序改变后变换不同，甚至无法相乘。

转置把行列交换，满足 $(AB)^{\mathrm{T}} = B^{\mathrm{T}}A^{\mathrm{T}}$。一个检验是考察元素：$(AB)_{ij} = \sum_k A_{ik}B_{kj}$；交换 i、j 后与右侧相符。**单位矩阵** I 保留向量不变；方阵 A 若有逆 A^{-1}，则 $A^{-1}A = AA^{-1} = I$，逆变换恢复原输入。长方形矩阵通常没有这种双侧逆。

6.4 秩、列空间、零空间与线性方程

秩表示独立方向的数量

A 的**列空间** $\mathcal{C}(A) = \{A\mathbf{x} : \mathbf{x} \in \mathbb{R}^d\}$ 是所有可能输出。**秩**（rank）为列空间维数，也等于行空间维数，满足 $\operatorname{rank}(A) \le \min(m, d)$。列数是存储的方向数量，秩是其中独立方向数量，两者可能不同。

零空间（null space）$\mathcal{N}(A) = \{\mathbf{x} : A\mathbf{x} = \mathbf{0}\}$ 表示在变换中消失的输入方向。若 $A\mathbf{x}_0 = \mathbf{b}$，那么任意 $\mathbf{z} \in \mathcal{N}(A)$ 都有 $A(\mathbf{x}_0 + \mathbf{z}) = \mathbf{b}$。因此

- $A\mathbf{x} = \mathbf{b}$ 可解，当且仅当 $\mathbf{b} \in \mathcal{C}(A)$；
- 在可解前提下，解唯一，当且仅当 $\mathcal{N}(A) = \{\mathbf{0}\}$，即满列秩；
- 输入维数满足 $d = \operatorname{rank}(A) + \dim \mathcal{N}(A)$。

最后一式称为秩与零度关系。直观上，输入空间被分成可辨认方向与被压掉方向；并不意味着任意给出的原始坐标列可以直接一一分开。

例题 6.3：重复特征造成不唯一

令

$$X = \begin{pmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 1 & 2 & 2 \end{pmatrix}.$$

求秩与一个非零零空间向量。若预测为截距加斜率乘 x，第二、三项系数分别能否唯一确定？

解 第一列常数方向与第二列不成比例，独立；第三列等于第二列，故秩为 2。$\mathbf{z} = (0, 1, -1)^{\mathrm{T}}$ 满足 $X\mathbf{z} = \mathbf{0}$。预测只依赖第一项系数和后两项系数之和。对任何 t，把后两项改成 $w_2 + t, w_3 - t$，预测均不变，参数不能唯一识别。

方阵满秩时可逆；有零特征值时不可逆。长方形满列秩 $m \geq d$ 保证不同参数产生不同输出，但任意目标未必在列空间。满行秩 $m \leq d$ 保证任意输出可达到，却通常存在多组输入。存在解、解唯一与拟合误差为零，是三个需要分别回答的问题。

数值秩还依赖容差

精确代数中的秩不需要阈值；浮点程序从数值上判断独立方向时，需要把很小的奇异值与误差尺度比较。`np.linalg.matrix_rank`采用数值容差；`lstsq`中的`rcond`也影响哪些方向视为可辨认。把 0 改成 10^{-16} 可能改变精确秩，却未必增加可可靠求解的信息。

因此报告数值秩时应同时记录形状、奇异值、尺度和容差规则。“程序报告秩为 2”不能直接推出原始测量系统严格只有两种独立变化。SVD 与阈值将在后文统一说明。

6.5 正交基与投影

投影到一条直线

给定非零方向 $\mathbf{a} \in \mathbb{R}^n$，寻找直线 $\{c\mathbf{a} : c \in \mathbb{R}\}$ 中离目标 $\mathbf{y}$ 最近的点。平方距离为

$$f(c) = \|\mathbf{y} - c\mathbf{a}\|_2^2 = \mathbf{y}^{\mathrm{T}}\mathbf{y} - 2c\mathbf{a}^{\mathrm{T}}\mathbf{y} + c^2\mathbf{a}^{\mathrm{T}}\mathbf{a}.$$

配方得

$$f(c) = \|\mathbf{a}\|_2^2 \left(c - \frac{\mathbf{a}^{\mathrm{T}}\mathbf{y}}{\|\mathbf{a}\|_2^2} \right)^2 + \|\mathbf{y}\|_2^2 - \frac{(\mathbf{a}^{\mathrm{T}}\mathbf{y})^2}{\|\mathbf{a}\|_2^2}.$$

第一项非负，最小时

$$c_* = \frac{\mathbf{a}^{\mathrm{T}}\mathbf{y}}{\mathbf{a}^{\mathrm{T}}\mathbf{a}}, \qquad \hat{\mathbf{y}} = \mathbf{a}c_*, \qquad \mathbf{a}^{\mathrm{T}}(\mathbf{y} - \hat{\mathbf{y}}) = 0.$$

最后一式说明误差垂直于允许移动的方向。若仍有该方向上的误差分量，就可以沿直线移动以进一步缩短距离。

投影到正交子空间

若 $Q \in \mathbb{R}^{n\times r}$ 的列正交且均为单位向量，则 $Q^{\mathrm{T}}Q = I_r$。这是**标准正交列**，Q 不一定是方阵。投影为

$$\hat{\mathbf{y}} = QQ^{\mathrm{T}}\mathbf{y}, \qquad P = QQ^{\mathrm{T}} \in \mathbb{R}^{n\times n}. \tag{6.1}$$

每项 $\mathbf{q}_j^{\mathrm{T}}\mathbf{y}$ 是沿第 j 个单位方向的坐标，再乘 $\mathbf{q}_j$ 并求和，得到子空间内部分。投影矩阵满足

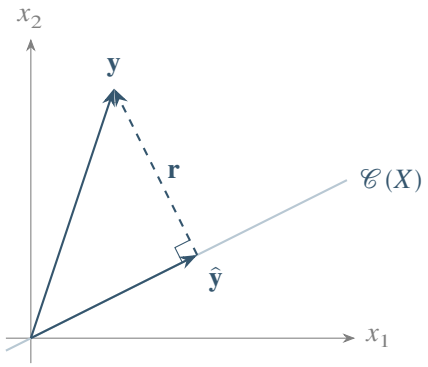
$$P^{\mathrm{T}} = P, \qquad P^2 = Q(Q^{\mathrm{T}}Q)Q^{\mathrm{T}} = P.$$

后者称为**幂等**：已在子空间中的投影再次投影不变。残差 $\mathbf{r} = (I - P)\mathbf{y}$ 满足 $Q^{\mathrm{T}}\mathbf{r} = 0$。若 $r < n$，QQ^{T} 是投影而不是 I_n；不能把 $Q^{\mathrm{T}}Q = I_r$ 颠倒为 $QQ^{\mathrm{T}} = I_n$。

对任意子空间内向量 $\mathbf{z}$，$\hat{\mathbf{y}} - \mathbf{z}$ 与残差正交，故

$$\|\mathbf{y} - \mathbf{z}\|_2^2 = \|\mathbf{r}\|_2^2 + \|\hat{\mathbf{y}} - \mathbf{z}\|_2^2.$$

这给出最短距离与投影唯一性的证明，不依赖图形维数是否能画出来。



图为二维几何示意：取目标 $(1,3)^{\mathrm{T}}$ 与沿 $(2,1)^{\mathrm{T}}$ 的子空间，投影为 $(2,1)^{\mathrm{T}}$，残差为 $(-1,2)^{\mathrm{T}}$。虚线误差与子空间正交；右角标记说明最短距离条件。实际拟合中目标在 n 维样本空间，几何条件仍相同。

例题 6.4：在三维空间投影到一条线

令 $\mathbf{q} = (1,1,0)^{\mathrm{T}}/\sqrt{2}$，$\mathbf{y} = (2,0,3)^{\mathrm{T}}$。求投影、残差、投影矩阵，并核对正交分解。

解 $\mathbf{q}^{\mathrm{T}}\mathbf{y} = \sqrt{2}$，投影为 $(1,1,0)^{\mathrm{T}}$，残差为 $(1,-1,3)^{\mathrm{T}}$。

$$P = \mathbf{q}\mathbf{q}^{\mathrm{T}} = \begin{pmatrix} 1/2 & 1/2 & 0 \\ 1/2 & 1/2 & 0 \\ 0 & 0 & 0 \end{pmatrix}.$$

$\mathbf{q}^{\mathrm{T}}\mathbf{r} = 0$，且 $\|\mathbf{y}\|_2^2 = 13 = 2 + 11 = \|\hat{\mathbf{y}}\|_2^2 + \|\mathbf{r}\|_2^2$。原始第三坐标 3 完全位于这条线的正交补中，不会被投影“解释”。

怎样获得正交列：QR 的意义

QR 分解把满列秩 $X \in \mathbb{R}^{n\times d}$（$n \geq d$）写成 $X = QR$，其中 $Q \in \mathbb{R}^{n\times d}$ 列标准正交，$R \in \mathbb{R}^{d\times d}$ 为可逆上三角矩阵。Q 描述相同列空间的正交坐标，R 描述从原系数到正交坐标的变换。

逐列减去前面方向上的投影再归一化，是构造正交基的一种直观方法，称为 Gram–Schmidt 正交化。近相关列可能使朴素实现出现明显舍入误差；实际库通常使用更稳定的变换实现 QR。这里理解分解的意义，数值求解采用库接口，不把短教学实现当作稳定性充分的工业实现。

6.6 最小二乘：从几何到正规方程

目标可能不在列空间

设 $X \in \mathbb{R}^{n\times d}$、$\mathbf{y} \in \mathbb{R}^n$，寻找 $\mathbf{w} \in \mathbb{R}^d$ 使预测 $X\mathbf{w}$ 接近目标。**最小二乘**（least squares）问题为

$$\min_{\mathbf{w}} \|X\mathbf{w} - \mathbf{y}\|_2^2. \tag{6.2}$$

“二乘”指误差平方；本章使用普通未加权欧氏距离，每样本贡献相同。若误差单位或可信度不同，需要另外说明权重或噪声假设，第 11 章再讨论回归建模。

所有预测都在 $\mathcal{C}(X)$ 中，因此最优预测是 $\mathbf{y}$ 在该子空间上的正交投影。设残差约定为 $\mathbf{r} = \mathbf{y} - X\mathbf{w}$，最优条件是残差与每一列正交：

$$X^{\mathrm{T}}\mathbf{r} = \mathbf{0} \quad \Longleftrightarrow \quad X^{\mathrm{T}}X\mathbf{w} = X^{\mathrm{T}}\mathbf{y}. \tag{6.3}$$

这称为**正规方程**（normal equations）；normal 在这里与几何垂直有关，不表示误差必须服从正态分布。要解释回归估计的统计性质，可以引入概率假设；定义和求解这个平方距离问题本身不需要正态假设。

为什么正交条件足够？若 $\mathbf{w}_*$ 满足它，对任意参数变化 $\mathbf{h}$ 有

$$\begin{aligned}\|\mathbf{y} - X(\mathbf{w}_* + \mathbf{h})\|_2^2 &= \|\mathbf{r}_* - X\mathbf{h}\|_2^2 \\ &= \|\mathbf{r}_*\|_2^2 - 2\mathbf{h}^{\mathrm{T}}X^{\mathrm{T}}\mathbf{r}_* + \|X\mathbf{h}\|_2^2 \\ &= \|\mathbf{r}_*\|_2^2 + \|X\mathbf{h}\|_2^2 \geq \|\mathbf{r}_*\|_2^2.\end{aligned}$$

因此所有满足正规方程的参数都是全局最优解。若 X 满列秩，非零 $\mathbf{h}$ 不会被 X 消去，解唯一；若秩亏，沿零空间方向移动不改变最优预测。

满列秩时的表达式与计算方法

对任意 $\mathbf{v}$，有 $\mathbf{v}^{\mathrm{T}}X^{\mathrm{T}}X\mathbf{v} = \|X\mathbf{v}\|_2^2$。满列秩时，非零向量对应严格正数，因此 $X^{\mathrm{T}}X$ 可逆。形式上

$$\mathbf{w}_* = (X^{\mathrm{T}}X)^{-1}X^{\mathrm{T}}\mathbf{y}, \quad P = X(X^{\mathrm{T}}X)^{-1}X^{\mathrm{T}}.$$

这是理论表达式，实际计算通常不显式求逆。即便使用正规方程，也应`solve(X.T @ X, X.T @ y)`求解线性系统；更一般地，优先`np.linalg.lstsq`，避免平方条件数带来的误差风险。

用 QR 可把目标分解为

$$\|QR\mathbf{w}-\mathbf{y}\|_2^2=\|R\mathbf{w}-Q^{\mathrm{T}}\mathbf{y}\|_2^2+\|(I-QQ^{\mathrm{T}})\mathbf{y}\|_2^2.$$

第二项不依赖参数，第一项可通过上三角方程 $R\mathbf{w}=Q^{\mathrm{T}}\mathbf{y}$ 置零。这样不必形成 $X^{\mathrm{T}}X$。对稠密 $n\ge d$ 输入，常见分解求解约需 $O(nd^2)$ 量级计算，具体成本随算法与矩阵结构变化。

例题 6.5：完整手算带截距的直线拟合

对输入 $x=0,1,2$ 与目标 $y=1,2,2$，拟合 $\hat{y}=b+ax$。写出设计矩阵，解正规方程，计算预测、残差与平方和，并核对正交性。

解 把参数写成 $\mathbf{w}=(b,a)^{\mathrm{T}}$，设计矩阵为

$$X=\begin{pmatrix}1&0\\1&1\\1&2\end{pmatrix},\qquad \mathbf{y}=\begin{pmatrix}1\\2\\2\end{pmatrix}.$$

第一列常数 1 负责截距。于是

$$X^{\mathrm{T}}X=\begin{pmatrix}3&3\\3&5\end{pmatrix},\qquad X^{\mathrm{T}}\mathbf{y}=\begin{pmatrix}5\\6\end{pmatrix}.$$

方程为 $3b+3a=5$、$3b+5a=6$，相减得 $a=1/2$，再得 $b=7/6$。预测与残差为

$$\hat{\mathbf{y}}=\begin{pmatrix}7/6\\5/3\\13/6\end{pmatrix},\qquad \mathbf{r}=\begin{pmatrix}-1/6\\1/3\\-1/6\end{pmatrix}.$$

残差和为 0，输入与残差的内积为 $0(-1/6)+1(1/3)+2(-1/6)=0$，故 $X^{\mathrm{T}}\mathbf{r}=0$。平方和为 $1/36+1/9+1/36=1/6$，MSE 为 $1/18$。残差不为零，因为这三个点不在一条直线上；参数唯一，因为两列线性无关。

“带截距时残差和为零”来自设计矩阵含常数列与未加权最小二乘最优条件。去掉截距、改变权重、加入参数约束或只做尚未收敛的迭代时，不能直接沿用该结论。

秩亏解与最小范数选择

把例题 6.5 的第二列复制一遍，预测只依赖 w_2+w_3。最优参数满足 $w_1=7/6$、$w_2+w_3=1/2$，有无穷多组。预测向量仍唯一，因为它是同一子空间中的正交投影。

如果额外要求参数欧氏范数最小，则后两项应均分为 $1/4,1/4$。证明是令 $w_2=t, w_3=1/2-t$，有

$$t^2+(1/2-t)^2=2(t-1/4)^2+1/8,$$

最小值在 $t=1/4$。这种选择给出了唯一代表，但没有证明两列特征具有独立可解释的作用。伪逆将在 SVD 之后给出一般表达。

6.7 特征值、二次型与正定性

特征方向保持方向不变

方阵 $A \in \mathbb{R}^{d\times d}$ 的**特征向量**（eigenvector）为满足

$$A\mathbf{v} = \lambda\mathbf{v}, \qquad \mathbf{v} \neq \mathbf{0}$$

的向量，λ 为对应**特征值**（eigenvalue）。在此方向，变换只作缩放或翻转。零向量被排除，否则任何数都能满足等式，定义失去辨别力。

一般实方阵可能有复特征值，也可能缺少足够多的线性无关特征向量；不能对任意方阵直接写正交特征分解。**实对称矩阵** $H = H^{\mathrm{T}}$ 具有实特征值和一组标准正交特征向量，能写成

$$H = Q\Lambda Q^{\mathrm{T}}, \quad Q^{\mathrm{T}}Q = QQ^{\mathrm{T}} = I, \quad \Lambda = \mathrm{diag}(\lambda_1, \dots, \lambda_d).$$

diag 表示把数放在主对角线上。本节只对这类对称矩阵使用正交谱分解。数值上可用 `np.linalg.eigh`或只求值的`eigvalsh`；它们利用对称结构，区别于一般`eig`。

二次型在各方向的符号

二次型 $q(\mathbf{x}) = \mathbf{x}^{\mathrm{T}}H\mathbf{x}$ 是标量。令 $\mathbf{z} = Q^{\mathrm{T}}\mathbf{x}$，则

$$q(\mathbf{x}) = \mathbf{z}^{\mathrm{T}}\Lambda\mathbf{z} = \sum_j \lambda_j z_j^2.$$

于是对实对称 H，有如下对应：

性质	对非零向量的条件	特征值条件
正定	$\mathbf{x}^{\mathrm{T}}H\mathbf{x} > 0$	所有 $\lambda_j > 0$
半正定	$\mathbf{x}^{\mathrm{T}}H\mathbf{x} \geq 0$	所有 $\lambda_j \geq 0$
不定	有正值方向也有负值方向	至少一个正值、一个负值

半正定允许某些非零方向的二次型为 0，因此不保证可逆。负定与半负定分别反转符号。只看矩阵元素是否为正不能判断正定；只看行列式为正也不充分。

例题 6.6：二次型的主方向

令 $H = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$。求特征值、单位特征向量与二次型表达，说明是否正定。再判断 $\left(\begin{smallmatrix} 1 & 2 \\ 2 & 1 \end{smallmatrix}\right)$。

解 第一矩阵作用于 $(1,1)^{\mathrm{T}}$ 得到 3 倍，作用于 $(1,-1)^{\mathrm{T}}$ 得到 1 倍；单位化后为 $\mathbf{q}_1 = (1,1)^{\mathrm{T}}/\sqrt{2}$ 与 $\mathbf{q}_2 = (1,-1)^{\mathrm{T}}/\sqrt{2}$。设 $z_1 = (x_1+x_2)/\sqrt{2}, z_2 = (x_1-x_2)/\sqrt{2}$，则

$$\mathbf{x}^{\mathrm{T}}H\mathbf{x} = 3z_1^2 + z_2^2 > 0 \quad (\mathbf{x} \neq 0).$$

第二矩阵沿同样两个方向的特征值为 3 和 -1，二次型为 $3z_1^2 - z_2^2$，是不定矩阵。其所有元素均为正，仍有负二次型方向。

$X^{\mathrm{T}}X$ 称为列向量的 **Gram 矩阵**，元素为列间内积。由平方范数恒非负可知它总半正定，满列秩时才正定。它在平方损失中描述参数变化引起的曲率；第 7 章会用 Hessian 把这一说法写成导数结论。

6.8 奇异值分解：长方形矩阵的方向与尺度

定义与形状

奇异值分解（singular value decomposition，SVD）适用于任意实矩阵 $A \in \mathbb{R}^{m\times d}$。完整形式为

$$A = U\Sigma V^{\mathrm{T}}, \quad U \in \mathbb{R}^{m\times m}, \quad V \in \mathbb{R}^{d\times d}, \quad \Sigma \in \mathbb{R}^{m\times d}.$$

U 和 V 为方形正交矩阵，Σ 只有主对角线可能非零，其 $k = \min(m, d)$ 个非负数 $\sigma_1 \geq \cdots \geq \sigma_k \geq 0$ 称为**奇异值**（singular values）。对长方形对角阵，主对角线之后的整行或整列为零。

NumPy 的`full_matrices=False`给出经济形状：

$$U_k : (m, k), \quad \Sigma_k : (k, k), \quad V_k^{\mathrm{T}} : (k, d).$$

它保留 k 个方向，可能仍包含零奇异值；不等同于只保留 r 个非零方向的秩分解，也不等同于只保留更小 q 的近似。NumPy 实际返回一维`s:(k,)`，不是对角矩阵。

```
A = np.array([[3., 0.], [0., 1.], [0., 0.]])
U, s, Vt = np.linalg.svd(A, full_matrices=False)
reconstructed = (U * s) @ Vt
print(U.shape, s.shape, Vt.shape)
print(s)
```

形状为`(3,2)`、`(2,)`、`(2,2)`，奇异值 3 与 1。`U * s`按列缩放 U，等价于 $U\,\mathrm{diag}(\mathbf{s})$；如果写`s[:, None] * U`，一般缩放方向错误或广播失败。`Vt`已经是 V^{T}，重建时不要再转置一次。

旋转坐标、缩放，再变到输出坐标

乘积 $A\mathbf{x} = U\Sigma V^{\mathrm{T}}\mathbf{x}$ 可分三步：V 转置把输入表示到右奇异向量坐标；Σ 沿这些方向缩放或消去；U 把结果表示到输出空间。U/V 中的正交变换保长度，可能包括反射，不能全称为严格的旋转。

对第 j 个右奇异向量 $\mathbf{v}_j$，有

$$A\mathbf{v}_j = \sigma_j \mathbf{u}_j, \qquad A^{\mathrm{T}}A\mathbf{v}_j = \sigma_j^2 \mathbf{v}_j.$$

因此奇异值是 Gram 矩阵非负特征值的平方根。最大奇异值描述单位输入的最大输出长度，零奇异值对应被消去的方向。非零奇异值数量是精确秩；对应左奇异向量张成列空间。

伪逆与最小范数最小二乘

设 r 个奇异值非零，秩分解为 $A = U_r \Sigma_r V_r^{\mathrm{T}}$。**Moore–Penrose 伪逆**为

$$A^{+} = V_r \Sigma_r^{-1} U_r^{\mathrm{T}} \in \mathbb{R}^{d \times m}.$$

它只对非零奇异值取倒数，零方向保持为零；不是对矩阵每个元素取倒数。若 $\mathbf{b} \in \mathbb{R}^m$，则

$$\mathbf{x}_* = A^{+}\mathbf{b}, \qquad A\mathbf{x}_* = U_r U_r^{\mathrm{T}}\mathbf{b}.$$

后者正是到列空间的投影。任意最小二乘解可写成 $\mathbf{x}_* + \mathbf{z}$，其中 $\mathbf{z} \in \mathcal{N}(A)$。$\mathbf{x}_*$ 在右奇异向量的非零方向中，与零空间正交，因此

$$\|\mathbf{x}_* + \mathbf{z}\|_2^2 = \|\mathbf{x}_*\|_2^2 + \|\mathbf{z}\|_2^2,$$

证明了它是最小范数解。

实际计算对很小的奇异值使用阈值，避免对近零值取极大倒数。`np.linalg.lstsq` 直接求最小范数最小二乘解；本章设置`rcond=None`，同时返回残差数组、数值秩与奇异值。返回残差数组为空可能是秩亏或形状条件不满足该返回形式，不能直接理解成拟合误差为零；应显式算`b - A @ x`。

低秩近似及误差含义

SVD 还可写为若干秩一外积之和：

$$A = \sum_{j=1}^{k} \sigma_j \mathbf{u}_j \mathbf{v}_j^{\mathrm{T}}.$$

内积 $\mathbf{u}^{\mathrm{T}}\mathbf{v}$ 是标量；外积 $\mathbf{u}\mathbf{v}^{\mathrm{T}}$ 是矩阵，两者不能混淆。保留前 q 项得到 A_q，秩至多 q。它在 **Frobenius 范数**下具有最优低秩近似性质：

$$\|A\|_F = \sqrt{\sum_{i,j} A_{ij}^2}, \qquad \|A - A_q\|_F^2 = \sum_{j=q+1}^{k} \sigma_j^2.$$

误差公式来自各外积在矩阵内积下标准正交。最优性质的完整证明不在本章要求内，但必须限定误差度量：它不保证任何下游分类指标或语义相似度最优。奇异值有并列时，最优近似表示可能不唯一。

> **例题 6.7：SVD、伪逆与截断**
> 对 $A = \left(\begin{smallmatrix} 3 & 0 \\ 0 & 1 \\ 0 & 0 \end{smallmatrix}\right)$，给出经济 SVD、伪逆、对 $\mathbf{b} = (3,2,4)^{\mathrm{T}}$ 的最小二乘解，以及秩一近似误差。

解 可取 U_k 为三维前两个标准基组成的矩阵、$\Sigma_k = \mathrm{diag}(3,1)$、$V = I_2$。伪逆为

$$A^{+} = \begin{pmatrix} 1/3 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix}, \quad \mathbf{x}_* = (1,2)^{\mathrm{T}}.$$

预测为 $(3,2,0)^{\mathrm{T}}$，第三项 4 不能由 A 的列表示，残差范数 4。秩一近似仅保留 3 对应项，得到第一元素 3、其余 0 的矩阵；Frobenius 误差为 1，平方误差为 1。这里矩阵近似误差与拟合 b 的误差是两个不同对象。

低秩近似为第 21 章 PCA 提供工具，第 34 章会用低秩矩阵表示参数更新；不能仅由“低秩”推出压缩后一定保留任务所需信息。实际库的奇异向量符号可能同时翻转，重建矩阵不变，核对时应比较重建、子空间与奇异值，不强制某一列符号固定。

6.9 条件数：问题的敏感性与算法的稳定性

微小输入变化可能引起很大解变化

对可逆方阵 A，采用欧氏范数对应的**谱范数** $\|A\|_2 = \max_{\|\mathbf{x}\|_2=1} \|A\mathbf{x}\|_2 = \sigma_{\max}$，**条件数**定义为

$$\kappa_2(A) = \|A\|_2\|A^{-1}\|_2 = \frac{\sigma_{\max}}{\sigma_{\min}} \geq 1.$$

条件数大表示某些方向被强烈压缩，反向求解时误差可能被放大。奇异矩阵没有有限的可逆求解条件数。满列秩长方形矩阵也常用最大与最小奇异值之比描述数值条件，但本节扰动上界首先针对可逆方阵。

若 A 固定，$A\mathbf{x} = \mathbf{b}$、$A(\mathbf{x} + \delta\mathbf{x}) = \mathbf{b} + \delta\mathbf{b}$，则 $\delta\mathbf{x} = A^{-1}\delta\mathbf{b}$。结合 $\|\mathbf{b}\| \leq \|A\|\|\mathbf{x}\|$，得到在 $\mathbf{b} \neq 0$ 时

$$\frac{\|\delta\mathbf{x}\|_2}{\|\mathbf{x}\|_2} \leq \kappa_2(A)\frac{\|\delta\mathbf{b}\|_2}{\|\mathbf{b}\|_2}.$$

这是最坏方向的上界，不是每次扰动必然放大这么多。若 A 本身也被扰动，还需要额外条件与误差项，不能直接把本式当作所有误差的完整界。

例题 6.8：残差很小，解仍敏感

令 $A = \mathrm{diag}(1, 10^{-8})$，$\mathbf{b} = (1,0)^{\mathrm{T}}$。把右端改为 $\mathbf{b} + \delta\mathbf{b} = (1, 10^{-8})^{\mathrm{T}}$，比较两次解、相对变化与条件数。

解 原解 $(1,0)^{\mathrm{T}}$，新解 $(1,1)^{\mathrm{T}}$。右端相对变化为 10^{-8}，解相对变化为 1，放大倍数 10^8。奇异值 1 与 10^{-8}，条件数 10^8，此例达到上界。两次解都可对各自方程产生极小残差，却不代表原始问题对输入误差不敏感。

正规方程为什么放大条件数

若 X 满列秩，其奇异值为 $\sigma_1, \ldots, \sigma_d$，则 $X^{\mathrm{T}}X$ 特征值为 $\sigma_1^2, \ldots, \sigma_d^2$，故

$$\kappa_2(X^{\mathrm{T}}X) = \kappa_2(X)^2.$$

当 $\kappa_2(X) = 10^8$ 时，Gram 矩阵条件数约 10^{16}，已接近`float64`舍入尺度的倒数。显式形成 Gram 矩阵可能丢失原先尚可辨认的小方向；再求逆不能恢复这些信息。

病态（ill-conditioned）描述问题本身对扰动敏感；**数值稳定性**描述算法怎样传播舍入误差。稳定算法不能消除问题固有的敏感性，却能避免额外放大。QR 或 SVD 一般优于直接形成正规方程，但仍应检查尺度、奇异值与残差。

特征缩放可能改善仅由单位差异造成的条件问题；它不能自动消除近重复列。把全部 A 乘同一非零标量不会改变奇异值比，而逐列缩放会改变坐标度量。第 11 章的岭回归会用正则化抑制小奇异值方向，但同时改变优化目标，不是“免费修复”原最小二乘问题。

6.10 矩阵微分：从标量输出核对梯度

微分与梯度的约定

本节只对标量函数 $f:\mathbb{R}^d\to\mathbb{R}$ 求导，梯度约定为列向量：

$$\nabla_{\mathbf{x}} f=\begin{pmatrix}\partial f/\partial x_1\\ \vdots\\ \partial f/\partial x_d\end{pmatrix},\qquad \mathrm{d}f=(\nabla_{\mathbf{x}} f)^{\mathrm{T}}\mathrm{d}\mathbf{x}.$$

$\mathrm{d}f$ 是输入微小变化的线性部分；后式乘法维度为 $(1\times d)(d\times 1)=1\times 1$。不同教材可能把导数排成行向量，因此阅读公式时先核对约定，不仅看转置符号。

若 $f(\mathbf{x})=\mathbf{a}^{\mathrm{T}}\mathbf{x}=\sum_j a_j x_j$，则 $\partial f/\partial x_j=a_j$，梯度为 $\mathbf{a}$。若 $f=\mathbf{x}^{\mathrm{T}}A\mathbf{x}=\sum_{i,j}x_iA_{ij}x_j$，对第 k 项求导有两条来源：

$$\frac{\partial f}{\partial x_k}=\sum_j A_{kj}x_j+\sum_i x_iA_{ik},$$
$$\nabla_{\mathbf{x}} f=(A+A^{\mathrm{T}})\mathbf{x}.$$

只有 A 对称时才可简化成 $2A\mathbf{x}$。特别地，$\nabla\|\mathbf{x}\|_2^2=2\mathbf{x}$，而不是误用范数本身的导数。$\|\mathbf{x}\|_2$ 在非零处梯度为 $\mathbf{x}/\|\mathbf{x}\|_2$，在零处不可微；平方范数处处可微。

平方损失的梯度保留每一步

定义 $n\ge 1$，$X:(n,d)$、$\mathbf{w}:(d,1)$、$\mathbf{y}:(n,1)$，并令

$$L(\mathbf{w})=\frac{1}{2n}\|X\mathbf{w}-\mathbf{y}\|_2^2,\quad \mathbf{e}=X\mathbf{w}-\mathbf{y}.$$

此处 $\mathbf{e}$ 为预测减真实值，与前面的残差 $\mathbf{r}$ 方向相反。按分量有

$$e_i=\sum_{j=1}^{d}X_{ij}w_j-y_i,$$
$$\frac{\partial L}{\partial w_k}=\frac{1}{2n}\sum_i 2e_i\frac{\partial e_i}{\partial w_k}=\frac{1}{n}\sum_i e_iX_{ik}.$$

因此

$$\boxed{\nabla_{\mathbf{w}}L=\frac{1}{n}X^{\mathrm{T}}(X\mathbf{w}-\mathbf{y})}. \tag{6.4}$$

外部 X^{T} 把 n 维样本误差汇总成 d 维参数导数。符号来自误差定义，系数 $1/n$ 来自所选损失归一化；若损失没有 1/2，梯度会多 2，若用误差平方和而非均值，又会多 n。

矩阵微分可再次核对：$\mathrm{d}\mathbf{e} = X\mathrm{d}\mathbf{w}$，$\mathrm{d}(\mathbf{e}^{\mathrm{T}}\mathbf{e}) = 2\mathbf{e}^{\mathrm{T}}\mathrm{d}\mathbf{e}$，于是

$$\mathrm{d}L = \frac{1}{n}\mathbf{e}^{\mathrm{T}}X\mathrm{d}\mathbf{w} = \left(\frac{1}{n}X^{\mathrm{T}}\mathbf{e}\right)^{\mathrm{T}}\mathrm{d}\mathbf{w}.$$

一次预测与梯度计算对稠密 X 约需 $O(nd)$ 运算，除输入矩阵外主要存储 n 维误差和 d 维梯度；不需要构造 $d \times d$ 的 Gram 矩阵。

与梯度定义比较即得同一结果。令梯度为 0，再次得到正规方程，几何正交条件与导数条件在这里一致。

> **例题 6.9：手算梯度与形状**
>
> 使用例题 6.5 的 X 与 y，在 $\mathbf{w} = (1/2, 1)^{\mathrm{T}}$ 计算预测、误差、损失 L 与梯度。分别说明两项梯度与哪些变化方向有关。

解 预测为 $(1/2, 3/2, 5/2)^{\mathrm{T}}$，误差为 $(-1/2, -1/2, 1/2)^{\mathrm{T}}$，平方和 $3/4$，故 $L = (3/4)/6 = 1/8$。

$$X^{\mathrm{T}}\mathbf{e} = \begin{pmatrix} -1/2 \\ 1/2 \end{pmatrix}, \qquad \nabla L = \begin{pmatrix} -1/6 \\ 1/6 \end{pmatrix}.$$

第一项对应截距变化，沿负梯度会提高截距；第二项对应斜率变化，沿负梯度会减小斜率。梯度给局部变化方向，不能只根据一项误差正负决定最终参数。第 7 章会讨论步长及下降条件。

多输出参数与 Frobenius 内积

若 $W \in \mathbb{R}^{d\times k}$、$Y \in \mathbb{R}^{n\times k}$，预测为 XW，定义

$$L(W) = \frac{1}{2n}\|XW - Y\|_F^2.$$

这里按样本数 n 归一化，未再除输出数 k。对每个输出列应用同样推导，得到

$$\nabla_W L = \frac{1}{n}X^{\mathrm{T}}(XW - Y) \in \mathbb{R}^{d\times k}.$$

梯度与参数同形状。若代码`np.mean((X @ W - Y)**2)/2`对所有 nk 个元素求平均，则相应梯度分母应为 nk；不能在正文、代码和梯度中混用两个损失定义。

矩阵微分中可用迹（trace）表示 Frobenius 内积：$\mathrm{tr}(A) = \sum_i A_{ii}$，$\langle G, \mathrm{d}W\rangle_F = \mathrm{tr}(G^{\mathrm{T}}\mathrm{d}W)$。只有乘法维度允许时才能循环移动迹中的因子，例如 $\mathrm{tr}(AB) = \mathrm{tr}(BA)$；不能从迹的等式推出 $AB = BA$。本章后续实验不要求掌握复杂迹运算，保留此入口用于阅读深度学习公式。

6.11 实验 6.1：几何条件、QR 与秩亏解

入口为配套实验`/ch06_geometry.py`，只依赖 NumPy，数据在脚本中固定，无需下载。运行：

```
python3 ch06_geometry.py
```

脚本先对例题 6.5 调用最小二乘求解，再显式计算残差和平方和；另外通过 QR 解上三角方程，与最小二乘结果比较；构造 $P = QQ^{\mathrm{T}}$，检查对称、幂等及 $P\mathbf{y} = X\mathbf{w}$。所有这些核对回答的是不同问题，单看参数打印值不足以验证投影关系。

```
X = np.array([[1., 0.], [1., 1.], [1., 2.]])
y = np.array([1., 2., 2.])
w, residuals, rank, singular = np.linalg.lstsq(X, y, rcond=None)
r = y - X @ w
Q, R = np.linalg.qr(X, mode="reduced")
w_qr = np.linalg.solve(R, Q.T @ y)
P = Q @ Q.T
np.testing.assert_allclose(X.T @ r, 0, atol=1e-12)
np.testing.assert_allclose(w_qr, w, atol=1e-12)
```

NumPy 的`y,w,r`在这里用一维数组实现数学列向量，形状为`(3,)`、`(2,)`、`(3,)`；结果约定要与第 4 章一致。若任意把 y 改为`(3,1)`，再减一维预测，会广播成错误的 3 乘 3 矩阵。

预期参数 $(7/6, 1/2)$，预测 $(7/6, 5/3, 13/6)$，残差 $(-1/6, 1/3, -1/6)$，平方和 $1/6$，数值秩 2；$X^{\mathrm{T}}\mathbf{r}$ 两项约为 0。数值比较允许 10^{-12} 量级绝对误差，符号或最后几位舍入可变化。

随后复制第二列，求解三参数问题。预期最小范数参数为 $(7/6, 1/4, 1/4)$，数值秩仍 2；残差返回数组为空，但显式残差平方和仍 $1/6$。再加零空间向量 $(0, 1, -1)$，得到 $(7/6, 5/4, -3/4)$，预测不变、参数范数增加。原最小范数约 1.219062，新范数约 1.867113。

修改任务：只把最后目标 2 改 3，三个点变为直线 $y = 1 + x$，应得参数 $(1, 1)$、误差接近 0；去掉常数列，手算单参数斜率 $\sum_i x_i y_i / \sum_i x_i^2 = 6/5$，残差和不再为 0；给重复列换一个非零比例，解释为何零空间方向随比例变化；不用 P 直接计算`Q @ (Q.T @ y)`，说明避免了哪些存储。

完整 P 含 n^2 元素，大数据中不应仅为做预测而显式构造它。$Q(Q^{\mathrm{T}}\mathbf{y})$ 只需分两次乘法，仍得到同一投影。数学上存在的矩阵表达，不一定是合适的程序存储结构。

6.12 实验 6.2：SVD、敏感性与梯度核对

主入口为配套实验`/ch06_svd.py`，只用 NumPy。

答案核对入口为`ch06_solutions.py`。

```
python3 ch06_svd.py
python3 ch06_solutions.py
```

先对例题 6.7 的矩阵求经济 SVD，检查重建与秩一近似误差；再运行例题 6.8 的扰动，记录 A 与 Gram 矩阵条件数；最后对例题 6.9 的损失核对解析梯度与数值梯度。

有限差分通过很小的输入变化近似导数。对参数第 j 项，中心差分为

$$g_j^{\mathrm{FD}} = \frac{L(\mathbf{w} + h\mathbf{e}_j) - L(\mathbf{w} - h\mathbf{e}_j)}{2h}.$$

本实验用 $h = 10^{-6}$。它是核对公式的数值办法，不是每次训练都逐参数调用两遍损失的高效训练方法。步长太大引入近似误差，太小会出现相近数相减的舍入影响，第 7 章将进一步分析。

```
h = 1e-6
finite = np.empty_like(w)
for j in range(w.size):
    step = np.zeros_like(w)
    step[j] = h
    plus = loss_and_gradient(X, y, w + step)[0]
    minus = loss_and_gradient(X, y, w - step)[0]
    finite[j] = (plus - minus) / (2 * h)
```

本框的 w 为例题 6.9 的`np.array([0.5, 1.])`，`loss_and_gradient`由配套脚本定义。接口先验证实数类型、有限性、非空与形状，再转换成独立`float64`数组，拒绝布尔、复数、字符串以及不等形标签；计算溢出时主动失败，不把无穷损失当作有效结果。

固定预期为：奇异值 3、1；秩一 Frobenius 误差 1；A 条件数 10^8、Gram 条件数约 10^{16}；解从 $(1,0)$ 变为 $(1,1)$；损失 0.125，解析梯度 $(-1/6, 1/6)$。梯度采用相对容差 10^{-7}、绝对容差 10^{-9}；差分最后几位与数值库有关，不能规定完全相等。

输出存于`ch06_results/summary.json`与`svd.npz`，记录环境、奇异值、条件数、梯度误差与分解数组。加载 NPZ 时采用`allow_pickle=False`，沿用第 4 章普通数值数组的保存规则。

修改任务：将 10^{-8} 改成 10^{-4} 并将右端扰动相应改成 10^{-4}，条件数应为 10^4，同样方向的解变化仍为 1；把保留秩从 1 改 2，重建误差应接近 0；给 U 某列和 V 转置对应行同时乘 -1，重建应不变；对 $h = 10^{-2}, 10^{-4}, 10^{-6}, 10^{-8}$ 逐一记录梯度误差。此处损失是二次函数，中心差分的截断误差有特殊抵消，不能据此推广任意函数的最佳步长。

6.13 公式维度表、常见错误与自检

表达式	数学形状	解释或条件
$X\mathbf{w}$	$n \times 1$	n 个预测
$X^{\mathrm{T}}\mathbf{e}$	$d \times 1$	参数方向的误差汇总
$X^{\mathrm{T}}X$	$d \times d$	特征列的 Gram 矩阵
XX^{T}	$n \times n$	样本行的 Gram 矩阵
$Q^{\mathrm{T}}\mathbf{y}$	$r \times 1$	标准正交列上的坐标
$QQ^{\mathrm{T}}\mathbf{y}$	$n \times 1$	r 维子空间中的投影
$A^{+}\mathbf{b}$	$d \times 1$	$A : (m, d)$, $\mathbf{b} : (m, 1)$
$X^{\mathrm{T}}(XW - Y)$	$d \times k$	多输出参数梯度的未归一化部分

维度相符只能说明乘法有定义，不能证明公式正确。$X^{\mathrm{T}}X$ 与 XX^{T} 都可能是方阵，但分别处于参数空间与样本空间；当 $n = d$ 时形状甚至相同，含义仍不同。核对公式要同时说明每个轴代表样本、特征还是输出。

常见误区包括：把相似度当距离；对零向量算余弦；把平方距离当满足三角不等式的距离；用元素全正判断正定；把满行秩当参数唯一；对长方形矩阵直接求普通逆；认为秩亏就没有最小二乘解；把空残差返回数组理解为零误差；把 $Q^{\mathrm{T}}Q = I$ 误反写；把特征分解套到任意实矩阵；直接显式求逆并忽略条件数。

知识联系可以沿一条链理解：字段确定坐标，内积定义方向比较，列空间规定预测能到达哪里，投影确定最佳拟合，秩与零空间确定参数是否唯一，SVD 刻画可辨认方向，条件数刻画求解敏感性，矩阵微分则把同一问题转换成可迭代优化的梯度公式。

自检应能说明为什么 X 的列空间在 $\mathbb{R}^n$ 而非 $\mathbb{R}^d$；为什么残差正交于列而不必为零；为什么预测唯一不能推出系数唯一；为什么小奇异值在反向求解中危险；为什么数值稳定与统计泛化是不同问题。

6.14 分层习题

以下编号为 6.1–6.14；默认实数、列向量与普通欧氏度量，数值实现注明数组形状。

6.1 **基础**。计算例题 6.1 的内积、范数、距离与余弦；若 b 改成 $-2\mathbf{a}$，余弦与欧氏距离分别是什么？

6.2 **推导**。展开两个单位向量的平方距离，证明它与余弦的关系；用 0、1、2 说明平方距离不满足三角不等式。

6.3 **坐标**。求 $(3,2)^{\mathrm{T}}$ 在 $(1,1)^{\mathrm{T}}$ 与 $(1,-1)^{\mathrm{T}}$ 下的坐标。$A\mathbf{x}+\mathbf{b}$ 什么时候是严格线性变换？

6.4 **秩**。完成例题 6.3；解释满列秩、满行秩分别对方程的可解性与唯一性意味着什么。

6.5 **投影**。完成例题 6.4，直接核对 $P^2 = P$ 与 $P^{\mathrm{T}} = P$；为什么 $P \neq I_3$？

6.6 **最小二乘**。完整重算例题 6.5。若去掉截距，求斜率、残差与残差和；误差平方和是多少？

6.7 **不唯一**。在重复特征问题中列出全部最优参数，证明 $(7/6, 1/4, 1/4)$ 是最小范数解。

6.8 **正定性**。求例题 6.6 两个矩阵的特征值；说明为什么 $X^{\mathrm{T}}X$ 总半正定、何时严格正定。

6.9 **SVD**。给出例题 6.7 矩阵的经济 SVD 形状、伪逆、秩一近似与误差。奇异向量同时变号为什么不影响重建？

6.10 **条件数**。将例题 6.8 中 10^{-8} 改为 10^{-4} 并相应改右端扰动，计算放大倍数与 Gram 条件数。为什么残差小不说明系数可靠？

6.11 **微分**。从分量求导证明 $\nabla(\mathbf{x}^{\mathrm{T}}A\mathbf{x}) = (A + A^{\mathrm{T}})\mathbf{x}$，说明何时可写成 $2A\mathbf{x}$。

6.12 **梯度**。重算例题 6.9，再把损失改为 $\frac{1}{n}\|X\mathbf{w} - \mathbf{y}\|^2$。梯度如何变化？多输出时若按全部元素平均，分母如何变化？

6.13 **编程**。运行两个实验与答案核对；改变最后目标为 3，核对参数 1、1 与正交条件；

对 SVD 比较重建而非固定奇异向量符号。

6.14 综合。某数据矩阵有两列几乎相同。解释参数不稳定、训练预测变化小、测试预测可能变化大怎样同时发生，并提出需记录的诊断量。

6.15 习题参考答案

6.1–6.5：度量、秩与投影

6.1 原题内积 2，a 范数 3、$\sqrt{5}$、2，b 范数均 2，欧氏距离 $\sqrt{5}$，余弦 $1/\sqrt{5}$。b 改为 $-2\mathbf{a}$ 时方向相反，余弦 -1，差向量为 $3\mathbf{a}$，距离 $3\sqrt{5}$。

6.2 $\|\mathbf{u}-\mathbf{v}\|^2 = \|\mathbf{u}\|^2 + \|\mathbf{v}\|^2 - 2\mathbf{u}^{\mathrm{T}}\mathbf{v} = 2 - 2\cos\theta$。单位化须排除零向量。0 到 2 平方距离 4，大于 0 到 1 与 1 到 2 之和 2，因此不满足距离公理中的三角不等式。

6.3 解 $\alpha+\beta=3, \alpha-\beta=2$，得 $\alpha=5/2, \beta=1/2$。严格线性要求映射零向量到零，故仿射式中的固定 b 必须为零。增广一个常数坐标可以用矩阵表示仿射式，但不改变原空间中它含平移的事实。

6.4 秩 2，零空间由 $(0,1,-1)$ 张成。满列秩保证在可解时唯一，但不保证任意右端可解；满行秩保证任意右端可解，但若列数多于行数就有非零零空间，解不唯一。方阵满秩才同时对任意右端可解且唯一。

6.5 投影 $(1,1,0)$、残差 $(1,-1,3)$；P 的左上 2 乘 2 块全为 1/2，其余 0。平方后该块每项为 $1/4+1/4=1/2$，P 对称且幂等。$P(0,0,1)^{\mathrm{T}}=0$，故不等于 I_3；它只保留一维方向。

6.6–6.10：拟合、分解与敏感性

6.6 带截距参数 $(7/6, 1/2)$，残差 $(-1/6, 1/3, -1/6)$，平方和 1/6。无截距时 $a=6/5$，预测 $(0, 6/5, 12/5)$，残差 $(1, 4/5, -2/5)$，残差和 $7/5$，平方和 $1+16/25+4/25=9/5$。输入与残差内积仍为 0，但常数方向未包含在设计矩阵中。

6.7 全部最优参数为 $(7/6, t, 1/2-t)$。参数范数平方为 $(7/6)^2 + 2(t-1/4)^2 + 1/8$，唯一最小点 $t=1/4$。最小范数选择不增加数据中不存在的辨识信息。

6.8 第一矩阵特征值 3、1，正定；第二矩阵 3、-1，不定。对任意 v，$\mathbf{v}^{\mathrm{T}}X^{\mathrm{T}}X\mathbf{v} = \|X\mathbf{v}\|^2 \geq 0$。严格正定当且仅当 X 没有非零零空间，即满列秩；若存在非零 v 被消去，则二次型可为零。

6.9 U 形状 3 乘 2，s 长 2，V 转置为 2 乘 2；奇异值 3、1，伪逆主非零项为 1/3、1。秩一只保留第一项 3，Frobenius 误差 1。外积中两向量同时变号，乘积 $(-\mathbf{u})(-\mathbf{v})^{\mathrm{T}} = \mathbf{u}\mathbf{v}^{\mathrm{T}}$，所以重建不变。

6.10 A 条件数 10^4，Gram 条件数 10^8，右端相对变化 10^{-4}，解相对变化 1，放大倍数 10^4。残差衡量当前计算解满足当前输入方程的程度，不能说明输入本身没有误差，也不能排除误差在弱方向上的放大。

6.11–6.14：微分、程序与综合判断

6.11 分量和 $\sum_{i,j} x_i A_{ij} x_j$ 中，求第 k 项导数需分别考虑左侧 x_i 与右侧 x_j；得到 $\sum_j A_{kj} x_j + \sum_i A_{ik} x_i$。向量化为 $(A + A^{\mathrm{T}})\mathbf{x}$；A 对称时化为 $2A\mathbf{x}$。反对称部分对二次型没有贡献。

6.12 原损失 1/8、梯度 $(-1/6, 1/6)$。去掉 1/2 后损失 1/4、梯度 $(-1/3, 1/3)$。多输出按 nk 个元素平均的半平方损失梯度为 $X^{\mathrm{T}}(XW - Y)/(nk)$，梯度形状仍与 W 一致。

6.13 改后 $\mathbf{y} = (1, 2, 3)$ 恰在 X 列空间中，参数 $(1, 1)$，残差数值上约为 0。仍应检查形状与有限性，并用容差比较。SVD 核对 $U\Sigma V^{\mathrm{T}}$ 与输入矩阵一致，U/V 对应列行符号不作固定要求。配套脚本额外拒绝列形标签、非数值、非有限与溢出输入。

6.14 近相关列产生小奇异值。参数沿相应右奇异方向变化时，被训练 X 强烈压缩，训练预测可能变化不大；新样本未必保持同样相关结构，参数变化就可能显现为大预测变化。应记录特征单位与缩放、奇异值谱、数值秩及阈值、条件数、残差范数、参数范数与分布变化；不能仅凭训练误差小判断可靠。后续可比较简化特征或正则化，但需遵守验证流程。

6.16 本章小结

线性代数把数值计算组织成方向、空间与变换。内积连接夹角与得分，范数连接大小与误差，列空间限制可实现的预测，正交投影给出平方误差下的最佳近似。秩与零空间解释参数的可辨认性，不能与拟合误差或方程数量简单等同。

特征分解揭示对称二次型的方向，SVD 适用于一般长方形矩阵，并统一解释秩、伪逆、低秩近似和敏感性。矩阵微分则把这些结构转换成参数梯度。后续学习算法时，仍应同时核对公式条件、数组形状、数值尺度与数据含义。

资料说明与配套实验索引

本累计版包含第 1 章《人工智能与机器学习全景》、第 2 章《Python 语言基础》、第 3 章《Python 程序组织与调试》、第 4 章《NumPy 与向量化计算》、第 5 章《pandas、数据探索与可视化》和第 6 章《线性代数复习》。后续 8 篇 44 章的整体规划保存在同目录大纲中，未完成章节不列入本 PDF 正文目录。已完成章节的编号、术语与实验编号供后续章节沿用。

本讲义中的例题、任务卡、教学数据与代码为针对本书学习目标编写的示例。基础语法可对照 Python 官方教程核对，可靠评价的流程可对照机器学习工具的官方实践说明。参考入口如下，访问日期为 2026 年 10 月 8 日；网页版本可能随官方维护更新，本章示例采用常见 Python 3 语法。

1. Python 官方教程：基础表达式与字符串。
   `https://docs.python.org/3/tutorial/introduction.html`
2. Python 官方教程：控制流程与函数。
   `https://docs.python.org/3/tutorial/controlflow.html`
3. Python 官方教程：数据结构。
   `https://docs.python.org/3/tutorial/datastructures.html`
4. Python 官方教程：虚拟环境与包。
   `https://docs.python.org/3/tutorial/venv.html`
5. Python 官方教程：浮点运算及其限制。
   `https://docs.python.org/3/tutorial/floatingpoint.html`
6. scikit-learn 官方文档：常见问题与实践建议，特别是数据泄漏和预处理的一致性。
   `https://scikit-learn.org/stable/common_pitfalls.html`

第 3、4 章另核对下列官方资料，访问日期为 2026 年 10 月 10 日。代码实际运行环境为 Python 3.12.14、NumPy 2.3.5；Python 通用网页可能指向官方当前版本，本文仅使用已在配套环境核对的接口。正文中的数据、例题和推导均为本书教学编写，不以官方文档的示例替代本章实验。

1. Python 官方教程：模块、异常与类。
   `https://docs.python.org/3.12/tutorial/modules.html`
   `https://docs.python.org/3/tutorial/errors.html`
   `https://docs.python.org/3/tutorial/classes.html`
2. Python 标准库：路径、CSV、JSON 与单元测试。
   `https://docs.python.org/3/library/pathlib.html`
   `https://docs.python.org/3/library/csv.html`
   `https://docs.python.org/3/library/json.html`
   `https://docs.python.org/3.12/library/unittest.html`
3. NumPy 2.3 官方文档：基础数组、广播、复制与视图。

https://numpy.org/doc/2.3/user/absolute_beginners.html
https://numpy.org/doc/2.3/user/basics.broadcasting.html
https://numpy.org/doc/2.3/user/basics.copies.html

4. NumPy 2.3 官方参考：矩阵乘法、随机生成器、标准差与数组保存。
https://numpy.org/doc/2.3/reference/generated/numpy.matmul.html
https://numpy.org/doc/2.3/reference/random/generator.html
https://numpy.org/doc/2.3/reference/generated/numpy.std.html
https://numpy.org/doc/2.3/reference/generated/numpy.savez.html

第 5、6 章另核对 pandas、Matplotlib 与 NumPy 线性代数官方文档，访问日期为 2026 年 10 月 11 日。pandas 参考固定在 2.2 系列，本批实际使用 2.2.3；Matplotlib 参考为当前稳定版；分位数、缺失与合并规则均在正文明确说明。

1. pandas：索引、缺失、分组与合并。
https://pandas.pydata.org/pandas-docs/version/2.2/user_guide/indexing.html
https://pandas.pydata.org/pandas-docs/version/2.2/user_guide/missing_data.html
https://pandas.pydata.org/pandas-docs/version/2.2/user_guide/groupby.html
https://pandas.pydata.org/pandas-docs/version/2.2/user_guide/merging.html
2. Matplotlib：画布、绘图区与基本作图。
https://matplotlib.org/stable/users/explain/quick_start.html
3. NumPy 2.3：最小二乘、SVD 与条件数。
https://numpy.org/doc/2.3/reference/generated/numpy.linalg.lstsq.html
https://numpy.org/doc/2.3/reference/generated/numpy.linalg.svd.html
https://numpy.org/doc/2.3/reference/generated/numpy.linalg.cond.html

上列资料为相关机制的核对与进阶入口，不要求先通读才能运行本批实验。第一章的近邻、风险与评价指标将在相应后续章节补充完整算法、统计假设和推导。

入口文件	对应位置	内容
ch01_regression.py	实验 1.1	基线、近邻选择、最终测试
ch01_classification.py	实验 1.2	阈值、四类计数与指标
ch02_basics.py	实验 2.1	文本清理、拒绝记录、统计
ch02_metrics.py	实验 2.2	MAE、MSE、RMSE 与错误输入
ch02_solutions.py	习题 2.5–2.16	可运行的编程答案与核对
ch03_objects.py	实验 3.1	引用、复制、文件往返
ch03_project/main.py	实验 3.2	多模块预测、评价与保存
ch03_solutions.py	第 3 章习题	数值验证、引用与绑定核对
ch04_arrays.py	实验 4.1	形状、视图、广播与稳定计算
ch04_numeric.py	实验 4.2	训练缩放、批量近邻与计时
ch04_solutions.py	第 4 章习题	形状、手算、分块与边界核对
ch05_tables.py	实验 5.1	类型、审计、分组与合并
ch05_eda.py	实验 5.2	统计、多图与探索报告
ch05_solutions.py	第 5 章习题	手算、冲突与合并核对
ch06_geometry.py	实验 6.1	投影、QR 与秩亏拟合
ch06_svd.py	实验 6.2	SVD、条件数与梯度
ch06_solutions.py	第 6 章习题	手算与非法输入核对
README.md	实验说明	环境、运行、预期与修改任务

实验输出记录另保存在“配套实验/预期输出”目录。主源文件为“源文件/机器学习讲义.tex”，可使用 XeLaTeX 继续编译与增补。源码集中定义版式与颜色，图示在文内生成，不需要另附图片才能编译。